

JAVA 17 OCP · YDS STUDY SERIES

Unit 13 · Concurrency · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) içindeki ilgili chapter gövdesini ve Appendix resmî cevaplarını kaynak sırasını koruyan English → Türkçe paragraf çiftleriyle bir araya getirir. Kod ve terminal çıktıları çevrilmeden, bir kez ve kaynak konumunda gösterilir.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak ve kapsam özeti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 13 · Concurrency
- Chapter PDF sayfaları: 721–784
- Appendix cevap sayfaları: 951–955
- Beklenen kaynak sayfa sayısı: 64
- Beklenen resmî cevap: 25
- Eşleme biçimi: English paragraf → Türkçe çeviri → varsa kod

İçindekiler

- 1 [Introducing Threads](#)
- 2 [Creating Threads with the Concurrency API](#)
- 3 [Writing Thread-Safe Code](#)
- 4 [Using Concurrent Collections](#)
- 5 [Identifying Threading Problems](#)
- 6 [Working with Parallel Streams](#)
- 7 [Summary](#)
- 8 [Exam Essentials](#)
- 9 [Review Questions](#)
- 10 [Official Review Question Answers / Resmî Cevaplar](#)

Chapter 13 · Concurrency · Eksiksiz çift dilli kaynak

Chapter 13 · Concurrency

English: OCP exam objectives covered in this chapter: Managing concurrent code execution and working with Streams and lambda expressions.

Türkçe: Bu bölümde ele alınan OCP sınav hedefleri: Concurrent code execution'ı yönetmek ve Streams ile lambda ifadeleriyle çalışmak.

English: Create worker threads with Runnable and Callable, and manage the thread life cycle using Executor services and the Concurrency API.

Türkçe: Runnable ve Callable ile worker thread'ler oluşturun; executor service'leri ve Concurrency API kullanarak thread yaşam döngüsünü yönetin.

English: Develop thread-safe code using locking mechanisms and concurrent APIs, and process Java collections concurrently with parallel streams.

Türkçe: Locking mechanism'ları ve concurrent API'leri kullanarak thread-safe kod geliştirin; Java collection'larını parallel stream'lerle eşzamanlı olarak işleyin.

English: Perform decomposition, concatenation, reduction, grouping, and partitioning on sequential and parallel streams.

Türkçe: Sequential ve parallel streams üzerinde decomposition, concatenation, reduction, grouping ve partitioning işlemlerini gerçekleştirin.

English: As you will learn in Chapter 14, “I/O,” and Chapter 15, “JDBC,” computers are capable of reading and writing data to external resources. Unfortunately, as compared to CPU operations, these disk/network operations tend to be extremely slow—so slow, in fact, that if your computer’s operating system were to stop and wait for every disk or network operation to finish, your computer would appear to freeze constantly.

Türkçe: Bölüm 14, “I/O” ve Bölüm 15, “JDBC” konularında öğreneceğiniz gibi bilgisayarlar haricî kaynaklardan veri okuyabilir ve bu kaynaklara veri yazabilir. Ne yazık ki disk/network işlemleri CPU işlemlerine kıyasla son derece yavaş olma eğilimindedir. İşletim sistemi her disk veya network işleminin tamamlanmasını bekleyerek dursaydı bilgisayar sürekli donuyormuş gibi görünürdü.

English: Luckily, all operating systems support what is known as multithreaded processing. The idea behind multithreaded processing is to allow an application or group of applications to execute multiple tasks at the same time. This allows tasks waiting for other resources to give way to other processing requests.

Türkçe: Neyse ki bütün işletim sistemleri multithreaded processing'i destekler. Bunun temel amacı, bir uygulamanın veya uygulama grubunun aynı anda birden fazla task yürütmesine olanak vermektir. Böylece başka kaynakları bekleyen task'lar, diğer işlem isteklerine çalışma fırsatı bırakabilir.

English: In this chapter, we introduce you to the concept of threads and provide numerous ways to manage threads using the Concurrency API. Threads and concurrency are challenging topics for many programmers to grasp, as problems with threads can be frustrating even for veteran developers. In practice, concurrency issues are among the most difficult problems to diagnose and resolve.

Türkçe: Bu bölümde thread kavramını tanıtıyor ve thread'leri Concurrency API ile yönetmenin çeşitli yollarını gösteriyoruz. Thread'ler ve concurrency, birçok programcının kavramakta zorlandığı konulardır; thread sorunları deneyimli geliştiriciler için bile yorucu olabilir. Uygulamada concurrency sorunları, teşhis edilmesi ve çözülmesi en zor sorunlar arasındadır.

Introducing Threads

English: We begin this chapter by reviewing common terminology associated with threads. A thread is the smallest unit of execution that can be scheduled by the operating system. A process is a group of associated threads that execute in the same shared environment. It follows, then, that a single-threaded process is one that contains exactly one thread, whereas a multithreaded process supports more than one thread.

Türkçe: Bölüme, thread'lerle ilişkili yaygın terminolojiyi gözden geçirerek başlıyoruz. Thread, işletim sistemi tarafından schedule edilebilen en küçük execution unit'tir. Process, aynı shared environment içinde çalışan ilişkili thread'ler grubudur. Buna göre single-threaded process tam olarak bir thread içerirken multithreaded process birden fazla thread'i destekler.

English: By shared environment, we mean that the threads in the same process share the same memory space and can communicate directly with one another. Refer to Figure 13.1 for an overview of threads and their shared environment within a process.

Türkçe: Shared environment ile aynı process'teki thread'lerin aynı memory space'i paylaşmasını ve birbirleriyle doğrudan iletişim kurabilmesini kastediyoruz. Bir process içindeki thread'lere ve paylaştıkları ortama genel bakış için Şekil 13.1'e bakın.

English: This figure shows a single process with three threads. It also shows how they are mapped to an arbitrary number of n CPUs available within the system. Keep this diagram in mind when we discuss task schedulers later in this section.

Türkçe: Bu şekil, üç thread içeren tek bir process'i gösterir. Ayrıca thread'lerin sistemde bulunan rastgele sayıdaki n CPU'ya nasıl eşlendiğini de gösterir. Bu bölümün ilerleyen kısmında task scheduler'ları ele alırken bu diyagramı aklınızda tutun.

English: In this chapter, we talk a lot about tasks and their relationships to threads. A task is a single unit of work performed by a thread. Throughout this chapter, a task will commonly be implemented as a lambda expression. A thread can complete multiple independent tasks but only one task at a time.

Türkçe: Bu bölümde task'lar ve bunların thread'lerle ilişkileri üzerinde sıkça duracağız. Task, bir thread tarafından gerçekleştirilen tek bir çalışma birimidir ve bu bölüm boyunca çoğunlukla lambda expression olarak uygulanacaktır. Bir thread birden fazla bağımsız task'ı tamamlayabilir, ancak aynı anda yalnızca bir task çalıştırabilir.

English: FIGURE 13.1 — Process model: a process (Java program) contains shared memory and multiple threads; the OS thread scheduler maps those threads to CPU 1 through CPU n .

Türkçe: **Şekil 13.1 — Process modeli:** Bir process (Java programı), shared memory ile birden fazla thread içerir; OS thread scheduler bu thread'leri CPU 1'den CPU n 'e kadar kullanılabilir işlemcilere eşler.

English: By shared memory in Figure 13.1, we are generally referring to static variables as well as instance and local variables passed to a thread. Yes, you finally see how static variables can be useful for performing complex, multithreaded tasks! Remember from Chapter 5, “Methods,” that static methods and variables are defined on a single class object that all instances share. For example, if one thread updates the value of a static object, this information is immediately available for other threads within the process to read.

Türkçe: Şekildeki shared memory; `static` alanların yanı sıra thread'e aktarılan instance ve local variable referansları üzerinden erişilen ortak verileri kapsar. Bölüm 5 “Methods” konusundan hatırlanacağı üzere `static` member'lar bütün instance'ların paylaştığı tek `Class` nesnesiyle ilişkilidir. Kaynak metne göre bir thread `static` bir object'in değerini güncellerse bu bilgi aynı process'teki diğer thread'lerin okuması için hemen kullanılabilir.

Java 17 editör notu: Bir nesnenin shared olması, güncelleme için diğer thread'lere kendiliğinden ve hemen görünmesini garanti etmez. Görünürlük için `volatile`, `synchronized` veya başka bir happens-before ilişkisi gerekir.

Understanding Thread Concurrency

English: The property of executing multiple threads and processes at the same time is referred to as concurrency. How does the system decide what to execute when there are more threads available than CPUs? Operating systems use a thread scheduler to determine which threads should be currently executing, as shown in Figure 13.1. For example, a thread scheduler may employ a round-robin schedule in which each available thread receives an equal number of CPU cycles with which to execute, with threads visited in a circular order.

Türkçe: Birden fazla thread ve process'in aynı zamanda yürütülmesi özelliğine concurrency denir. Kullanılabilir thread sayısı CPU sayısından fazlaysa sistem hangisini çalıştıracağına nasıl karar verir? İşletim sistemleri, Şekil 13.1'de gösterildiği gibi o anda hangi thread'lerin çalışacağını belirlemek için thread scheduler kullanır. Örneğin scheduler, kullanılabilir her thread'e eşit sayıda CPU cycle veren ve thread'leri döngüsel sırayla ele alan round-robin scheduling uygulayabilir.

English: When a thread's allotted time is complete but the thread has not finished processing, a context switch occurs. A context switch is the process of storing a thread's current state and later restoring the state of the thread to continue execution. Be aware that a cost is often associated with a context switch due to lost time and having to reload a thread's state. Intelligent thread schedulers do their best to minimize the number of context switches while keeping an application running smoothly.

Türkçe: Bir thread'e ayrılan süre dolduğu hâlde thread işlemini tamamlamamışsa context switch gerçekleşir. Context switch, thread'in mevcut state'ini saklayıp yürütmeye daha sonra devam edebilmek için bu state'i geri yükleme işlemidir. Kaybedilen zaman ve state'i yeniden yükleme gereği nedeniyle context switch'in bir maliyeti vardır. Akıllı thread scheduler'lar, uygulamanın akıcı çalışmasını korurken context switch sayısını en aza indirmeye çalışır.

English: Finally, a thread can interrupt or supersede another thread if it has a higher thread priority than the other thread. A thread priority is a numeric value associated with a thread that is taken into consideration by the thread scheduler when determining which threads should currently be executing. In Java, thread priorities are specified as integer values.

Türkçe: Son olarak daha yüksek thread priority, scheduler'ın bir thread'i diğerinden önce çalıştırmayı tercih etmesine yol açabilir. Priority, scheduling kararında dikkate alınabilen integer bir ipucudur; execution order'ı garanti etmez ve `Thread.interrupt()` çağrısıyla aynı şey değildir.

Creating a Thread

English: One of the most common ways to define a task for a thread is by using the Runnable instance. Runnable is a functional interface that takes no arguments and returns no data.

Türkçe: Bir thread için task tanımlamanın en yaygın yollarından biri Runnable instance kullanmaktır. Runnable, argüman almayan ve veri döndürmeyen bir functional interface'dir.

```
@FunctionalInterface public interface Runnable {  
    void run();  
}
```

English: With this, it's easy to create and start a thread. In fact, you can do so in one line of code using the Thread class:

Türkçe: Bu sayede bir thread oluşturup başlatmak kolaydır. Hatta Thread sınıfını kullanarak bunu tek bir kod satırında yapabilirsiniz:

```
new Thread(() -> System.out.print("Hello")).start();  
System.out.print("World");
```

English: The first line creates a new Thread object and then starts it with the start() method. Does this code print HelloWorld or WorldHello? The answer is that we don't know. Depending on the thread priority/scheduler, either is possible. Remember that order of thread execution is not often guaranteed. The exam commonly presents questions in which multiple tasks are started at the same time, and you must determine the result.

Türkçe: İlk satır yeni bir Thread object oluşturur ve ardından start() method ile başlatır. Kod HelloWorld mü, yoksa WorldHello mu yazdırır? Bunu önceden bilemeyiz; thread priority/scheduler'a bağlı olarak iki çıktı da mümkündür. Thread execution order'ın genellikle garanti edilmediğini unutmayın. Sınavda birden fazla task'ın aynı anda başlatıldığı ve sonucu belirlemeniz gereken sorular sıkça görülür.

English: Let's take a look at a more complex example:

Türkçe: Daha karmaşık bir örneğe bakalım:

```
Runnable printInventory = () -> System.out.println("Printing zoo inventory");
Runnable printRecords = () -> {
    for (int i = 0; i < 3; i++)
        System.out.println("Printing record: " + i);
};
```

English: Given these instances, what is the output of the following?

Türkçe: Bu instance'lar kullanıldığında aşağıdaki kodun çıktısı nedir?

```
System.out.println("begin");
new Thread(printInventory).start();
new Thread(printRecords).start();
new Thread(printInventory).start();
System.out.println("end");
```

English: The answer is that it is unknown until runtime. The following is just one possible output:

Türkçe: Cevap runtime'a kadar bilinemez. Aşağıdaki yalnızca olası çıktılarından biridir:

English: begin Printing record: 0 Printing zoo inventory end Printing record: 1 Printing zoo inventory
Printing record: 2 This sample uses a total of four threads: the main() user thread and three additional
threads created on lines 4–6. Each thread created on these lines is executed as an

Türkçe: begin Printing record: 0 Printing zoo inventory end Printing record: 1 Printing zoo inventory
Printing record: 2 Bu örnek toplam dört thread kullanır: main() user thread ve 4–6. satırlarda oluşturulan üç
ek thread. Bu satırlarda oluşturulan her thread bir

English: asynchronous task. By asynchronous, we mean that the thread executing the main() method
does not wait for the results of each newly created thread before continuing. For example, lines 5 and 6
may be executed before the thread created on line 4 finishes. The opposite of this behavior is a
synchronous task in which the program waits (or blocks) on line 4 for the thread to finish executing before
moving on to the next line. The vast majority of method calls used in this book have been synchronous up
until this chapter.

Türkçe: asynchronous task olarak yürütülür. Asynchronous ile main() method'u çalıştıran thread'in,
devam etmeden önce yeni oluşturulan her thread'in sonucunu beklememesini kastediyoruz. Örneğin 5. ve
6. satırlar, 4. satırda oluşturulan thread tamamlanmadan çalıştırılabilir. Bunun tersi synchronous task'tır:
program, sonraki satıra geçmeden önce 4. satırdaki thread'in çalışmasını tamamlamasını bekler, yani
block olur. Bu bölüme gelene kadar kitapta kullanılan method call'ların büyük çoğunluğu synchronous idi.

English: While the order of thread execution is indeterminate once the threads have been started, the
order within a single thread is still linear. In particular, the for() loop is still ordered. Also, begin always
appears before end.

Türkçe: Thread'ler başlatıldıktan sonra thread execution order belirsiz olsa da tek bir thread içindeki sıra
yine doğrusaldır. Özellikle for() loop kendi sırasını korur. Ayrıca begin her zaman end'den önce görünür.

English: Calling run() Instead of start()

Türkçe: start() Yerine run() Çağırarak

English: On the exam, be mindful of code that attempts to start a thread by calling run() instead of start(). Calling run() on a Thread or a Runnable does not start a new thread. While the following code snippets will compile, none will execute a task on a separate thread:

Türkçe: Sınavda `start()` yerine `run()` çağırarak thread başlatmaya çalışan kodlara dikkat edin. Bir `Thread` veya `Runnable` üzerinde `run()` çağırarak yeni bir thread başlatmaz. Aşağıdaki code snippet'lerin tümü derlenir (`Does compile`), ancak hiçbirisi task'ı ayrı bir thread üzerinde çalıştırmaz:

```
System.out.println("begin");
new Thread(printInventory).run();
new Thread(printRecords).run();
new Thread(printInventory).run();
System.out.println("end");
```

English: Unlike the previous example, each line of this code will wait until the run() method is complete before moving on to the next line. Also unlike the previous program, the output for this code sample will be the same every time it is executed.

Türkçe: Önceki örnekten farklı olarak bu kodun her satırı, sonraki satıra geçmeden önce run() method'un tamamlanmasını bekler. Önceki programın aksine bu code sample her çalıştırıldığında aynı çıktıyı üretir.

English: More generally, we can create a Thread and its associated task one of two ways in Java: • Provide a Runnable object or lambda expression to the Thread constructor. • Create a class that extends Thread and overrides the run() method.

Türkçe: Daha genel olarak Java'da bir Thread ve ilişkili task iki yoldan biriyle oluşturulabilir: Thread constructor'a bir Runnable object veya lambda expression vermek ya da Thread'i extends eden ve run() method'u override eden bir class oluşturmak.

English: Throughout this book, we prefer creating tasks with lambda expressions. After all, it's a lot easier, especially when we get to the Concurrency API! Creating a class that extends Thread is relatively uncommon and should only be done under certain circumstances, such as if you need to overwrite other thread methods.

Türkçe: Bu kitap boyunca task'ları lambda expression'larla oluşturmayı tercih ediyoruz; özellikle `Concurrency API` söz konusu olduğunda bu yaklaşım çok daha kolaydır. `Thread`'i extend eden bir class oluşturmak görece nadirdir ve yalnızca başka thread metotlarını override etmeniz gereken belirli durumlarda kullanılmalıdır.

Distinguishing Thread Types

English: It might surprise you that all Java applications, including all of the ones that we have presented in this book, are multithreaded because they include system threads. A system thread is created by the Java Virtual Machine (JVM) and runs in the background of the application. For example, garbage collection is managed by a system thread created by the JVM.

Türkçe: Bu kitapta sunulanlar da dâhil bütün Java application'ları system thread'ler içerdiği için multithreaded'dır. Bu sizi şaşırtabilir. System thread, Java Virtual Machine (JVM) tarafından oluşturulur ve application'ın arka planında çalışır. Örneğin garbage collection, JVM'nin oluşturduğu bir system thread tarafından yönetilir.

English: Alternatively, a user-defined thread is one created by the application developer to accomplish a specific task. The majority of the programs we've presented so far have contained only one user-defined thread, which calls the main() method. For simplicity, we commonly refer to programs that contain only a single user-defined thread as single-threaded applications.

Türkçe: User-defined thread ise belirli bir task'ı gerçekleştirmek üzere application developer tarafından oluşturulur. Şimdiye kadar sunduğumuz programların çoğu, main() method'u çağıran tek bir user-defined thread içeriyordu. Kolaylık olması için yalnızca bir user-defined thread içeren programlardan genellikle single-threaded application diye söz ederiz.

English: System and user-defined threads can both be created as daemon threads. A daemon thread is one that will not prevent the JVM from exiting when the program finishes. A Java application terminates when the only threads that are running are daemon threads. For example, if garbage collection is the only thread left running, the JVM will automatically shut down.

Türkçe: Hem system thread'ler hem de user-defined thread'ler daemon thread olarak oluşturulabilir. Daemon thread, program bittiğinde JVM'nin çıkmasını engellemez. Çalışan thread'lerin yalnızca daemon thread'lerden ibaret olduğu anda Java application sona erer. Örneğin çalışmaya devam eden tek thread garbage collection thread'i ise JVM otomatik olarak kapanır.

English: Let's take a look at an example. What do you think this outputs?

Türkçe: Bir örneğe bakalım. Sizce çıktısı nedir?

```
public class Zoo {
    public static void pause() { // Defines the thread task
        try {
            Thread.sleep(10_000); // Wait for 10 seconds
        } catch (InterruptedException e) {}
        System.out.println("Thread finished!");
    }

    public static void main(String[] unused) {
        var job = new Thread(() -> pause()); // Create thread
        job.start(); // Start thread
        System.out.println("Main method finished!");
    } }
```

English: The program will output two statements roughly 10 seconds apart:

Türkçe: Program yaklaşık 10 saniye arayla şu iki literal ifadeyi yazdırır. Aradaki `--- 10 second wait` satırı Java code değil, bekleme süresini gösteren metin işaretidir:

English: Main method finished!

Türkçe: Main method finished!

```
--- 10 second wait ---
```

English: Thread finished!

Türkçe: Thread finished!

English: That's right. Even though the `main()` method is done, the JVM will wait for the user thread to be done before ending the program. What if we change `job` to be a daemon thread by adding this to line 11?

Türkçe: Evet, doğru. `main()` method tamamlanmış olsa bile JVM, programı sonlandırmadan önce user thread'in tamamlanmasını bekler. Peki 11. satıra aşağıdaki ifadeyi ekleyerek `job`'ı daemon thread yaparsak ne olur?

```
job.setDaemon(true);
```

English: The program will print the first statement and terminate without ever printing the second line.

Türkçe: Program ilk ifadeyi yazdırır ve ikinci satırı hiç yazdırmadan sona erer.

English: Main method finished!

Türkçe: Main method finished!

English: For the exam, just remember that by default, user-defined threads are not daemons, and the program will wait for them to finish.

Türkçe: Sınav için önemli ayrıntı şudur: Yeni bir thread, daemon durumunu kendisini oluşturan thread'den inherit eder. `main` thread non-daemon olduğundan onun doğrudan oluşturduğu user-defined thread'ler de varsayılan olarak non-daemon'dır ve JVM bunların tamamlanmasını bekler. `setDaemon(true)`, thread'e `start()` çağrılmadan önce uygulanmalıdır; aksi hâlde `IllegalThreadStateException` atılır.

Managing a Thread's Life Cycle

English: After a thread has been created, it is in one of six states, shown in Figure 13.2. You can query a thread's state by calling `getState()` on the thread object.

Türkçe: Bir thread oluşturulduktan sonra Şekil 13.2'de gösterilen altı state'ten birinde bulunur. Thread object üzerinde `getState()` çağırarak thread'in state'ini sorgulayabilirsiniz.

English: FIGURE 13.2 — Thread states

State descriptions: **NEW** — created but not started; **RUNNABLE** — running or able to be run; **BLOCKED** — waiting to enter a synchronized block; **WAITING** — waiting indefinitely to be notified; **TIMED_WAITING** — waiting a specified time; **TERMINATED** — task complete.

Common transitions: Create thread → **NEW**; **NEW** —`start()`→ **RUNNABLE**; **RUNNABLE** —`run()` completes→ **TERMINATED**; attempting to enter an occupied synchronized block moves a thread to **BLOCKED**, and acquiring that monitor makes it eligible for **RUNNABLE** again; **RUNNABLE** —`sleep()`→ **TIMED_WAITING** —time elapsed→ **RUNNABLE**; **RUNNABLE** —`wait()`→ **WAITING** —notification and monitor reacquisition→ **RUNNABLE**.

Türkçe: Şekil 13.2 — Thread durumları

Durum açıklamaları: **NEW** — oluşturulmuş fakat başlatılmamış; **RUNNABLE** — çalışıyor veya çalıştırılmaya uygun; **BLOCKED** — synchronized block'a girmeyi bekliyor; **WAITING** — bildirim gelene kadar süresiz bekliyor; **TIMED_WAITING** — belirli bir süre bekliyor; **TERMINATED** — task tamamlanmış.

Yaygın geçişler: Thread oluşturma → **NEW**; **NEW** —`start()`→ **RUNNABLE**; **RUNNABLE** —`run()` tamamlanır→ **TERMINATED**; kullanımda olan bir monitor'ün synchronized block'una girmeye çalışan thread **BLOCKED** olur, monitor'ü edindiğinde yeniden **RUNNABLE** olmaya uygun hâle gelir; **RUNNABLE** —`sleep()`→ **TIMED_WAITING** —süre dolar→ **RUNNABLE**; **RUNNABLE** —`wait()`→ **WAITING** —bildirim ve monitor'ü yeniden edinme→ **RUNNABLE**.

Java 17 editör notu: `notify()`/`notifyAll()` ile bildirilen bir thread doğrudan çalışmaya başlamaz. `wait()` öncesinde sahip olduğu monitor'ü yeniden edinmelidir; monitor başka bir thread'deyse bu arada **BLOCKED** durumda kalır.

English: Every thread is initialized with a **NEW** state. As soon as `start()` is called, the thread is moved to a **RUNNABLE** state. Does that mean it is actually running? Not exactly: it may be running, or it may not be. The **RUNNABLE** state just means the thread is able to be run. Once the work for the thread is completed or an uncaught exception is thrown, the thread state becomes **TERMINATED**, and no more work is performed.

Türkçe: Her thread **NEW** state ile başlar. `start()` çağrılır çağrılmaz **RUNNABLE** state'e geçer. Bu, gerçekten o anda çalıştığı anlamına gelmez; çalışıyor da olabilir, çalışmıyor da. **RUNNABLE** yalnızca thread'in çalıştırılabilir olduğunu belirtir. Thread'in işi tamamlandığında veya uncaught exception fırlatıldığında state **TERMINATED** olur ve artık başka iş yapılmaz.

English: While in a **RUNNABLE** state, the thread may transition to one of three states where it pauses its work: **BLOCKED**, **WAITING**, or **TIMED_WAITING**. This figure includes common transitions between thread states, but there are other possibilities. For example, a thread in a **WAITING** state might be triggered by `notifyAll()`. Likewise, a thread that is interrupted by another thread will exit **TIMED_WAITING** and go straight back into **RUNNABLE**.

Türkçe: **RUNNABLE** durumundaki thread, çalışmasını duraklatan üç durumdan birine geçebilir: **BLOCKED**, **WAITING** veya **TIMED_WAITING**. Şekil yaygın geçişleri gösterir; başka olasılıklar da vardır. Örneğin **WAITING** durumundaki thread `notifyAll()` ile uyandırılabilir. Interruptible bir beklemedeki thread interrupt edilirse beklemeden çıkar; `sleep()`, `wait()` ve `join()` gibi metotlarda bu genellikle `InterruptedException` atılmasıyla olur ve thread yeniden çalıştırılabilir hâle gelir.

English: We cover some (but not all) of these transitions in this chapter. Some thread-related methods—such as `wait()`, `notify()`, and `join()`—are beyond the scope of the exam and, frankly, difficult to use well. You should avoid them and use the Concurrency API as much as possible. It takes a large amount of skill (and some luck!) to use these methods correctly.

Türkçe: Bu bölümde bu geçişlerin bazılarını, ancak tamamını değil, ele alıyoruz. `wait()`, `notify()` ve `join()` gibi bazı thread metotları sınav kapsamı dışındadır ve doğru kullanılmaları zordur. Bunlardan kaçınılmalı ve mümkün olduğunca **Concurrency API** kullanmalısınız. Bu metotları doğru kullanmak ciddi beceri ve biraz da şans gerektirir.

Polling with Sleep

English: Even though multithreaded programming allows you to execute multiple tasks at the same time, one thread often needs to wait for the results of another thread to proceed. One solution is to use polling. Polling is the process of intermittently checking data at some fixed interval.

Türkçe: Multithreaded programming aynı anda birden fazla task çalıştırmayı sağlasa da bir thread'in ilerleyebilmek için çoğu zaman başka bir thread'in sonucunu beklemesi gerekir. Çözümlerden biri polling kullanmaktır. Polling, veriyi sabit bir zaman aralığıyla periyodik olarak kontrol etme işlemidir.

English: Let's say you have a thread that modifies a shared static counter value, and your `main()` thread is waiting for the thread to reach 1 million:

Türkçe: Shared static counter değerini değiştiren bir thread bulunduğunu ve `main()` thread'in bu değer in 1 milyona ulaşmasını beklediğini varsayalım:

```
public class CheckResults {
    private static int counter = 0;
    public static void main(String[] args) {
        new Thread(() -> {
            for(int i = 0; i < 1_000_000; i++) counter++;
        }).start();
        while(counter < 1_000_000) {
            System.out.println("Not reached yet");
        }
        System.out.println("Reached: "+counter);
    } }
```

English: How many times does this program print Not reached yet? The answer is, we don't know! It could output 0, 10, or a million times. Using a `while()` loop to check for data without some kind of delay is considered a bad coding practice as it ties up CPU resources for no reason.

Türkçe: Bu program literal `Not reached yet` ifadesini kaç kez yazdırır? Sayı bilinemez; 0, 10 veya bir milyon olabilir. Ayrıca `counter` ne `volatile` olduğundan ne de synchronization ile okunduğundan `main` thread güncel değeri görmek zorunda değildir; Java Memory Model açısından loop'un sona ermesi bile garanti edilmez. Gecikmesiz `while()` loop ile polling yapmak CPU kaynaklarını gereksiz yere tükettiğinden kötü bir coding practice'tir.

English: We can improve this result by using the `Thread.sleep()` method to implement polling and sleep for 1,000 milliseconds, aka 1 second:

Türkçe: Polling uygulamak ve 1.000 milisaniye, yani 1 saniye beklemek için `Thread.sleep()` method kullanarak bu sonucu iyileştirebiliriz:

```
public class CheckResultsWithSleep {
    private static int counter = 0;
    public static void main(String[] a) {
        new Thread(() -> {
            for(int i = 0; i < 1_000_000; i++) counter++;
        }).start();
        while(counter < 1_000_000) {
            System.out.println("Not reached yet");
            try {
                Thread.sleep(1_000); // 1 SECOND
            } catch (InterruptedException e) {
                System.out.println("Interrupted!");
            }
        }
        System.out.println("Reached: "+counter);
    } }
```

English: While one second may seem like a small amount, we have now freed the CPU to do other work instead of checking the counter variable infinitely within a loop. Notice that the `main()` thread alternates between `TIMED_WAITING` and `RUNNABLE` when `sleep()` is entered and exited, respectively.

Türkçe: Bir saniye kısa görünse de CPU artık loop içinde `counter` değişkenini aralıksız kontrol etmek yerine başka işler yapabilir. Main thread `sleep()` sırasında `TIMED_WAITING`, beklemeden çıktığında `RUNNABLE` durumundadır. Bununla birlikte `Thread.sleep()` bir memory-visibility garantisi oluşturmaz; shared `counter` sorunu bu örnekte devam eder.

English: How many times does the `while()` loop execute in this revised class? Still unknown! While polling does prevent the CPU from being overwhelmed with a potentially infinite loop, it does not guarantee when the loop will terminate. For example, the separate thread could be losing CPU time to a higher-priority process, resulting in multiple executions of the `while()` loop before it finishes.

Türkçe: Bu güncellenmiş class'ta `while()` loop kaç kez çalışır? Yine bilinemez. Polling, CPU'nun potansiyel olarak sonsuz bir loop nedeniyle aşırı yüklenmesini önler; fakat loop'un ne zaman sona ereceğini garanti etmez. Örneğin ayrı thread, CPU time'ı daha yüksek priority'li bir process'e kaptırabilir; bu durumda task tamamlanmadan önce `while()` loop birden fazla kez çalışır.

English: Another issue to be concerned about is the shared counter variable. What if one thread is reading the counter variable while another thread is writing it? The thread reading the shared variable may end up with an invalid or unexpected value. We discuss these issues in detail in the upcoming section on writing thread-safe code.

Türkçe: Dikkat edilmesi gereken başka bir konu da shared counter variable'dır. Bir thread counter'ı okurken başka bir thread yazarsa ne olur? Shared variable'ı okuyan thread geçersiz veya beklenmedik bir değer elde edebilir. Bu sorunları ilerideki thread-safe code bölümünde ayrıntılı olarak ele alacağız.

Interrupting a Thread

English: While our previous solution prevented the CPU from waiting endlessly on a `while()` loop, it did come at the cost of inserting one-second delays into our program. If the task takes 2.1 seconds to run, the program will use the full 3 seconds, wasting 0.9 seconds.

Türkçe: Önceki çözüm CPU'nun `while()` loop içinde sürekli meşgul olmasını engelledi, ancak programa birer saniyelik gecikmeler ekledi. Task 2,1 saniyede tamamlanıyorsa program tam 3 saniye sürer ve 0,9 saniye boşa gider.

English: One way to improve this program is to allow the thread to interrupt the `main()` thread when it's done:

Türkçe: Programı iyileştirmenin bir yolu, worker thread tamamlandığında `main()` thread'i interrupt etmesini sağlamaktır:

```
public class CheckResultsWithSleepAndInterrupt {
    private static int counter = 0;
    public static void main(String[] a) {
        final var mainThread = Thread.currentThread();
        new Thread(() -> {
            for(int i = 0; i < 1_000_000; i++) counter++;
            mainThread.interrupt();
        }).start();
        while(counter < 1_000_000) {
            System.out.println("Not reached yet");
            try {
                Thread.sleep(1_000); // 1 SECOND
            } catch (InterruptedException e) {
                System.out.println("Interrupted!");
            }
        }
        System.out.println("Reached: "+counter);
    } }
```

English: This improved version includes both `sleep()`, to avoid tying up the CPU, and `interrupt()`, so the thread's work ends without delaying the program. As before, our `main()` thread's state alternates between `TIMED_WAITING` and `RUNNABLE`. Calling `interrupt()` on a thread in the `TIMED_WAITING` or `WAITING` state causes the `main()` thread to become `RUNNABLE` again, triggering an `InterruptedException`. The thread may also move to a `BLOCKED` state if it needs to reacquire resources when it wakes up.

Türkçe: Bu sürüm CPU'yu gereksiz meşgul etmemek için `sleep()`, worker task bittiğinde programı gecikmeden uyandırmak için `interrupt()` kullanır. Main thread `TIMED_WAITING` ile `RUNNABLE` arasında geçiş yapar. Thread `sleep()` gibi interruptible bir beklemedeyken interrupt edilirse bekleme `InterruptedException` atarak sona erer; exception atıldığı anda interrupt status temizlenir, `catch` bloğuna girilmesi temizleyen işlem değildir. Thread uyanırken bir monitor'ü yeniden edinmesi gerekiyorsa `BLOCKED` durumuna da geçebilir. Interrupt çağrısı beklemeden hemen önce geldiyse status set edilir ve sonraki `sleep()` çağrısı exception'ı hemen atar.

English: Calling `interrupt()` on a thread already in a `RUNNABLE` state doesn't change the state. In fact, it only changes the behavior if the thread is periodically checking the `Thread.isInterrupted()` value state.

Türkçe: Zaten `RUNNABLE` durumundaki bir thread üzerinde `interrupt()` çağırılması state'i doğrudan değiştirmez; thread'in `interrupt` status'ünü set eder. Thread bunu `Thread.isInterrupted()` ile kontrol ederek tepki verebilir veya daha sonra `sleep()`/`wait()`/`join()` gibi interruptible bir metoda girerse metot `InterruptedException` atabilir.

Creating Threads with the Concurrency API

English: Java includes the `java.util.concurrent` package, which we refer to as the Concurrency API, to handle the complicated work of managing threads for you. The Concurrency API includes the `ExecutorService` interface, which defines services that create and manage threads.

Türkçe: Java, thread yönetiminin karmaşık işlerini üstlenen ve `Concurrency API` olarak adlandırdığımız `java.util.concurrent` package'ını içerir. Bu API'de thread'leri oluşturan ve yöneten hizmetleri tanımlayan `ExecutorService` interface'i bulunur.

English: You first obtain an instance of an `ExecutorService` interface, and then you send the service tasks to be processed. The framework includes numerous useful features, such as thread pooling and scheduling. It is recommended that you use this framework any time you need to create and execute a separate task, even if you need only a single thread.

Türkçe: Önce bir `ExecutorService` instance edinir, ardından işlenecek task'ları service'e gönderirsiniz. Framework, thread pooling ve scheduling gibi birçok yararlı özellik sunar. Yalnızca tek bir thread'e ihtiyacınız olsa bile ayrı bir task oluşturup çalıştırmanız gereken her durumda bu framework'ü kullanmanız önerilir.

English: When writing multithreaded programs in practice, it is often better to use the Concurrency API (or some other multithreaded SDK) rather than work with `Thread` objects directly. The libraries are much more robust, and it is easier to handle complex interactions.

Türkçe: Uygulamada multithreaded program yazarken doğrudan `Thread` nesneleriyle çalışmak yerine `Concurrency API` veya başka bir multithreaded SDK kullanmak çoğu zaman daha iyidir. Bu kütüphaneler daha sağlamdır ve karmaşık etkileşimleri yönetmeyi kolaylaştırır.

Introducing the Single-Thread Executor

English: Since `ExecutorService` is an interface, how do you obtain an instance of it? The Concurrency API includes the `Executors` factory class that can be used to create instances of the `ExecutorService` object. Let's rewrite our earlier example with the two `Runnable` instances to using an `ExecutorService`.

Türkçe: `ExecutorService` bir interface olduğuna göre bunun instance'ını nasıl ediniriz? `Concurrency API`, `ExecutorService` örnekleri oluşturan `Executors` factory class'ını içerir. İki `Runnable` instance kullanan önceki örneği `ExecutorService` ile yeniden yazalım.

```
ExecutorService service = Executors.newSingleThreadExecutor();
try {
    System.out.println("begin");
    service.execute(printInventory);
    service.execute(printRecords);
    service.execute(printInventory);
    System.out.println("end");
} finally {
    service.shutdown();
}
```

English: In this example, we use the `newSingleThreadExecutor()` method to create the service. Unlike our earlier example, in which we had four threads (one `main()` and three new threads), we have only two threads (one `main()` and one new thread). This means that the output, while still unpredictable, will have less variation than before. For example, the following is one possible output:

Türkçe: Bu örnekte `service`'i oluşturmak için `newSingleThreadExecutor()` method kullanılır. Önceki örnekte dört thread (bir `main()` ve üç yeni thread) varken burada yalnızca iki thread (bir `main()` ve bir yeni thread) vardır. Çıktı yine öngörülemez olsa da önceki örneğe göre daha az değişkenlik gösterir. Aşağıdaki olası çıktılarından biridir:

English: begin Printing zoo inventory Printing record: 0 Printing record: 1 end Printing record: 2 Printing zoo inventory Notice that the `printRecords` loop is no longer interrupted by other `Runnable` tasks sent to the thread executor. With a single-thread executor, tasks are guaranteed to be executed sequentially. Notice that the end text is output while our thread executor tasks are still running. This is because the `main()` method is still an independent thread from the `ExecutorService`.

Türkçe: begin Printing zoo inventory Printing record: 0 Printing record: 1 end Printing record: 2 Printing zoo inventory `printRecords` loop'un artık thread executor'a gönderilen diğer `Runnable` task'lar tarafından kesilmediğine dikkat edin. Single-thread executor ile task'ların sequential olarak çalıştırılması garanti edilir. Thread executor task'ları hâlâ çalışırken end metninin yazdırılabilmesinin nedeni, `main()` method'u çalıştıran thread'in `ExecutorService` worker thread'inden bağımsız olmasıdır.

Shutting Down a Thread Executor

Dil çalışması: Bu başlıktaki kelimeler için [ünite sözlüğüne](#) bak.

English: Once you have finished using a thread executor, it is important that you call the `shutdown()` method. A thread executor creates a non-daemon thread on the first task that is executed, so failing to call `shutdown()` will result in your application never terminating.

Türkçe: Thread executor kullanımı bitince `shutdown()` çağırılması önemlidir. `Executors` factory metotlarının varsayılan thread factory'si ilk task yürütülürken non-daemon worker thread oluşturur. Bu nedenle executor kapatılmazsa application sonlanmayabilir.

English: The shutdown process for a thread executor involves first rejecting any new tasks submitted to the thread executor while continuing to execute any previously submitted tasks. During this time, calling `isShutdown()` will return `true`, while `isTerminated()` will return `false`. If a new task is submitted to the thread executor while it is shutting down, a `RejectedExecutionException` will be thrown. Once all active tasks have been completed, `isShutdown()` and `isTerminated()` will both return `true`. Figure 13.3 shows the life cycle of an `ExecutorService` object.

Türkçe: Thread executor'ın shutdown sürecinde yeni task'lar reddedilirken daha önce submit edilmiş task'lar çalıştırılmaya devam eder. Bu sırada `isShutdown()` `true`, `isTerminated()` `false` döndürür. Kapanmakta olan thread executor'a yeni bir task gönderilirse `RejectedExecutionException` fırlatılır. Bütün active task'lar tamamlandığında hem `isShutdown()` hem `isTerminated()` `true` döndürür. Şekil 13.3 bir `ExecutorService` object'in yaşam döngüsünü gösterir.

English: FIGURE 13.3 — `ExecutorService` life cycle

Active: Accepts new tasks and executes tasks; `isShutdown()` is `false` and `isTerminated()` is `false`.

Calling `shutdown()` moves the service to **Shutting down**: it rejects new tasks but continues executing accepted tasks; `isShutdown()` is `true` and `isTerminated()` is `false`.

When all tasks finish, the service reaches **Shutdown**: it rejects new tasks and no tasks are running; `isShutdown()` and `isTerminated()` are both `true`.

Türkçe: Şekil 13.3 — `ExecutorService` yaşam döngüsü

Active: Yeni task'ları kabul eder ve task'ları çalıştırır; `isShutdown()` ile `isTerminated()` değerleri `false` olur.

`shutdown()` çağırısı service'i **Shutting down** durumuna geçirir: yeni task'ları reddeder, fakat kabul edilmiş task'ları çalıştırmayı sürdürür; `isShutdown()` `true`, `isTerminated()` `false` olur.

Bütün task'lar tamamlandığında service **Shutdown** durumuna ulaşır: yeni task'ları reddeder ve çalışan task kalmaz; `isShutdown()` ile `isTerminated()` değerlerinin ikisi de `true` olur.

English: For the exam, you should be aware that `shutdown()` does not stop any tasks that have already been submitted to the thread executor.

Türkçe: Sınav için `shutdown()` method'un, thread executor'a daha önce submit edilmiş task'ları durdurmadığını bilmelisiniz.

English: What if you want to cancel all running and upcoming tasks? The `ExecutorService` provides a method called `shutdownNow()`, which attempts to stop all running tasks and discards any that have not been started yet. It is not guaranteed to succeed because it is possible to create a thread that will never terminate, so any attempt to interrupt it may be ignored.

Türkçe: Çalışan ve sırada bekleyen task'ları iptal etmek için `shutdownNow()` kullanılabilir. Bu metot çalışan task'ları genellikle interrupt ederek durdurmayı dener ve henüz başlamamış task'ları kuyruktan çıkarıp `List<Runnable>` olarak döndürür. Interrupt cooperative olduğundan çalışan task'ların gerçekten duracağı garanti edilmez; task interrupt isteğini dikkate almayabilir.

English: As you learned in Chapter 11, “Exceptions and Localization,” resources such as thread executors should be properly closed to prevent memory leaks. Unfortunately, the `ExecutorService` interface does not extend the `AutoCloseable` interface, so you cannot use a try-with-resources statement. You can still use a finally block, as we do throughout this chapter. While you are not required to use a finally block, it is considered a good practice to do so.

Türkçe: Bölüm 11 “Exceptions and Localization” konusunda görüldüğü gibi thread executor gibi resource'lar sızıntıları ve çalışan thread'lerin açık kalmasını önlemek için düzgün kapatılmalıdır. Java 17'de `ExecutorService`, `AutoCloseable` interface'ini extend etmez; bu nedenle doğrudan try-with-resources içinde kullanılamaz. Bu bölümdeki örneklerde olduğu gibi `finally` block kullanmak zorunlu olmasa da iyi pratiktir.

Submitting Tasks

English: You can submit tasks to an `ExecutorService` instance multiple ways. The first method we presented, `execute()`, is inherited from the `Executor` interface, which the `ExecutorService` interface extends. The `execute()` method takes a `Runnable` instance and completes the task asynchronously. Because the return type of the method is `void`, it does not tell us anything about the result of the task. It is considered a “fire-and-forget” method, as once it is submitted, the results are not directly available to the calling thread.

Türkçe: Task'ları bir `ExecutorService` instance'a birden fazla yolla gönderebilirsiniz. İlk gördüğümüz `execute()`, `ExecutorService`'in extend ettiği `Executor` interface'ten miras alınır. `execute()` bir `Runnable` instance alır ve task'ı asynchronously çalıştırır. Method'un return type'ı `void` olduğu için task'ın sonucu hakkında bilgi vermez. Bu nedenle “fire-and-forget” method kabul edilir; task gönderildikten sonra sonucu calling thread tarafından doğrudan kullanılamaz.

English: Fortunately, the writers of Java added `submit()` methods to the `ExecutorService` interface, which, like `execute()`, can be used to complete tasks asynchronously. Unlike `execute()`, though, `submit()` returns a `Future` instance that can be used to determine whether the task is complete. It can also be used to return a generic result object after the task has been completed.

Türkçe: Java geliştiricileri `ExecutorService` interface'ine, `execute()` gibi task'ları asynchronously çalıştıran `submit()` overload'ları eklemiştir. Ancak `execute()` 'den farklı olarak `submit()`, task'ın tamamlanıp tamamlanmadığını belirlemekte kullanılabilecek bir `Future` instance döndürür. Task tamamlandıktan sonra generic bir result object almak için de kullanılabilir.

English: Table 13.1 shows the five methods, including `execute()` and two `submit()` methods, that you should know for the exam. Don't worry if you haven't seen `Future` or `Callable` before; we discuss them in detail in the next section.

Türkçe: Tablo 13.1, `execute()` ve iki `submit()` overload'u dâhil sınav için bilmeniz gereken beş method'u gösterir. `Future` veya `Callable` ile daha önce karşılaşmadıysanız endişelenmeyin; sonraki bölümde ikisini de ayrıntılı olarak ele alacağız.

English: In practice, using the `submit()` method is quite similar to using the `execute()` method, except that the `submit()` method returns a `Future` instance that can be used to determine whether the task has completed execution.

Türkçe: Uygulamada `submit()` kullanımı `execute()` kullanımına oldukça benzer. Temel fark, `submit()` method'un task'ın tamamlanıp tamamlanmadığını belirlemekte kullanılabilecek bir `Future` instance döndürmesidir.

English: TABLE 13.1 — `ExecutorService` methods - `void execute(Runnable command)` — Executes a `Runnable` task at some point in the future. - `Future<?> submit(Runnable task)` — Executes a `Runnable` task at some point in the future and returns a `Future<?>` representing the task. - `<T> Future<T> submit(Callable<T> task)` — Executes a `Callable<T>` task at some point in the future and returns a `Future<T>` representing the pending result. - `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)` throws `InterruptedException` — Executes the given tasks, waits for all of them to complete, and returns their `Future<T>` instances in the same order as the tasks in the original collection. - `<T> T invokeAny(Collection<? extends Callable<T>> tasks)` throws `InterruptedException`, `ExecutionException` — Executes the given tasks, waits for at least one to complete successfully, and returns the result of one successfully completed task.

Türkçe: TABLO 13.1 — `ExecutorService` metotları - `void execute(Runnable command)` — Bir `Runnable` task'ını gelecekte uygun bir zamanda çalıştırır. - `Future<?> submit(Runnable task)` — Bir `Runnable` task'ını gelecekte uygun bir zamanda çalıştırır ve task'ı temsil eden `Future<?>` döndürür. - `<T> Future<T> submit(Callable<T> task)` — Bir `Callable<T>` task'ını gelecekte uygun bir zamanda çalıştırır ve bekleyen sonucu temsil eden `Future<T>` döndürür. - `<T> List<Future<T>> invokeAll(Collection<? extends Callable<T>> tasks)` throws `InterruptedException` — Verilen task'ları çalıştırır, hepsinin tamamlanmasını bekler ve `Future<T>` instance'larını original collection'daki task sırasıyla döndürür. - `<T> T invokeAny(Collection<? extends Callable<T>> tasks)` throws `InterruptedException`, `ExecutionException` — Verilen task'ları çalıştırır, en az birinin başarıyla tamamlanmasını bekler ve başarıyla tamamlanan task'lardan birinin sonucunu döndürür.

English: Submitting Tasks: `execute()` vs. `submit()`

Türkçe: Task Gönderme: `execute()` vs. `submit()`

English: As you might have noticed, the `execute()` and `submit()` methods are nearly identical when applied to `Runnable` expressions. The `submit()` method has the obvious advantage of doing the same thing `execute()` does, but with a return object that can be used to track the result. Because of this advantage and the fact that `execute()` does not support `Callable` expressions, we tend to prefer `submit()` over `execute()`, even if we don't store the `Future` reference.

Türkçe: Fark etmiş olabileceğiniz gibi `execute()` ve `submit()`, `Runnable` expression'ları için neredeyse aynı işi yapar. `submit()`, `execute()` ile aynı işi yaparken sonucu izlemekte kullanılabilecek bir return object sunar. Bu avantaj ve `execute()` metodunun `Callable` expression'larını desteklememesi nedeniyle `Future` reference'ını saklamasak bile `submit()` tercih edilir.

English: For the exam, you need to be familiar with both `execute()` and `submit()`, but in your own code we recommend `submit()` over `execute()` whenever possible.

Türkçe: Sınav için hem `execute()` hem `submit()` method'u bilmeniz gerekir; kendi kodunuzda ise mümkün olduğunda `execute()` yerine `submit()` kullanmanız önerilir.

Waiting for Results

English: How do we know when a task submitted to an `ExecutorService` is complete? As mentioned in the previous section, the `submit()` method returns a `Future<V>` instance that can be used to determine this result.

Türkçe: `ExecutorService`'e `submit` edilen bir task'ın tamamlandığını nasıl anlarız? Önceki bölümde belirtildiği gibi `submit()` method, bunu belirlemekte kullanılabilecek bir `Future<V>` instance döndürür.

```
Future<?> future = service.submit(() -> System.out.println("Hello"));
```

English: The `Future` type is actually an interface. For the exam, you don't need to know any of the classes that implement `Future`, just that a `Future` instance is returned by various API methods. Table 13.2 includes useful methods for determining the state of a task.

Türkçe: `Future` aslında bir interface'tir. Sınav için `Future`'i implement eden class'ları bilmeniz gerekmez; çeşitli API metotlarının bir `Future` instance döndürdüğünü bilmeniz yeterlidir. Tablo 13.2, bir task'ın state'ini belirlemekte kullanılan yararlı metotları içerir.

English: TABLE 13.2 — `Future` methods

- `boolean isDone()` — Returns `true` if the task completed normally, threw an exception, or was cancelled.
- `boolean isCancelled()` — Returns `true` if the task was cancelled before it completed normally.
- `boolean cancel(boolean mayInterruptIfRunning)` — Attempts to cancel the task. It returns `true` if the cancellation request was accepted and `false` if the task could not be cancelled, typically because it had already completed.
- `V get()` throws `InterruptedException`, `ExecutionException` — Retrieves the task result, waiting indefinitely if the result is not yet available.
- `V get(long timeout, TimeUnit unit)` throws `InterruptedException`, `ExecutionException`, `TimeoutException` — Waits up to the specified time for the result. It throws the checked `TimeoutException` if the result is not ready when the timeout expires.

Türkçe: TABLO 13.2 — `Future` metotları

- `boolean isDone()` — Task normal tamamlanmış, exception atmış veya iptal edilmişse `true` döndürür.
- `boolean isCancelled()` — Task normal biçimde tamamlanmadan iptal edilmişse `true` döndürür.
- `boolean cancel(boolean mayInterruptIfRunning)` — Task'ı iptal etmeyi dener. İptal isteği kabul edilirse `true`; task genellikle zaten tamamlandığı için iptal edilemiyorsa `false` döndürür. `mayInterruptIfRunning`, çalışan task'ın thread'ine interrupt gönderilip gönderilmeyeceğini belirler; task'ın gerçekten duracağını garanti etmez.
- `V get()` throws `InterruptedException`, `ExecutionException` — Task'ın sonucunu alır; sonuç hazır değilse süresiz bekler.
- `V get(long timeout, TimeUnit unit)` throws `InterruptedException`, `ExecutionException`, `TimeoutException` — Sonucu almak için en fazla belirtilen süre kadar bekler; timeout dolduğunda sonuç hazır değilse checked `TimeoutException` atar.

English: The following is an updated version of our earlier polling example CheckResults class, which uses a Future instance to wait for the results:

Türkçe: Aşağıda, sonuçları beklemek için Future instance kullanan önceki polling örneğimiz CheckResults class'ın güncellenmiş sürümü yer almaktadır:

```
import java.util.concurrent.*;
public class CheckResults {
    private static int counter = 0;
    public static void main(String[] unused) throws Exception {
        ExecutorService service = Executors.newSingleThreadExecutor();
        try {
            Future<?> result = service.submit(() -> {
                for(int i = 0; i < 1_000_000; i++) counter++;
            });
            result.get(10, TimeUnit.SECONDS); // Returns null for Runnable
            System.out.println("Reached!");
        } catch (TimeoutException e) {
            System.out.println("Not reached in time");
        } finally {
            service.shutdown();
        } } }
```

English: This example is similar to our earlier polling implementation, but it does not use the Thread class directly. In part, this is the essence of the Concurrency API: to do complex things with threads without having to manage threads directly. It also waits at most 10 seconds, throwing a TimeoutException on the call to result.get() if the task is not done.

Türkçe: Bu örnek önceki polling implementation'a benzer, ancak Thread class'ını doğrudan kullanmaz. Concurrency API'nin özü kısmen budur: Thread'leri doğrudan yönetmeden onlarla karmaşık işler yapabilmek. Kod en fazla 10 saniye bekler; task tamamlanmazsa result.get() çağırısı TimeoutException fırlatır.

English: What is the return value of this task? As Future<V> is a generic interface, the type V is determined by the return type of the Runnable method. Since the return type of Runnable.run() is void, the get() method always returns null when working with Runnable expressions.

Türkçe: Bu task'ın dönüş değeri nedir? Runnable.run() metodunun dönüş tipi void olduğundan bir Runnable değer üretmez. Bu nedenle submit(Runnable) tarafından döndürülen Future<?> başarıyla tamamlandığında get() her zaman null döndürür.

English: The Future.get() method can take an optional value and enum type java.util.concurrent.TimeUnit. Table 13.3 presents the full list of TimeUnit values since numerous methods in the Concurrency API use this enum.

Türkçe: Future.get() metodunun zaman aşımı belirten overload'u, V get(long timeout, TimeUnit unit) throws InterruptedException, ExecutionException, TimeoutException, sayısal bir süre ile java.util.concurrent.TimeUnit enum sabitini alır. Concurrency API içindeki birçok metot bu enum'u kullandığı için Tablo 13.3 tüm TimeUnit değerlerini gösterir.

English: TABLE 13.3 — TimeUnit values - `TimeUnit.NANOSECONDS` — Time in one-billionths of a second (1/1,000,000,000). - `TimeUnit.MICROSECONDS` — Time in one-millionths of a second (1/1,000,000). - `TimeUnit.MILLISECONDS` — Time in one-thousandths of a second (1/1,000). - `TimeUnit.SECONDS` — Time in seconds. - `TimeUnit.MINUTES` — Time in minutes. - `TimeUnit.HOURS` — Time in hours. - `TimeUnit.DAYS` — Time in days.

Türkçe: TABLO 13.3 — TimeUnit değerleri - `TimeUnit.NANOSECONDS` — Saniyenin milyarda biri (1/1.000.000.000). - `TimeUnit.MICROSECONDS` — Saniyenin milyonda biri (1/1.000.000). - `TimeUnit.MILLISECONDS` — Saniyenin binde biri (1/1.000). - `TimeUnit.SECONDS` — Saniye. - `TimeUnit.MINUTES` — Dakika. - `TimeUnit.HOURS` — Saat. - `TimeUnit.DAYS` — Gün.

Introducing Callable

English: The `java.util.concurrent.Callable` functional interface is similar to `Runnable` except that its `call()` method returns a value and can throw a checked exception. The following is the definition of the `Callable` interface:

Türkçe: `java.util.concurrent.Callable` functional interface'i (işlevsel interface), `call()` metodunun bir değer döndürmesi ve checked exception atabilmesi dışında `Runnable`'a benzer. `Callable` interface'inin tanımı şöyledir:

```
@FunctionalInterface public interface Callable<V> {
    V call() throws Exception;
}
```

English: The `Callable` interface is often preferable over `Runnable`, since it allows more details to be retrieved easily from the task after it is completed. That said, we use both interfaces throughout this chapter, as they are interchangeable in situations where the lambda does not throw an exception, and there is no return type. Luckily, the `ExecutorService` includes an overloaded version of the `submit()` method that takes a `Callable` object and returns a generic `Future<T>` instance.

Türkçe: `Callable`, task tamamlandıktan sonra sonuç bilgisinin kolayca alınmasını sağladığı için çoğu durumda `Runnable`'dan daha kullanışlıdır. Bununla birlikte lambda checked exception atmıyor ve bir değer döndürmüyorsa bu bölümde iki interface de uygun olduğu yerde kullanılır. `ExecutorService`, bir `Callable<T>` alan ve `Future<T>` döndüren `<T> Future<T> submit(Callable<T> task)` overload'unu sağlar.

English: Unlike `Runnable`, in which the `get()` methods always return null, the `get()` methods on a `Future` instance return the matching generic type (which could also be a null value).

Türkçe: Bir `Runnable` için `get()` her zaman null döndürür. Buna karşılık, `Callable<T>` için elde edilen `Future<T>` üzerindeki `get()`, eşleşen generic tipteki sonucu döndürür; task'ın döndürdüğü değer yine de null olabilir.

English: Let's take a look at an example using `Callable`:

Türkçe: `Callable` kullanılan bir örneğe bakalım:

```
var service = Executors.newSingleThreadExecutor();
try {
    Future<Integer> result = service.submit(() -> 30 + 11);
    System.out.println(result.get()); // 41
} finally {
    service.shutdown();
}
```

English: We could rewrite this example using `Runnable`, some shared object, and an `interrupt()` or `timed wait`, but this implementation is a lot easier to code and understand. In essence, that's the spirit of the Concurrency API, giving you the tools to write multithreaded code that is thread-safe, performant, and easy to follow.

Türkçe: Bu örnek `Runnable`, paylaşılan bir nesne ve `interrupt()` ya da süreli bekleme kullanılarak da yazılabilirdi; ancak mevcut uygulamanın kodlanması ve anlaşılması çok daha kolaydır. `Concurrency API`'nin amacı da thread-safe (thread güvenli), performanslı ve izlenmesi kolay multithreaded kod yazmayı sağlayan araçlar sunmaktır.

Waiting for All Tasks to Finish

English: After submitting a set of tasks to a thread executor, it is common to wait for the results. As you saw in the previous sections, one solution is to call `get()` on each `Future` object returned by the `submit()` method. If we don't need the results of the tasks and are finished using our thread executor, there is a simpler approach.

Türkçe: Bir task kümesini executor'a gönderdikten sonra sonuçları beklemek yaygın bir ihtiyaçtır. Önceki bölümlerde görüldüğü gibi çözümlerden biri, `submit()` metodunun döndürdüğü her `Future` üzerinde `get()` çağırmasıdır. Task sonuçlarına ihtiyaç yoksa ve executor artık kullanılmıyacaksa daha basit bir yaklaşım vardır.

English: First, we shut down the thread executor using the `shutdown()` method. Next, we use the `awaitTermination()` method available for all thread executors. The method waits the specified time to complete all tasks, returning sooner if all tasks finish or an `InterruptedException` is detected. You can see an example of this in the following code snippet:

Türkçe: Önce executor'a `shutdown()` çağırısı yaparız. Ardından `ExecutorService` tarafından sağlanan `awaitTermination()` ile sonlanmayı belirli bir süre bekleriz. `boolean awaitTermination(long timeout, TimeUnit unit) throws InterruptedException`, executor süre dolmadan sonlanırsa `true`, süre dolarsa `false` döndürür; bekleyen thread kesilirse `InterruptedException` atar. Aşağıdaki kod bunun bir örneğidir:

```
ExecutorService service = Executors.newSingleThreadExecutor();
try {
    // Add tasks to the thread executor
    ...
} finally {
    service.shutdown();
}
service.awaitTermination(1, TimeUnit.MINUTES);
// Check whether all tasks are finished
if(service.isTerminated()) System.out.println("Finished!");
else System.out.println("At least one task is still running");
```

English: In this example, we submit a number of tasks to the thread executor and then shut down the thread executor and wait up to one minute for the results. Notice that we can call `isTerminated()` after the `awaitTermination()` method finishes to confirm that all tasks are finished.

Türkçe: Bu örnekte executor'a bir dizi task gönderilir, ardından executor kapatılır ve sonlanması en fazla bir dakika beklenir. `awaitTermination()` tamamlandıktan sonra `isTerminated()` çağrılarak bütün task'ların bitip executor'ın gerçekten sonlandığı doğrulanabilir.

Scheduling Tasks

English: Often in Java, we need to schedule a task to happen at some future time. We might even need to schedule the task to happen repeatedly, at some set interval. For example, imagine that we want to check the supply of food for zoo animals once an hour and fill it as needed. `ScheduledExecutorService`, which is a subinterface of `ExecutorService`, can be used for just such a task.

Türkçe: Java'da çoğu zaman bir task'ı gelecekteki bir anda çalışacak biçimde zamanlamamız gerekir. Task'ın belirli aralıklarla yinelenmesi de istenebilir. Örneğin, hayvanat bahçesindeki hayvanların yiyeceğini saatte bir kontrol edip gerektiğinde tamamlamak isteyebiliriz. `ExecutorService`'in subinterface'i olan `ScheduledExecutorService` tam olarak bu tür işler için kullanılabilir.

English: Like `ExecutorService`, we obtain an instance of `ScheduledExecutorService` using a factory method in the `Executors` class, as shown in the following snippet:

Türkçe: `ExecutorService`'te olduğu gibi `ScheduledExecutorService` örneğini de `Executors` sınıfındaki bir factory method ile elde ederiz:

```
ScheduledExecutorService service =
    Executors.newSingleThreadScheduledExecutor();
```

English: We could store an instance of `ScheduledExecutorService` in an `ExecutorService` variable, although doing so would mean we'd have to cast the object to call any scheduling methods.

Türkçe: Bir `ScheduledExecutorService` örneği `ExecutorService` değişkeninde saklanabilir; ancak bu durumda zamanlama metotlarını çağırmak için nesnenin `ScheduledExecutorService` tipine cast edilmesi gerekir.

English: Refer to Table 13.4 for our summary of `ScheduledExecutorService` methods. Each of these methods returns a `ScheduledFuture` object.

Türkçe: `ScheduledExecutorService` metotlarının özeti Tablo 13.4'te verilmiştir. Bu metotların her biri bir `ScheduledFuture` nesnesi döndürür.

English: TABLE 13.4 — `ScheduledExecutorService` methods - `<V> ScheduledFuture<V>`
`schedule(Callable<V> callable, long delay, TimeUnit unit)` — Creates and executes a `Callable<V>` task after the given delay. - `ScheduledFuture<?> schedule(Runnable command, long delay, TimeUnit unit)` — Creates and executes a `Runnable` task after the given delay. - `ScheduledFuture<?> scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit)` — Creates and executes a `Runnable` task after the given initial delay and then starts executions according to the given period. - `ScheduledFuture<?> scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit)` — Creates and executes a `Runnable` task after the given initial delay, then leaves the given delay between the end of one execution and the start of the next.

Türkçe: TABLO 13.4 — `ScheduledExecutorService` metotları - `<V> ScheduledFuture<V>`
`schedule(Callable<V> callable, long delay, TimeUnit unit)` — Verilen gecikmenin ardından `Callable<V>` task'ını oluşturur ve çalıştırır. - `ScheduledFuture<?> schedule(Runnable command, long delay, TimeUnit unit)` — Verilen gecikmenin ardından `Runnable` task'ını oluşturur ve çalıştırır. - `ScheduledFuture<?> scheduleAtFixedRate(Runnable command, long initialDelay, long period, TimeUnit unit)` — İlk gecikmeden sonra task'ı çalıştırır ve sonraki çalışmaları verilen period'a göre başlatır. - `ScheduledFuture<?> scheduleWithFixedDelay(Runnable command, long initialDelay, long delay, TimeUnit unit)` — İlk gecikmeden sonra task'ı çalıştırır; sonraki çalışmanın başlangıcı ile önceki çalışmanın bitişi arasında verilen delay kadar süre bırakır.

English: In practice, these methods are among the most convenient in the Concurrency API, as they perform relatively complex tasks with a single line of code. The delay and period parameters rely on the `TimeUnit` argument to determine the format of the value, such as seconds or milliseconds.

Türkçe: Bu metotlar nispeten karmaşık zamanlama işlemlerini tek satırla ifade edebildiği için `Concurrency API`'nin en kullanışlı araçlarından. `delay` ve `period` değerlerinin saniye mi, milisaniye mi olduğu `TimeUnit` argümanı ile belirlenir.

English: The first two `schedule()` methods in Table 13.4 take a `Callable` or `Runnable`, respectively; perform the task after some delay; and return a `ScheduledFuture` instance. The `ScheduledFuture` interface is identical to the `Future` interface, except that it includes a

Türkçe: Tablo 13.4'teki ilk iki `schedule()` metodu sırasıyla bir `Callable` ve `Runnable` alır, task'ı belirli bir gecikmeden sonra çalıştırır ve bir `ScheduledFuture` örneği döndürür. `ScheduledFuture`, `Future` interface'ine ek olarak kalan gecikmeyi sorgulayan bir

English: `getDelay()` method that returns the remaining delay. The following uses the `schedule()` method with `Callable` and `Runnable` tasks:

Türkçe: `long getDelay(TimeUnit unit)` metoduna sahiptir. Aşağıdaki örnek `schedule()` metodunu `Callable` ve `Runnable` task'larıyla kullanır:

```
ScheduledExecutorService service =
    Executors.newSingleThreadScheduledExecutor();
Runnable task1 = () -> System.out.println("Hello Zoo");
Callable<String> task2 = () -> "Monkey";
ScheduledFuture<?> r1 = service.schedule(task1, 10, TimeUnit.SECONDS);
ScheduledFuture<?> r2 = service.schedule(task2, 8, TimeUnit.MINUTES);
```

English: The first task is scheduled 10 seconds in the future, whereas the second task is scheduled 8 minutes in the future.

Türkçe: İlk task gelecekte 10 saniye sonrasına, ikinci task ise 8 dakika sonrasına zamanlanır.

English: While these tasks are scheduled in the future, the actual execution may be delayed. For example, there may be no threads available to perform the tasks, at which point they will just wait in the queue. Also, if the `ScheduledExecutorService` is shut down by the time the scheduled task execution time is reached, then these tasks will be discarded.

Türkçe: Task'lar gelecekteki bir ana zamanlansa da gerçek yürütme daha geç başlayabilir; örneğin uygun thread yoksa kuyrukta beklerler. Kaynak metne göre, planlanan yürütme zamanına ulaşıldığında `ScheduledExecutorService` kapatılmışsa bu task'lar atılır.

Java 17 editör notu: Kaynak paragraftaki son cümle genel bir API garantisi değildir.

`ScheduledThreadPoolExecutor`, varsayılan olarak `shutdown()` öncesi gönderilmiş tek seferlik gecikmeli task'ları yürütmeye devam eder, ancak mevcut periyodik task'ları iptal eder. Davranış ilgili `shutdown policy` metotlarıyla değiştirilebilir.

English: Each of the `ScheduledExecutorService` methods is important and has real-world applications. For example, you can use the `schedule()` command to check on the state of cleaning a lion's cage. It can then send out notifications if it is not finished or even call `schedule()` to check again later.

Türkçe: `ScheduledExecutorService` metotlarının her birinin gerçek kullanım alanları vardır. Örneğin `schedule()` ile aslan kafesinin temizlenme durumu denetlenebilir; işlem bitmemişse bildirim gönderilebilir veya daha sonra yeniden kontrol etmek üzere tekrar `schedule()` çağrılabilir.

English: The last two methods in Table 13.4 might be a little confusing if you have not seen them before. Conceptually, they are similar as they both perform the same task repeatedly after an initial delay. The difference is related to the timing of the process and when the next task starts.

Türkçe: Tablo 13.4'teki son iki metot ilk bakışta karıştırılabilir. İkisi de ilk gecikmeden sonra aynı task'ı periyodik olarak çalıştırır; fark, sonraki çalışmanın zamanının nasıl hesaplandığıdır.

English: The `scheduleAtFixedRate()` method creates a new task and submits it to the executor every period, regardless of whether the previous task finished. The following example executes a `Runnable` task every minute, following an initial five-minute delay:

Türkçe: `scheduleAtFixedRate()` sonraki başlangıç zamanlarını sabit bir periyoda göre hesaplar. Aynı periyodik task'ın çalışmaları birbiriyle çakışmaz; bir çalışma periyottan uzun sürerse sonraki çalışma geç başlayabilir. Aşağıdaki örnek, beş dakikalık ilk gecikmeden sonra bir `Runnable` task'ını dakikada bir çalışacak biçimde zamanlar:

```
service.scheduleAtFixedRate(command, 5, 1, TimeUnit.MINUTES);
```

English: The `scheduleAtFixedRate()` method is useful for tasks that need to be run at specific intervals, such as checking the health of the animals once a day. Even if it takes two hours to examine an animal on Monday, this doesn't mean that Tuesday's exam should start any later in the day.

Türkçe: `scheduleAtFixedRate()`, hayvanların sağlığını günde bir kez kontrol etmek gibi sabit zaman aralıkları hedeflenen task'lar için yararlıdır. Pazartesi günü muayene iki saat sürse bile bu, salı günü muayenenin gün içinde iki saat daha geç başlaması gerektiği anlamına gelmez.

English: Bad things can happen with `scheduleAtFixedRate()` if each task consistently takes longer to run than the execution interval. Imagine if your boss came by your desk every minute and dropped off a piece of paper. Now imagine that it took you five minutes to read each piece of paper. Before long, you would be drowning in piles of paper. This is how an executor feels. Given enough time, the program would submit more tasks to the executor service than could fit in memory, causing the program to crash.

Türkçe: Her çalışma sürekli olarak belirlenen periyottan uzun sürüyorsa `scheduleAtFixedRate()` hedeflenen takvimin gerisinde kalır. Bunu, patronunuzun her dakika masanıza yeni bir kâğıt bırakmasına karşın her kâğıdı okumanın beş dakika sürmesine benzetebilirsiniz. Ancak Java 17'nin `ScheduledThreadPoolExecutor` uygulamasında aynı periyodik task'ın çalışmaları üst üste binmez ve her periyot için ayrı sınırsız task kopyaları oluşturulmaz; esas risk gecikme, kaynak tüketimi ve bekleyen başka task'ların aksamasıdır.

English: On the other hand, the `scheduleWithFixedDelay()` method creates a new task only after the previous task has finished. For example, if a task runs at 12:00 and takes five minutes to finish, with a period between executions of two minutes, the next task will start at 12:07.

Türkçe: Buna karşılık `scheduleWithFixedDelay()`, sonraki çalışmayı ancak önceki çalışma bittikten sonra verilen gecikme kadar bekleyerek başlatır. Örneğin bir task 12.00'de başlayıp beş dakikada biter ve çalışmalar arasındaki gecikme iki dakika ise sonraki çalışma 12.07'de başlar.

```
service.scheduleWithFixedDelay(task1, 0, 2, TimeUnit.MINUTES);
```

English: The `scheduleWithFixedDelay()` method is useful for processes that you want to happen repeatedly but whose specific time is unimportant. For example, imagine that we have a zoo cafeteria worker who periodically restocks the salad bar throughout the day. The process can take 20 minutes or more, since it requires the worker to haul a large number of items from the back room. Once the worker has filled the salad bar with fresh food, they don't need to check at some specific time, just after enough time has passed for it to become low on stock again.

Türkçe: `scheduleWithFixedDelay()`, yinelenmesi gereken fakat kesin başlangıç anı önemli olmayan işlemler için uygundur. Örneğin hayvanat bahçesi kafeteryasındaki bir çalışan gün boyunca salata barını belirli aralıklarla yeniden doldurabilir. Malzemeleri depodan taşımak gerektiği için işlem 20 dakika veya daha uzun sürebilir. Bar taze yiyeceklerle doldurulduktan sonra belirli bir saatte değil, stokun yeniden azalmasına yetecek süre geçince kontrol edilmesi yeterlidir.

Increasing Concurrency with Pools

English: All of our examples up until now have been with a single-thread executor, which, while interesting, weren't particularly useful. After all, the name of this chapter is "Concurrency," and you can't do a lot of that with a single-thread executor!

Türkçe: Şimdiye kadarki örneklerin tamamında single-thread executor kullandık. Bu yapı öğretici olsa da gerçek concurrency kapasitesi sınırlıdır; sonuçta bölümün adı "Concurrency" ve tek bir worker thread ile aynı anda çok fazla iş yürütülemez.

English: We now present three additional factory methods in the Executors class that act on a pool of threads rather than on a single thread. A thread pool is a group of pre-instantiated reusable threads that are available to perform a set of arbitrary tasks. Table 13.5 includes our two previous single-thread executor methods, along with the new ones that you should know for the exam.

Türkçe: Şimdi `Executors` sınıfında tek bir thread yerine thread pool üzerinde çalışan üç ek factory method ele alıyoruz. Thread pool (thread havuzu), farklı task'ları yürütmeye hazır, önceden oluşturulmuş ve yeniden kullanılabilen thread grubudur. Tablo 13.5, önceki iki single-thread executor metoduyla birlikte sınav için bilinmesi gereken yeni metotları da içerir.

English: TABLE 13.5 — Executors factory methods - `ExecutorService`
`newSingleThreadExecutor()` — Creates a single-threaded executor with one worker thread operating from an unbounded queue. Tasks are processed sequentially in submission order. - `ScheduledExecutorService newSingleThreadScheduledExecutor()` — Creates a single-threaded executor that can run commands after a delay or periodically. - `ExecutorService newCachedThreadPool()` — Creates a thread pool that creates new threads as needed and reuses previously constructed threads when they are available. - `ExecutorService newFixedThreadPool(int nThreads)` — Creates a thread pool that reuses a fixed number of threads operating from a shared unbounded queue. - `ScheduledExecutorService newScheduledThreadPool(int corePoolSize)` — Creates a thread pool that can run commands after a delay or periodically.

Türkçe: TABLO 13.5 — Executors factory metotları - `ExecutorService`
`newSingleThreadExecutor()` — Sınırsız bir kuyruktan task alan tek bir worker thread'e sahip executor oluşturur. Task'lar gönderildikleri sırayla, sequential (ardışık) olarak işlenir. - `ScheduledExecutorService newSingleThreadScheduledExecutor()` — Komutları belirli bir gecikmeden sonra veya periyodik olarak çalıştırabilen single-thread scheduled executor oluşturur. - `ExecutorService newCachedThreadPool()` — Gerekğinde yeni thread'ler oluşturan, uygun durumdaki önceden oluşturulmuş thread'leri yeniden kullanan bir thread pool oluşturur. - `ExecutorService newFixedThreadPool(int nThreads)` — Paylaşılan sınırsız kuyruktan task alan sabit sayıda thread'i yeniden kullanan bir thread pool oluşturur. - `ScheduledExecutorService newScheduledThreadPool(int corePoolSize)` — Komutları belirli bir gecikmeden sonra veya periyodik olarak çalıştırabilen bir thread pool oluşturur.

English: As shown in Table 13.5, these methods return the same instance types, `ExecutorService` and `ScheduledExecutorService`, that we used earlier in this chapter. In other words, all of our previous examples are compatible with these new pooled-thread executors!

Türkçe: Tablo 13.5'te görüldüğü gibi bu metotlar, bölümde daha önce kullanılan `ExecutorService` ve `ScheduledExecutorService` tiplerini döndürür. Başka bir deyişle, önceki örneklerin tamamı bu yeni pooled-thread executor'larla da uyumludur.

English: The difference between a single-thread and a pooled-thread executor is what happens when a task is already running. While a single-thread executor will wait for the thread to become available before running the next task, a pooled-thread executor can execute the next task concurrently. If the pool runs out of available threads, the task will be queued by the thread executor and wait to be completed.

Türkçe: Single-thread executor ile pooled-thread executor arasındaki temel fark, başka bir task zaten çalışırken ne olduğudur. Single-thread executor sıradaki task'ı çalıştırmak için tek thread'in boşalmasını bekler; pooled-thread executor ise uygun başka bir thread varsa sıradaki task'ı concurrent (eşzamanlı) çalıştırabilir. Havuzda uygun thread kalmazsa task kuyruğa alınır ve yürütülmeyi bekler.

Writing Thread-Safe Code

English: Thread-safety is the property of an object that guarantees safe execution by multiple threads at the same time. Since threads run in a shared environment and memory space, how do we prevent two threads from interfering with each other? We must organize access to data so that we don't end up with invalid or unexpected results.

Türkçe: Thread-safety (thread güvenliği), bir nesnenin aynı anda birden fazla thread tarafından güvenli kullanılabilmesini sağlayan özelliktir. Thread'ler aynı ortamı ve bellek alanını paylaştığından birbirlerinin işlemlerine müdahale edebilir. Geçersiz veya beklenmedik sonuçları önlemek için paylaşılan veriye erişimi doğru biçimde koordine etmemiz gerekir.

English: In this part of the chapter, we show how to use a variety of techniques to protect data, including atomic classes, synchronized blocks, the Lock framework, and cyclic barriers.

Türkçe: Bölümün bu kısmında veriyi korumak için atomic class'lar, `synchronized` bloklar, `Lock` framework'ü ve cyclic barrier'lar gibi çeşitli tekniklerin kullanımı gösterilir.

Understanding Thread-Safety

English: Imagine that our zoo has a program to count sheep, preferably one that won't put the zoo workers to sleep! Each zoo worker runs out to a field, adds a new sheep to the flock, counts the total number of sheep, and runs back to us to report the results. We present the following code to represent this conceptually, choosing a thread pool size so that all tasks can be run concurrently:

Türkçe: Hayvanat bahçemizde koyunları sayan, umarız çalışanları uyutmayan, bir program bulunduğunu düşünelim. Her çalışan tarlaya gider, sürüye bir koyun ekler, toplam koyun sayısını belirler ve sonucu bildirmek için geri döner. Bu fikri aşağıdaki kodla temsil ediyor ve bütün task'ların concurrent çalışabilmesi için yeterli büyüklükte bir thread pool seçiyoruz:

```
import java.util.concurrent.*;
public class SheepManager {
    private int sheepCount = 0;
    private void incrementAndReport() {
        System.out.print(++sheepCount+" ");
    }
    public static void main(String[] args) {
        ExecutorService service = Executors.newFixedThreadPool(20);
        try {
            SheepManager manager = new SheepManager();
            for(int i = 0; i < 10; i++)
                service.submit(() -> manager.incrementAndReport());
        } finally {
            service.shutdown();
        }
    }
}
```

English: What does this program output? You might think it will output numbers from 1 to 10, in order, but that is far from guaranteed. It may output in a different order. Worse yet, it may print some numbers twice and not print some numbers at all! The following are possible outputs of this program:

Türkçe: Bu program ne yazdırır? 1 ile 10 arasındaki sayıları sırayla yazdıracağını düşünebilirsiniz, fakat bu garanti edilmez. Sıra farklı olabilir; daha kötüsü, bazı sayılar iki kez yazdırılırken bazıları hiç yazdırılmayabilir. Aşağıdakiler olası çıktılarıdır:

English: 1 2 3 4 5 6 7 8 9 10 1 9 8 7 3 6 6 2 4 5 1 8 7 3 2 6 5 4 2 9 So, what went wrong? In this example, we use the pre-increment (++) operator to update the sheepCount variable. A problem occurs when two threads both execute the right side of the expression, reading the “old” value before either thread writes the “new” value of the variable. The two assignments become redundant; they both assign the same new value, with one thread overwriting the results of the other. Figure 13.4 demonstrates this problem with two threads, assuming that sheepCount has a starting value of 1.

Türkçe: Olası literal çıktılar: 1 2 3 4 5 6 7 8 9 10, 1 9 8 7 3 6 6 2 4 5 ve 1 8 7 3 2 6 5 4 2 9. Peki sorun nedir? Örnekte `sheepCount` değişkeni prefix increment (++) ile güncellenir. İki thread, ikisi de yeni değeri yazmadan önce eski değeri okuyabilir. Böylece ikisi de aynı yeni değeri yazar ve thread'lerden birinin güncellemesi diğerinkini ezer; buna lost update (kayıp güncelleme) denir. Şekil 13.4, `sheepCount` başlangıçta 1 iken bu durumu iki thread ile gösterir.

English: FIGURE 13.4 — Lack of thread synchronization: Thread 1 and Thread 2 each read `sheepCount` as 1 from shared memory, and each writes `sheepCount` as 2.

Türkçe: Şekil 13.4 — Thread synchronization eksikliği: Thread 1 ve Thread 2, shared memory içindeki `sheepCount` değerini 1 olarak okur ve ikisi de 2 yazar.

English: You can see in Figure 13.4 that both threads read and write the same values, causing one of the two ++sheepCount operations to be lost. Therefore, the increment operator ++ is not thread-safe. As you will see later in this chapter, the unexpected result of two tasks executing at the same time is referred to as a race condition.

Türkçe: Şekil 13.4'te iki thread aynı değerleri okuyup yazdığı için iki ++`sheepCount` işleminden biri kaybolur. Bu nedenle increment operator ++ bu kullanımda thread-safe değildir. İki task'ın zamanlamaya bağlı olarak beklenmedik sonuç üretmesine race condition (yarış durumu) denir.

English: Conceptually, the idea here is that some zoo workers may run faster on their way to the field but more slowly on their way back and report late. Other workers may get to the field last but somehow be the first ones back to report the results.

Türkçe: Kavramsal olarak bazı çalışanlar tarlaya daha hızlı gidip dönüşte yavaş kalabilir ve sonucu geç bildirebilir. Başka çalışanlar tarlaya en son ulaştıkları hâlde sonucu ilk bildirenler olabilir. Thread yürütme sırasının önceden garanti edilmemesi de buna benzer.

Accessing Data with volatile

English: The volatile keyword is used to guarantee that access to data within memory is consistent. For example, it is possible (albeit unlikely) that our SheepManager example using

Türkçe: `volatile` anahtar kelimesi, bir değişkene yapılan yazmanın diğer thread'ler tarafından görünür olmasını ve ilgili bellek erişimlerinin belirli bir ordering (sıralama) garantisine uymasını sağlar. Örneğin `SheepManager` içindeki

English: `++sheepCount` returns an unexpected value due to invalid memory access while the code is executing a critical section. Conceptually, this corresponds to one of our zoo employees tripping on the way back from the field and someone asking them the current number of sheep while they are still trying to get up!

Türkçe: `++sheepCount`, critical section yürütülürken thread'ler arası görünürlük veya yarış sorunları nedeniyle beklenmedik bir değer üretebilir. Kavramsal benzetmede bu, tarladan dönen bir çalışanın ayağı takılmışken ona güncel koyun sayısının sorulmasına benzer.

English: The volatile attribute ensures that only one thread is modifying a variable at one time and that data read among multiple threads is consistent. In this manner, we don't interrupt one of our zoo workers in the middle of running. So, does volatile provide thread-safety? Not exactly. Consider this replacement to our previous application:

Türkçe: Burada kritik ayırım şudur: `volatile`, aynı anda yalnızca bir thread'in değişkeni değiştirmesini **sağlamaz**; mutual exclusion sunmaz. Bir thread'in yaptığı yazmanın diğer thread'lerce görülmesini sağlar, fakat bileşik işlemleri atomic hâle getirmez. Dolayısıyla `volatile` tek başına thread-safety sağlamaz. Önceki uygulamadaki alanı şu şekilde değiştirdiğimizi düşünelim:

Java 17 teknik düzeltme: İngilizce kaynak paragrafındaki “only one thread is modifying” iddiası yanlıştır. `volatile` visibility ve ordering sağlar; mutual exclusion veya `++` gibi compound operation'lar için atomicity sağlamaz. Türkçe blok, bu düzeltmeyi bilinçli olarak yansıtır.

```
private volatile int sheepCount = 0;
private void incrementAndReport() {
    System.out.print(++sheepCount+" ");
}
```

English: Unfortunately, this code is not thread-safe and could still result in numbers being missed: 2 6 1 7 5 3 2 9 4 8 The reason this code is not thread-safe is that ++sheepCount is still two distinct operations. Put another way, if the increment operator represents the expression `sheepCount = sheepCount + 1`, then each read and write operation is thread-safe, but the combined operation is not. Referring back to our sheep example, we don't interrupt the employee while running, but we could still have multiple people in the field at the same time.

Türkçe: Bu kod thread-safe değildir ve yine 2 6 1 7 5 3 2 9 4 8 gibi eksik değerli bir çıktı üretebilir. Nedeni, ++sheepCount işleminin read-modify-write biçiminde bileşik bir işlem olmasıdır. `sheepCount = sheepCount + 1` olarak düşünülürse tek tek okuma ve yazmalar görünür olsa bile bütün işlem atomic değildir. Koyun benzetmesinde çalışanı koşarken yarıda kesmiyoruz, ancak aynı anda birden fazla çalışan hâlâ tarlada olabilir.

English: In practice, volatile is rarely used. We only cover it because it has been known to show up on the exam from time to time.

Türkçe: Pratikte `volatile` görece sınırlı durumlarda kullanılır. Burada ele alınmasının nedenlerinden biri de sınav sorularında zaman zaman karşınıza çıkabilmesidir.

Protecting Data with Atomic Classes

English: In our previous SheepManager applications, the same values were printed twice, with the highest counter being 9 instead of 10. As we saw, the increment operator ++ is not thread-safe, even when volatile is used. It is not thread-safe because the operation is not atomic, carrying out two tasks, read and write, that can be interrupted by other threads.

Türkçe: Önceki `SheepManager` uygulamalarında aynı değerler iki kez yazdırıldı ve en büyük sayaç değeri 10 yerine 9 oldu. Görüldüğü gibi ++, `volatile` kullanılsa bile thread-safe değildir. Çünkü işlem atomic değildir: okuma ve yazma adımları arasında başka thread'ler devreye girebilir.

English: Atomic is the property of an operation to be carried out as a single unit of execution without any interference from another thread. A thread-safe atomic version of the increment operator would perform the read and write of the variable as a single operation, not allowing any other threads to access the variable during the operation. Figure 13.5 shows the result of making the sheepCount variable atomic.

Türkçe: Atomic (atomik), bir işlemin başka bir thread'in araya giremeyeceği tek bir yürütme birimi gibi gerçekleşmesi özelliğidir. Artırmanın thread-safe atomic sürümü, değişkeni okuma ve yazmayı bölünemez tek bir işlem olarak gerçekleştirir. Şekil 13.5, `sheepCount` güncellemesinin atomic yapılmasının sonucunu gösterir.

English: In this case, any thread trying to access the sheepCount variable while an atomic operation is in process will have to wait until the atomic operation on the variable is complete. Conceptually, this is like setting a rule for our zoo workers that there can be only one employee in the field at a time, although they may not each report their results in order.

Türkçe: Atomic güncelleme sırasında başka bir thread aynı güncellemeyi araya sokamaz. Uygulama mutlaka blocking bir lock kullanmak zorunda değildir; contention durumunda işlem yeniden denenebilir. Kavramsal olarak bu, sonuçların bildirilme sırası değişebilse de tarlada aynı anda yalnızca bir çalışanın koyun sayısını güncellemesine izin vermeye benzer.

English: FIGURE 13.5 — Thread synchronization using atomic operations: Thread 1 atomically reads `sheepCount` as 1 and writes it as 2 in shared memory; Thread 2 then reads it as 2 and writes it as 3.

Türkçe: Şekil 13.5 — Atomic işlemlerle thread synchronization: Thread 1, shared memory içindeki `sheepCount` değerini atomic olarak 1'den 2'ye günceller; ardından Thread 2 değeri 2'den 3'e günceller.

English: Since accessing primitives and references is common in Java, the Concurrency API includes numerous useful classes in the `java.util.concurrent.atomic` package. Table 13.6 lists the atomic classes with which you should be familiar for the exam. As with many of the classes in the Concurrency API, these classes exist to make your life easier.

Türkçe: Primitive ve reference değerlerle concurrent çalışmak yaygın olduğundan `Concurrency API`, `java.util.concurrent.atomic` paketinde çeşitli yardımcı class'lar sunar. Sınav için bilinmesi gerekenler Tablo 13.6'da listelenmiştir.

English: TABLE 13.6 — Atomic classes - `AtomicBoolean` — A `boolean` value that may be updated atomically. - `AtomicInteger` — An `int` value that may be updated atomically. - `AtomicLong` — A `long` value that may be updated atomically.

Türkçe: TABLO 13.6 — Atomic class'lar - `AtomicBoolean` — Atomic olarak güncellenebilen bir `boolean` değer. - `AtomicInteger` — Atomic olarak güncellenebilen bir `int` değer. - `AtomicLong` — Atomic olarak güncellenebilen bir `long` değer.

English: How do we use an atomic class? Each class includes numerous methods that correspond to primitive built-in operators, such as assignment (`=`) and increment (`++`). TABLE 13.7 describes the common atomic methods you should know for the exam. The placeholder `type` is determined by the atomic class.

Türkçe: Atomic class nasıl kullanılır? Her class, assignment (`=`) ve increment (`++`) gibi primitive operatörlere karşılık gelen çok sayıda metot içerir. Sınav için bilinmesi gereken ortak atomic metotlar TABLO 13.7'de gösterilir. `type` yer tutucusu, kullanılan atomic class'a göre belirlenir.

English: In the following example, assume we import the atomic package and then update our `SheepManager` class with an `AtomicInteger`:

Türkçe: Aşağıdaki örnekte atomic paketin import edildiğini ve `SheepManager` class'ının `AtomicInteger` kullanacak biçimde güncellendiğini varsayalım:

```
private AtomicInteger sheepCount = new AtomicInteger(0);
private void incrementAndReport() {
    System.out.print(sheepCount.incrementAndGet() + " ");
}
```

English: TABLE 13.7 — Common atomic methods - `type get()` — Retrieves the current value. - `void set(type newValue)` — Sets the given value, equivalent to the assignment (=) operator. - `type getAndSet(type newValue)` — Atomically sets the new value and returns the old value. - `type incrementAndGet()` — For numeric classes, performs the atomic pre-increment operation equivalent to `++value`. - `type getAndIncrement()` — For numeric classes, performs the atomic post-increment operation equivalent to `value++`. - `type decrementAndGet()` — For numeric classes, performs the atomic pre-decrement operation equivalent to `--value`. - `type getAndDecrement()` — For numeric classes, performs the atomic post-decrement operation equivalent to `value--`.

Türkçe: TABLO 13.7 — Yaygın atomic metotlar - `type get()` — Geçerli değeri döndürür. - `void set(type newValue)` — Verilen değeri atar; assignment (=) operatörüne eşdeğerdir. - `type getAndSet(type newValue)` — Yeni değeri atomic olarak atar ve eski değeri döndürür. - `type incrementAndGet()` — Sayısal class'larda `++value` ile eşdeğer atomic prefix increment işlemini yapar. - `type getAndIncrement()` — Sayısal class'larda `value++` ile eşdeğer atomic postfix increment işlemini yapar. - `type decrementAndGet()` — Sayısal class'larda `--value` ile eşdeğer atomic prefix decrement işlemini yapar. - `type getAndDecrement()` — Sayısal class'larda `value--` ile eşdeğer atomic postfix decrement işlemini yapar.

English: How does this implementation differ from our previous examples? When we run this modification, we get varying output, such as the following:

Türkçe: Bu uygulama önceki örneklerden nasıl farklıdır? Değiştirilmiş program çalıştırıldığında aşağıdakiler gibi farklı sıralara sahip çıktılar elde edilebilir:

English: 2 3 1 4 5 6 7 8 9 10 1 4 3 2 5 6 7 8 9 10 1 4 3 5 6 2 7 8 10 9 Unlike our previous sample output, the numbers 1 through 10 will always be printed, although the order is still not guaranteed. Don't worry; we address that issue shortly. The key in this section is that using the atomic classes ensures that the data is consistent between workers and that no values are lost due to concurrent modifications.

Türkçe: Olası literal çıktılar: 2 3 1 4 5 6 7 8 9 10, 1 4 3 2 5 6 7 8 9 10 ve 1 4 3 5 6 2 7 8 10 9. Önceki örnekten farklı olarak sıra hâlâ garanti edilmese de 1 ile 10 arasındaki bütün sayılar birer kez yazdırılır. Atomic class kullanımı, concurrent güncellemeler yüzünden değer kaybolmasını önler; yazdırma sırasını tek başına garanti etmez.

Improving Access with synchronized Blocks

English: While atomic classes are great at protecting a single variable, they aren't particularly useful if you need to execute a series of commands or call a method. For example, we can't use them to update two atomic variables at the same time. How do we improve the results so that each worker is able to increment and report the results in order?

Türkçe: Atomic class'lar tek bir değişken üzerindeki işlemleri korumada çok kullanışlıdır; ancak bir komut dizisinin veya birden fazla değişkeni kapsayan invariant'ın tamamını tek işlem hâline getirmez. Örneğin iki ayrı atomic değişkeni birlikte tek bir atomic işlem olarak güncelleyemezler. Her çalışanın sayacı artırıp sonucu sırayla bildirmesini nasıl sağlayabiliriz?

English: The most common technique is to use a monitor to synchronize access. A monitor, also called a lock, is a structure that supports mutual exclusion, which is the property that at most one thread is executing a particular segment of code at a given time.

Türkçe: En yaygın teknik, erişimi synchronize etmek için bir monitor kullanmaktır. Lock olarak da anılan monitor, mutual exclusion (karşılıklı dışlama) sağlar: belirli bir anda ilgili kritik kod bölümünü en fazla bir thread yürütür.

English: In Java, any Object can be used as a monitor, along with the synchronized keyword, as shown in the following example:

Türkçe: Java'da herhangi bir **Object**, aşağıdaki örnekte olduğu gibi **synchronized** anahtar kelimesiyle monitor olarak kullanılabilir:

```
var manager = new SheepManager();
synchronized(manager) {
    // Work to be completed by one thread at a time
}
```

English: This example is referred to as a synchronized block. Each thread that arrives will first check if any threads are already running the block. If the lock is not available, the thread will transition to a BLOCKED state until it can “acquire the lock.” If the lock is available (or the thread already holds the lock), the single thread will enter the block, preventing all other threads from entering. Once the thread finishes executing the block, it will release the lock, allowing one of the waiting threads to proceed.

Türkçe: Bu yapıya **synchronized** block denir. Her thread bloğa girmeden önce ilgili monitor lock'unu edinmeye çalışır. Lock başka bir thread'deyse **BLOCKED** durumuna geçer. Lock uygunsa veya reentrant biçimde zaten aynı thread tarafından tutuluyorsa thread bloğa girer; aynı monitor üzerinde bekleyen diğer thread'ler giremez. Thread bloktan çıkınca lock'u serbest bırakır ve bekleyen thread'lerden biri ilerleyebilir.

English: To synchronize access across multiple threads, each thread must have access to the same Object. If each thread synchronizes on different objects, the code is not thread-safe.

Türkçe: Birden fazla thread'in erişimini gerçekten synchronize etmek için hepsinin **aynı monitor nesnesini** kullanması gerekir. Her thread farklı bir nesne üzerinde synchronize olursa birbirlerini dışlamazlar ve kod bu nedenle thread-safe olmaz.

English: Let's revisit our SheepManager example that used ++sheepCount and see whether we can improve the results so that each worker increments and outputs the counter in order. Let's say that we replaced our for() loop with the following implementation:

Türkçe: ++sheepCount kullanan **SheepManager** örneğine dönelim ve her çalışanın sayacı artırıp sonucu sırayla yazdırmasını sağlamaya çalışalım. **for()** loop'unu aşağıdaki uygulamayla değiştirdiğimizi varsayalım:

```
for(int i = 0; i < 10; i++) {
    synchronized(manager) {
        service.submit(() -> manager.incrementAndReport());
    }
}
```

English: Does this solution fix the problem? No, it does not! Can you spot the problem? We've synchronized the creation of the threads but not the execution of the threads. In this example, the threads would be created one at a time, but they might all still execute and perform their work simultaneously, resulting in the same type of output that you saw earlier. We did say diagnosing and resolving thread problems is difficult in practice!

Türkçe: Bu çözüm sorunu gidermez. `synchronized(manager)` yalnızca main thread'in `submit()` çağrılarını sıraya koyar; gönderilen task'ların daha sonra executor thread'lerinde yürütülmesini synchronize etmez. Task'lar yine concurrent çalışıp önceki türde hatalı çıktılar üretebilir. Task gönderimini serial hâle getirmek, task execution'ı serial hâle getirmek değildir.

English: We now present a corrected version of the SheepManager class that orders the workers:

Türkçe: Şimdi çalışanların sayaç güncellemelerini sıralayan düzeltilmiş `SheepManager` class'ını görelim:

```
import java.util.concurrent.*;
public class SheepManager {
    private int sheepCount = 0;
    private void incrementAndReport() {
        synchronized(this) {
            System.out.print(++sheepCount+" ");
        }
    }
    public static void main(String[] args) {
        ExecutorService service = Executors.newFixedThreadPool(20);
```

```
    try {
        var manager = new SheepManager();
        for(int i = 0; i < 10; i++)
            service.submit(() -> manager.incrementAndReport());
    } finally {
        service.shutdown();
    } }
```

English: When this code executes, it will consistently output the following:

Türkçe: Bu kod her çalıştırıldığında aşağıdaki çıktıyı üretir:

English: 1 2 3 4 5 6 7 8 9 10 Although all threads are still created and executed at the same time, they each wait at the synchronized block for the worker to increment and report the result before entering. In this manner, each zoo worker waits for the previous zoo worker to come back before running out on the field. While it's random which zoo worker will run out next, it is guaranteed that there will be at most one on the field and that the results will be reported in order.

Türkçe: Literal çıktı: 1 2 3 4 5 6 7 8 9 10. Task'lar concurrent çalışsa da her thread, `synchronized(this)` bloğuna girmeden önce aynı `manager` nesnesinin monitor'ünü bekler. Lock'u sırada hangi thread'in alacağı belirsizdir; fakat kritik bölümde aynı anda en fazla bir thread bulunur ve artırma ile yazdırma birlikte korunduğundan sayılar sırayla raporlanır.

English: We could have synchronized on any object, as long as it was the same object. For example, the following code snippet would also work:

Türkçe: Bütün thread'ler aynı nesneyi kullandığı sürece başka bir nesne üzerinde de synchronize edilebilir. Örneğin aşağıdaki code snippet de çalışır:

```
private final Object herd = new Object();
private void incrementAndReport() {
    synchronized(herd) {
        System.out.print(++sheepCount+" ");
    }
}
```

English: Although we didn't need to make the herd variable final, doing so ensures that it is not reassigned after threads start using it.

Türkçe: `herd` değişkeninin `final` olması zorunlu değildir; ancak `final`, thread'ler bu monitor'ü kullanmaya başladıktan sonra referansın başka bir nesneye atanmasını önler.

Synchronizing on Methods

English: In the previous example, we established our monitor using `synchronized(this)` around the body of the method. Java provides a more convenient compiler enhancement for doing so. We can add the `synchronized` modifier to any instance method to synchronize automatically on the object itself. For example, the following two method definitions are equivalent:

Türkçe: Önceki örnekte method body'yi `synchronized(this)` ile çevreleyerek monitor oluşturduk. Java bunun için daha kısa bir sözdizimi sunar: Bir instance method'a `synchronized` modifier'ı eklendiğinde method otomatik olarak `this` nesnesinin monitor'ünü kullanır. Aşağıdaki iki method tanımı eşdeğerdir:

```
void sing() {
    synchronized(this) {
        System.out.print("La la la!");
    }
}

synchronized void sing() {
    System.out.print("La la la!");
}
```

English: The first uses a synchronized block, whereas the second uses the synchronized method modifier. Which you use is completely up to you.

Türkçe: İlk tanım `synchronized` block, ikincisi `synchronized` method modifier'ı kullanır. İkisi aynı monitor üzerinde eşdeğer koruma sağlar; hangisinin seçileceği ihtiyaca bağlıdır.

English: We can also apply the synchronized modifier to static methods. What object is used as the monitor when we synchronize on a static method? The class object, of course! For example, the following two methods are equivalent for static synchronization inside our SheepManager class:

Türkçe: `synchronized` modifier'ı `static` method'lara da uygulanabilir. Bu durumda monitor bir instance değil, ilgili `Class` nesnesidir. `SheepManager` içindeki aşağıdaki iki static synchronization biçimi eşdeğerdir:

```
static void dance() {  
    synchronized(SheepManager.class) {  
        System.out.print("Time to dance!");  
    }  
}  
  
static synchronized void dance() {  
    System.out.print("Time to dance!");  
}
```

English: As before, the first uses a synchronized block, with the second example using the synchronized modifier. You can use static synchronization if you need to order thread access across all instances rather than a single instance.

Türkçe: İlk örnek `SheepManager.class` üzerinde `synchronized` block, ikincisi `synchronized` modifier kullanır. Static synchronization bütün instance'lar için aynı class-level lock'u kullanır; instance `synchronized` method ise yalnızca kendi `this` nesnesini kilitler. Bu iki lock birbirinden farklıdır.

Understanding the Lock Framework

English: A synchronized block supports only a limited set of functionality. For example, what if we want to check whether a lock is available and, if it is not, perform some other task? Furthermore, if the lock is never available and we synchronize on it, we might wait forever.

Türkçe: `synchronized` block yalnızca sınırlı işlev sunar. Örneğin lock'un uygun olup olmadığını denetleyip uygun değilse başka bir iş yapmak isteyebiliriz. Üstelik lock hiç uygun hâle gelmezse ona girmeye çalışan thread süresiz bekleyebilir.

English: The Concurrency API includes the Lock interface, which is conceptually similar to using the synchronized keyword but with a lot more bells and whistles. Instead of synchronizing on any Object, though, we can “lock” only on an object that implements the Lock interface.

Türkçe: `Concurrency API`, kavramsal olarak `synchronized` kullanımına benzeyen fakat daha gelişmiş özellikler sunan `Lock` interface'ini içerir. `synchronized` herhangi bir `Object` monitor'üyle kullanılabilirken açık kilitleme işlemleri yalnızca `Lock` interface'ini implement eden nesneler üzerinde yapılır.

Applying a ReentrantLock

English: The Lock interface is pretty easy to use. When you need to protect a piece of code from multithreaded processing, create an instance of Lock that all threads have access to. Each thread then calls lock() before it enters the protected code and calls unlock() before it exits the protected code.

Türkçe: Lock interface'inin temel kullanımı basittir. Multithreaded erişimden korunacak kod için bütün thread'lerin paylaştığı bir Lock örneği oluşturulur. Her thread korunan bölüme girmeden önce lock(), bölümden çıkarken unlock() çağırır.

English: For contrast, the following shows two implementations, one with a synchronized block and one with a Lock instance. While longer, the Lock solution has a number of features not available to the synchronized block.

Türkçe: Karşılaştırma için aşağıda biri synchronized block, diğeri Lock örneği kullanan iki uygulama gösterilir. Lock çözümü daha uzundur, ancak synchronized block'ta bulunmayan çeşitli özellikler sağlar.

```
// Implementation #1 with a synchronized block
Object object = new Object();
synchronized(object) {
    // Protected code
}
```

```
// Implementation #2 with a Lock
Lock lock = new ReentrantLock();
try {
    lock.lock();
    // Protected code
} finally {
    lock.unlock();
}
```

English: These two implementations are conceptually equivalent. The ReentrantLock class is a simple monitor that implements the Lock interface and supports mutual exclusion. In other words, at most one thread is allowed to hold a lock at any given time.

Türkçe: Bu iki uygulama kavramsal olarak eşdeğerdir. ReentrantLock, Lock interface'ini implement eden ve mutual exclusion sağlayan bir lock'tur. Belirli bir anda lock'un sahibi en fazla bir thread olabilir; reentrant olduğu için aynı sahibi lock'u birden çok kez edinebilir.

English: While certainly not required, it is a good practice to use a try/finally block with Lock instances. Doing so ensures that any acquired locks are properly released.

Türkçe: Sözdizimsel olarak zorunlu olmasa da Lock ile try/finally kullanmak iyi pratiktir. Böylece korunan kod normal tamamlansa da exception atsa da edinilmiş lock finally içinde serbest bırakılır.

English: The `ReentrantLock` class ensures that once a thread has called `lock()` and obtained the lock, all other threads that call `lock()` will wait until the first thread calls `unlock()`. Which thread gets the lock next depends on the parameters used to create the Lock object.

Türkçe: Bir thread `ReentrantLock` üzerinde `lock()` çağırıp lock'u edindiğinde, diğer thread'lerin `lock()` çağrıları sahibi thread gerekli `unlock()` çağrılarını yapıp lock'u tamamen serbest bırakana kadar bekler. Lock'u sırada hangi thread'in alacağı, `Lock` nesnesinin oluşturulma biçimine ve scheduling'e bağlıdır.

English: The `ReentrantLock` class includes a constructor that takes a single boolean and sets a "fairness" parameter. If the parameter is set to true, the lock will usually be granted to each thread in the order in which it was requested. It is false by default when using the no-argument constructor. In practice, you should enable fairness only when ordering is absolutely required, as it could lead to a significant slowdown.

Türkçe: `ReentrantLock(boolean fair)` constructor'ı fairness (adil sıra) seçeneğini ayarlar. Argüman `true` olduğunda contention altındaki lock genellikle en uzun süredir bekleyen thread'e verilir. No-argument constructor'da fairness varsayılan olarak `false`'tur. Fairness performansı belirgin biçimde düşürebileceğinden yalnızca erişim sırası gerçekten gerekli olduğunda etkinleştirilmelidir.

English: Besides always making sure to release a lock, you also need to be sure that you only release a lock that you have. If you attempt to release a lock that you do not have, you will get an exception at runtime.

Türkçe: Lock'u serbest bırakmayı unutmamak kadar yalnızca mevcut thread'in sahip olduğu lock'u serbest bırakmak da önemlidir. Sahibi olmayan bir thread `unlock()` çağırırsa çalışma zamanında `IllegalMonitorStateException` atılır.

```
Lock lock = new ReentrantLock();
lock.unlock(); // IllegalMonitorStateException
```

English: The `Lock` interface includes four methods you should know for the exam, as listed in Table 13.8.

Türkçe: `Lock` interface'inin sınav için bilinmesi gereken dört temel metodu Tablo 13.8'de listelenir.

Attempting to Acquire a Lock

English: While the `ReentrantLock` class allows you to wait for a lock, it so far suffers from the same problem as a `synchronized` block. A thread could end up waiting forever to obtain a lock. Luckily, Table 13.8 includes two additional methods that make the `Lock` interface a lot safer to use than a `synchronized` block.

Türkçe: Yalnızca `lock()` kullanıldığında `ReentrantLock`, `synchronized` block ile aynı süresiz bekleme riskini taşır: Bir thread lock'u edinmek için sonsuza kadar bekleyebilir. Tablo 13.8'deki iki `tryLock()` overload'u, beklemeyi önleme veya sınırlama olanağı sağlar.

English: TABLE 13.8 — Lock methods - `void lock()` — Requests the lock and blocks until the lock is acquired. - `void unlock()` — Releases the lock. - `boolean tryLock()` — Requests the lock and returns immediately. Its result indicates whether the lock was acquired. - `boolean tryLock(long timeout, TimeUnit unit) throws InterruptedException` — Requests the lock and blocks for the specified time or until the lock is acquired. Its result indicates whether the lock was acquired.

Türkçe: TABLO 13.8 — Lock metotları - `void lock()` — Lock'u ister ve lock edinilene kadar thread'i bekletir. - `void unlock()` — Mevcut thread'in sahip olduğu lock'u serbest bırakır. - `boolean tryLock()` — Lock'u edinmeyi dener ve hemen döner; sonucu lock'un edinilip edinilmediğini gösterir. - `boolean tryLock(long timeout, TimeUnit unit) throws InterruptedException` — Lock edinilene veya belirtilen süre dolana kadar bekler; lock edinildiyse `true`, süre dolduysa `false` döndürür. Bekleme sırasında thread kesilirse `InterruptedException` atar.

English: For convenience, we use the following `printHello()` method for the code in this section:

Türkçe: Kolaylık amacıyla bu bölümdeki örneklerde aşağıdaki `printHello()` metodu kullanılır:

```
public static void printHello(Lock lock) {
    try {
        lock.lock();
        System.out.println("Hello");
    } finally {
        lock.unlock();
    }
}
```

English: tryLock() The `tryLock()` method will attempt to acquire a lock and immediately return a boolean result indicating whether the lock was obtained. Unlike the `lock()` method, it does not wait if another thread already holds the lock. It returns immediately, regardless of whether a lock is available.

Türkçe: tryLock() lock'u edinmeyi dener ve edinip edinmediğini belirten `boolean` sonucu hemen döndürür. `lock()` metodundan farklı olarak lock başka bir thread'deyse beklemes; lock uygun olsa da olmasa da çağrı hemen tamamlanır.

English: The following is a sample implementation using the `tryLock()` method:

Türkçe: Aşağıda `tryLock()` kullanan örnek bir uygulama verilmiştir:

```
Lock lock = new ReentrantLock();
new Thread(() -> printHello(lock)).start();
if(lock.tryLock()) {
    try {
        System.out.println("Lock obtained, entering protected code");
    } finally {
        lock.unlock();
    }
} else {
    System.out.println("Unable to acquire lock, doing something else");
}
```

English: When you run this code, it could produce either the if or else message, depending on the order of execution. It will always print Hello, though, as the call to lock() in printHello() will wait indefinitely for the lock to become available. A fun exercise is to insert some Thread.sleep() delays into this snippet to encourage a particular message to be displayed.

Türkçe: Execution order'a bağlı olarak bu kod if ya da else dalındaki mesajı yazdırabilir. Bununla birlikte printHello() içindeki lock() lock uygun hâle gelene kadar süresiz beklediğinden literal Hello çıktısı her durumda yazdırılır. Belirli bir interleaving'i teşvik etmek için örneğe Thread.sleep() gecikmeleri eklenebilir.

English: Like lock(), the tryLock() method should be used with a try/finally block. Fortunately, you need to release the lock only if it was successfully acquired. For this reason, it is common to use the output of tryLock() in an if statement, so that unlock() is called only when the lock is obtained.

Türkçe: lock() gibi tryLock() da edinilmiş lock'un finally içinde bırakılacağı bir yapı ile kullanılmalıdır. Fakat unlock() yalnızca tryLock() başarıyla lock edindiye çağırılmalıdır. Bu nedenle dönüş değerini if ile kontrol etmek yaygındır.

English: It is imperative that your program always check the return value of the tryLock() method. It tells your program whether it is safe to proceed with the operation and whether the lock needs to be released later.

Türkçe: Program mutlaka tryLock() dönüş değerini kontrol etmelidir. Bu değer hem korunan işleme girmenin güvenli olup olmadığını hem de daha sonra unlock() çağırılması gerekip gerekmediğini bildirir.

English: tryLock(long, TimeUnit) The Lock interface includes an overloaded version of tryLock(long, TimeUnit) that acts like a hybrid of lock() and tryLock(). Like the other two methods, if a lock is available, it will immediately return with it. If a lock is unavailable, though, it will wait up to the specified time limit for the lock.

Türkçe: tryLock(long, TimeUnit), lock() ile parametresiz tryLock() arasında bir davranış sunan overload'dur. Lock uygunsa hemen ediniş true döndürür; uygun değilse belirtilen zaman sınırına kadar bekler. Süre dolarsa false, beklerken kesilirse checked InterruptedException atar.

English: The following code snippet uses the overloaded version of tryLock(long, TimeUnit):

Türkçe: Aşağıdaki code snippet tryLock(long, TimeUnit) overload'unu kullanır. İçinde bulunduğu method checked InterruptedException'ı catch veya declare etmelidir:

```
Lock lock = new ReentrantLock();
new Thread(() -> printHello(lock)).start();
if(lock.tryLock(10, TimeUnit.SECONDS)) {
    try {
        System.out.println("Lock obtained, entering protected code");
    } finally {
        lock.unlock();
    }
} else {
    System.out.println("Unable to acquire lock, doing something else");
}
```

English: The code is the same as before, except this time, one of the threads waits up to 10 seconds to acquire the lock.

Türkçe: Kod önceki örnekle aynıdır; ancak bu kez thread'lerden biri lock'u edinmek için en fazla 10 saniye bekler.

Acquiring the Same Lock Twice

English: The `ReentrantLock` class maintains a counter of the number of times a lock has been successfully granted to a thread. To release the lock for other threads to use, `unlock()` must be called the same number of times the lock was granted. The following code snippet contains an error. Can you spot it?

Türkçe: `ReentrantLock`, lock'un sahibi thread tarafından başarıyla kaç kez edinildiğini gösteren bir hold count tutar. Lock'un başka thread'lere tamamen bırakılabilmesi için başarılı her reentrant edinime karşılık bir `unlock()` çağrılmalıdır. Aşağıdaki code snippet bu kurala aykırı bir hata içerir:

```
Lock lock = new ReentrantLock();
if(lock.tryLock()) {
    try {
        lock.lock();
        System.out.println("Lock obtained, entering protected code");
    } finally {
        lock.unlock();
    }
}
```

English: The thread obtains the lock twice but releases it only once. You can verify this by spawning a new thread after this code runs that attempts to obtain a lock. The following prints false:

Türkçe: Thread lock'u iki kez edinir fakat yalnızca bir kez serbest bırakır; hold count 1 olarak kaldığından lock hâlâ bu thread'e aittir. Ardından oluşturulan başka bir thread lock'u edinmeyi denerse aşağıdaki kod literal `false` yazdırır:

```
new Thread(() -> System.out.print(lock.tryLock())).start(); // false
```

English: It is critical that you release a lock the same number of times it is acquired! For calls with `tryLock()`, you need to call `unlock()` only if the method returned true.

Türkçe: Lock'u başarıyla edinildiği kadar serbest bırakmak kritik önemdedir. `tryLock()` kullanılan her edinim denemesinde `unlock()` yalnızca metot `true` döndürmüştse çağrılmalıdır.

Reviewing the Lock Framework

English: To review, the `ReentrantLock` class supports the same features as a synchronized block while adding a number of improvements: • Ability to request a lock without blocking. • Ability to request a lock while blocking for a specified amount of time. • A lock can be created with a fairness property, in which the lock is granted to threads in the order in which it was requested.

Türkçe: Özetle `ReentrantLock`, `synchronized` block'un mutual exclusion özelliğini destekler ve buna şu olanakları ekler: blocking olmadan lock isteme, en fazla belirli bir süre bekleyerek lock isteme ve fairness seçeneğiyle bekleyen thread'lere yaklaşık istek sırasına göre öncelik verme.

English: While not on the exam, `ReentrantReadWriteLock` is a really useful class. It includes separate locks for reading and writing data and is useful on data structures where reads are far more common than writes. For example, if you have a thousand threads reading data but only one thread writing data, this class can help you maximize concurrent access.

Türkçe: Sınav kapsamında olmasa da `ReentrantReadWriteLock` yararlı bir class'tır. Okuma ve yazma için ayrı lock'lar sağlar; okumaların yazmalardan çok daha sık olduğu veri yapılarında özellikle kullanışlıdır. Örneğin bin thread veriyi okurken yalnızca bir thread yazıyorsa concurrent erişim kapasitesini artırabilir.

Orchestrating Tasks with a `CyclicBarrier`

English: We started the thread-safety topic by discussing protecting individual variables and then moved on to blocks of code and locks. We complete our discussion of thread-safety by showing how to orchestrate complex tasks with many steps.

Türkçe: Thread-safety konusuna tek tek değişkenleri koruyarak başladık; ardından kod bloklarına ve lock'lara geçtik. Şimdi çok aşamalı karmaşık task'ların nasıl koordine edileceğini ele alarak konuyu tamamlıyoruz.

English: Our zoo workers are back, and this time they are cleaning pens. Imagine a lion pen that needs to be emptied, cleaned, and then refilled with the lions. To complete the task, we have assigned four zoo workers. Obviously, we don't want to start cleaning the cage while a lion is roaming in it, lest we end up losing a zoo worker! Furthermore, we don't want to let the lions back into the pen while it is still being cleaned.

Türkçe: Hayvanat bahçesi çalışanları bu kez hayvan bölmelerini temizliyor. Önce boşaltılması, sonra temizlenmesi ve en son aslanların geri konması gereken bir aslan bölmesi düşünün. İş için dört çalışan görevlendirilmiştir. İçinde aslan varken temizliğe başlanmamalı; temizlik bitmeden de aslanlar bölmeye geri alınmamalıdır.

English: We could have all of the work completed by a single worker, but this would be slow and ignore the fact that we have three zoo workers standing by to help. A better solution would be to have all four zoo employees work concurrently, pausing between the end of one set of tasks and the start of the next.

Türkçe: Bütün işi tek bir çalışan tamamlayabilir; ancak bu yavaş olur ve yardıma hazır diğer üç çalışan kullanılmamış olur. Daha iyi çözüm, dört çalışanın concurrent çalışması ve her task aşamasının bitişi ile sonraki aşamanın başlangıcı arasında birbirlerini beklemesidir. Barrier kullanan task sayısı kadar task'ın aynı anda ilerleyebilmesi gerekir; yetersiz worker thread bulunan sabit havuz, task'ların bariyerde beklerken kuyruktaki diğer task'ları aç bırakmasına ve programın takılmasına yol açabilir.

English: To coordinate these tasks, we can use the `CyclicBarrier` class:

Türkçe: Bu task'ları koordine etmek için `CyclicBarrier` class'ını kullanabiliriz:

```
import java.util.concurrent.*;
public class LionPenManager {
    private void removeLions() { System.out.println("Removing lions"); }
    private void cleanPen() { System.out.println("Cleaning the pen"); }
    private void addLions() { System.out.println("Adding lions"); }
    public void performTask() {
        removeLions();
        cleanPen();
        addLions();
    }
    public static void main(String[] args) {
        var service = Executors.newFixedThreadPool(4);
        try {
            var manager = new LionPenManager();
            for (int i = 0; i < 4; i++)
                service.submit(() -> manager.performTask());
        } finally {
            service.shutdown();
        } } }
```

English: The following is sample output based on this implementation:

Türkçe: Bu uygulamanın olası çıktılarından biri aşağıda verilmiştir:

English: Removing lions Removing lions Cleaning the pen Adding lions Removing lions Cleaning the pen Adding lions Removing lions Cleaning the pen Adding lions Cleaning the pen Adding lions Although the results are ordered within a single thread, the output is entirely random among multiple workers. We see that some lions are still being removed while the cage is being cleaned, and other lions are added before the cleaning process is finished. Let's hope none of the zoo workers get eaten!

Türkçe: Literal örnek çıktı: Removing lions Removing lions Cleaning the pen Adding lions Removing lions Cleaning the pen Adding lions Removing lions Cleaning the pen Adding lions Cleaning the pen Adding lions. Tek bir thread içindeki işlemler sıralı olsa da farklı çalışanların çıktıları rastgele iç içe geçebilir. Bazı aslanlar hâlâ çıkarılırken temizlik başlayabilir veya temizlik bitmeden başka aslanlar geri konabilir; bu nedenle aşamalar arasında coordination gerekir.

English: We can improve these results by using the `CyclicBarrier` class. The `CyclicBarrier` takes in its constructors a limit value, indicating the number of threads to wait for. As each thread finishes, it calls the `await()` method on the cyclic barrier. Once the specified number of threads have each called `await()`, the barrier is released, and all threads can continue.

Türkçe: Bu sonucu `CyclicBarrier` ile düzeltebiliriz. Constructor'daki parties (katılımcı sayısı), her barrier trip'inde kaç thread'in bekleneceğini belirtir. Her thread ilgili aşamayı bitirince bariyerde `await()` çağırır. Gerekli sayıda thread `await()` noktasına ulaştığında varsa barrier action bir kez ve bekleyen thread'ler serbest bırakılmadan önce çalışır; ardından bütün thread'ler sonraki aşamaya geçer. `int await() throws InterruptedException, BrokenBarrierException` imzasına sahip olduğundan kod bu checked exception'ları handle etmek veya declare etmek zorundadır.

```
import java.util.concurrent.*;

public class LionPenManager {
    private void removeLions() { System.out.println("Removing lions"); }
    private void cleanPen() { System.out.println("Cleaning the pen"); }
    private void addLions() { System.out.println("Adding lions"); }
    public void performTask(CyclicBarrier c1, CyclicBarrier c2) {
        try {
            removeLions();
            c1.await();
            cleanPen();
            c2.await();
            addLions();
        } catch (InterruptedException | BrokenBarrierException e) {
            // Handle checked exceptions here
        }
    }

    public static void main(String[] args) {
        var service = Executors.newFixedThreadPool(4);
        try {
            var manager = new LionPenManager();
            var c1 = new CyclicBarrier(4);
            var c2 = new CyclicBarrier(4,
                () -> System.out.println("*** Pen Cleaned!"));
            for (int i = 0; i < 4; i++)
                service.submit(() -> manager.performTask(c1, c2));
        } finally {
            service.shutdown();
        }
    }
}
```

English: The following is sample output based on this revised implementation of our `LionPenManager` class: Removing lions Removing lions Removing lions Removing lions Cleaning the pen Cleaning the pen

Türkçe: Güncellenmiş `LionPenManager` uygulamasının örnek çıktısının bu sayfadaki literal bölümü şöyledir: Removing lions Removing lions Removing lions Removing lions Cleaning the pen Cleaning the pen.

English: Cleaning the pen Cleaning the pen *** Pen Cleaned! Adding lions Adding lions Adding lions Adding lions As you can see, all of the results are now organized. Removing the lions happens in one step, as does cleaning the pen and adding the lions back in. In this example, we used two different constructors for our `CyclicBarrier` objects, the latter of which executes a `Runnable` instance upon completion.

Türkçe: Literal çıktının devamı `Cleaning the pen` iki kez, `*** Pen Cleaned!` bir kez ve `Adding lions` dört kez şeklindedir. Görüldüğü gibi sonuçlar artık aşamalar hâlinde düzenlenmiştir: önce aslanlar çıkarılır, sonra bölme temizlenir ve son olarak aslanlar geri konur. Örnekte iki farklı `CyclicBarrier` constructor'ı kullanılmıştır; ikincisi, bariyer açıldığında bir `Runnable` çalıştırır.

English: The `CyclicBarrier` class allows us to perform complex, multithreaded tasks while all threads stop and wait at logical barriers. This solution is superior to a single-threaded solution, as the individual tasks, such as removing the lions, can be completed in parallel by all four zoo workers.

Türkçe: `CyclicBarrier`, bütün thread'lerin mantıksal eşiklerde durup birbirini beklediği karmaşık ve çok thread'li görevleri koordine etmemizi sağlar. Aslanları çıkarma gibi tek tek işler dört hayvanat bahçesi çalışanı tarafından paralel yürütülebildiği için bu yaklaşım, tek thread'li çözüme göre üstündür.

English: Reusing `CyclicBarrier` After a `CyclicBarrier` limit is reached (aka the barrier is broken), all threads are released, and the number of threads waiting on the `CyclicBarrier` goes back to zero. At this point, the `CyclicBarrier` may be used again for a new set of waiting threads. For example, if our `CyclicBarrier` limit is 5 and we have 15 threads that call `await()`, the `CyclicBarrier` will be activated a total of three times.

Türkçe: `CyclicBarrier`'i yeniden kullanma: Limit normal biçimde karşılandığında bütün thread'ler serbest bırakılır ve bariyerde bekleyen thread sayısı yeniden sıfır olur. Bundan sonra aynı `CyclicBarrier`, yeni bir thread grubu için tekrar kullanılabilir. Örneğin limit 5 ise ve 15 thread `await()` çağırırsa bariyer toplam üç kez açılır.

Java 17 editor notu: Kaynak paragraf normal limit karşılanmasını “barrier is broken” diye adlandırıyor. API bakımından bu normal bir `trip` durumudur; `isBroken()` bu nedenle `false` kalır. “Broken” durumu interruption, timeout veya barrier action failure gibi başarısızlıklarda oluşur.

Using Concurrent Collections

English: Besides managing threads, the Concurrency API includes interfaces and classes that help you coordinate access to collections shared by multiple tasks. By collections, we are of course referring to the Java Collections Framework that we introduced in Chapter 9, “Collections and Generics.” In this section, we demonstrate many of the concurrent classes available to you when using the Concurrency API.

Türkçe: `Concurrency API`, thread yönetiminin yanında, birden fazla task tarafından paylaşılan collection'lara erişimi koordine eden interface ve class'lar da içerir. Buradaki collection, Chapter 9 “Collections and Generics” bölümünde tanıtılan Java Collections Framework yapılarıdır. Bu bölümde `Concurrency API`'nin başlıca concurrent collection class'ları gösterilir.

Understanding Memory Consistency Errors

English: The purpose of the concurrent collection classes is to solve common memory consistency errors. A memory consistency error occurs when two threads have inconsistent views of what should be the same data. Conceptually, we want writes on one thread to be available to another thread if it accesses the concurrent collection after the write has occurred.

Türkçe: Concurrent collection class'larının amacı yaygın memory consistency hatalarını önlemektir. İki thread aynı olması gereken veriyi farklı görüyorsa memory consistency error oluşur. Kavramsal amaç, bir thread'in collection üzerinden publish ettiği element/value'nun onu daha sonra gözlemleyen thread'e güvenli biçimde aktarılmasıdır.

Java 17 editor notu: Garanti, yalnız “zamanca sonra herhangi bir collection erişimi” için global visibility anlamına gelmez. İlgili concurrent class'ın API contract'ında tanımlanan `put`, `offer` gibi publication operation'ı ile o value'yu gözlemleyen sonraki access/removal arasında happens-before ilişkisi aranır.

English: When two threads try to modify the same nonconcurrent collection, the JVM may throw a `ConcurrentModificationException` at runtime. In fact, it can happen with a single thread. Take a look at the following code snippet:

Türkçe: İki thread aynı nonconcurrent collection'ı değiştirmeye çalıştığında JVM, runtime'da `ConcurrentModificationException` atabilir. Hatta bu durum tek bir thread'le bile oluşabilir. Aşağıdaki kod parçasına bakalım:

```
var foodData = new HashMap<String, Integer>();
foodData.put("penguin", 1);
foodData.put("flamingo", 2);
for(String key: foodData.keySet())
    foodData.remove(key);
```

English: This snippet will throw a `ConcurrentModificationException` during the second iteration of the loop, since the iterator on `keySet()` is not properly updated after the first element is removed. Changing the first line to use a `ConcurrentHashMap` will prevent the code from throwing an exception at runtime.

Türkçe: Bu kod parçası, `keySet()` iterator'ı ilk element kaldırıldıktan sonra uygun biçimde güncellenmediği için loop'un ikinci iteration'ında `ConcurrentModificationException` atar. İlk satırda `ConcurrentHashMap` kullanılması, kodun runtime'da bu exception'ı atmasını önler.

```
var foodData = new ConcurrentHashMap<String, Integer>();
```

English: Although we don't usually modify a loop variable, this example highlights the fact that the `ConcurrentHashMap` is ordering read/write access such that all access to the class is consistent. In this code snippet, the iterator created by `keySet()` is updated as soon as an object is removed from the Map.

Türkçe: Normalde iteration sırasında collection'ı değiştirmeyiz; bu örnek, `ConcurrentHashMap` kullanımında read/write erişimlerinin memory-consistency garantileriyle koordine edildiğini vurgular. Kaynak, `keySet()` iterator'ının öge silinir silinmez güncellendiğini söyler.

Java 17 editor notu: `ConcurrentHashMap` iterator'ı **weakly consistent**'tir; sonradan yapılan değişikliği görebilir veya görmeyebilir. Garanti edilen nokta, iteration sırasında `ConcurrentModificationException` atmamasıdır. Concurrent collection'lar her thread'e aynı anda özdeş bir global snapshot sunmaz; thread-safe operation'lar ve API'nin belirttiği per-operation visibility garantilerini sağlar.

English: The concurrent classes were created to help avoid common issues in which multiple threads are adding and removing objects from the same collections. At any given instance, all threads should have the same consistent view of the structure of the collection.

Türkçe: Concurrent class'lar, birden fazla thread'in aynı collection'a element ekleyip çıkarmasından doğan yaygın sorunları önlemeye yardımcı olmak için oluşturulmuştur. Buradaki "consistent view", bütün thread'lerin her anda aynı global snapshot'ı görmesi değil; operation'ların class contract'ına uygun, thread-safe ve tanımlı visibility semantics ile çalışmasıdır.

Working with Concurrent Classes

English: You should use a concurrent collection class any time you have multiple threads modify a collection outside a synchronized block or method, even if you don't expect a concurrency problem. Without the concurrent collections, multiple threads accessing a collection could result in an exception being thrown or, worse, corrupt data!

Türkçe: Concurrency sorunu beklemesiniz bile birden fazla thread, `synchronized` block veya method dışında bir collection'ı değiştiriyorsa concurrent collection class'ı kullanmalısınız. Aksi hâlde collection'a eşzamanlı erişim bir exception'a, daha kötüsü verinin bozulmasına yol açabilir.

English: If the collection is immutable (and contains immutable objects), the concurrent collections are not necessary. Immutable objects can be accessed by any number of threads and do not require synchronization.

Türkçe: Collection immutable ise ve immutable object'ler içeriyorsa concurrent collection kullanmak gerekmez. Immutable object'lere istenen sayıda thread erişebilir; synchronization gerekmez.

English: By definition, they do not change, so there is no chance of a memory consistency error.

Türkçe: Tanımları gereği değişmediklerinden memory consistency error oluşma olasılığı yoktur.

English: When passing around a concurrent collection, a caller may need to know the particular implementation class. That said, it is considered a good practice to pass around a nonconcurrent interface reference when possible, similar to how we instantiate a `HashMap` but often pass around a `Map` reference:

Türkçe: Concurrent collection başka bir koda aktarılırken caller'ın belirli implementation class'ını bilmesi gerekebilir. Yine de mümkün olduğunda nonconcurrent interface türü üzerinden aktarmak iyi pratiktir; tıpkı `HashMap` oluşturup çoğu zaman onu `Map` reference'ı üzerinden kullanmamız gibi:

```
Map<String,Integer> map = new ConcurrentHashMap<>();
```

English: Table 13.9 lists the common concurrent classes with which you should be familiar for the exam.

Türkçe: Tablo 13.9, sınav için bilmeniz gereken yaygın concurrent class'ları listeler.

English: **TABLE 13.9 — Concurrent collection classes** - `ConcurrentHashMap` — Interfaces: `Map`, `ConcurrentMap`; sorted: no; blocking: no. - `ConcurrentLinkedQueue` — Interface: `Queue`; sorted: no; blocking: no. - `ConcurrentSkipListMap` — Interfaces: `Map`, `SortedMap`, `NavigableMap`, `ConcurrentMap`, `ConcurrentNavigableMap`; sorted: yes; blocking: no. - `ConcurrentSkipListSet` — Interfaces: `Set`, `SortedSet`, `NavigableSet`; sorted: yes; blocking: no. - `CopyOnWriteArrayList` — Interface: `List`; sorted: no; blocking: no. - `CopyOnWriteArraySet` — Interface: `Set`; sorted: no; blocking: no. - `LinkedBlockingQueue` — Interfaces: `Queue`, `BlockingQueue`; sorted: no; blocking: yes.

Türkçe: **TABLO 13.9 — Concurrent collection class'ları** - `ConcurrentHashMap` — Interface'ler: `Map`, `ConcurrentMap`; sorted: hayır; blocking: hayır. - `ConcurrentLinkedQueue` — Interface: `Queue`; sorted: hayır; blocking: hayır. - `ConcurrentSkipListMap` — Interface'ler: `Map`, `SortedMap`, `NavigableMap`, `ConcurrentMap`, `ConcurrentNavigableMap`; sorted: evet; blocking: hayır. - `ConcurrentSkipListSet` — Interface'ler: `Set`, `SortedSet`, `NavigableSet`; sorted: evet; blocking: hayır. - `CopyOnWriteArrayList` — Interface: `List`; sorted: hayır; blocking: hayır. - `CopyOnWriteArraySet` — Interface: `Set`; sorted: hayır; blocking: hayır. - `LinkedBlockingQueue` — Interface'ler: `Queue`, `BlockingQueue`; sorted: hayır; blocking: evet.

English: Most of the classes in TABLE 13.9 are concurrent versions of their nonconcurrent counterparts, such as `ConcurrentHashMap` versus `Map` or `ConcurrentLinkedQueue` versus `Queue`. For the exam, you do not need to know class-specific concurrent methods. You need to know inherited methods, such as `get()` and `set()` for `List` instances.

Türkçe: TABLO 13.9'daki class'ların çoğu, `ConcurrentHashMap` ile `Map` veya `ConcurrentLinkedQueue` ile `Queue` örneklerinde olduğu gibi, concurrent olmayan karşılıklarının concurrent sürümüdür. Sınav için class'a özgü concurrent metotları değil, örneğin `List` instance'ları için `get()` ve `set()` gibi inherited metotları bilmek gerekir.

English: The Skip classes might sound strange, but they are just “sorted” versions of the associated concurrent collections. When you see a class with Skip in the name, just think “sorted concurrent” collections, and the rest should follow naturally.

Türkçe: Adında `Skip` geçen class'lar, ilişkili concurrent collection'ların **sorted** sürümleridir. Sınavda `Skip` gördüğünüzde “sorted concurrent collection” çağrışımını kullanabilirsiniz.

English: The CopyOnWrite classes behave a little differently than the other concurrent examples you have seen. These classes create a copy of the collection any time a reference is added, removed, or changed in the collection and then update the original collection reference to point to the copy. These classes are commonly used to ensure an iterator doesn't see modifications to the collection.

Türkçe: `CopyOnWrite` class'ları diğer concurrent örneklerden biraz farklı davranır. Collection'a bir reference eklendiğinde, kaldırıldığında veya değiştirildiğinde yeni bir collection kopyası oluşturur ve özgün reference'ı bu kopyaya yöneltir. Böylece mevcut iterator'ların collection'daki sonraki değişiklikleri görmemesi sağlanır.

English: Let's take a look at how this works with an example:

Türkçe: Bunun nasıl çalıştığına bir örnek üzerinden bakalım:

```
List<Integer> favNumbers = new CopyOnWriteArrayList<>(List.of(4, 3, 42));
for (var n: favNumbers) {
    System.out.print(n + " "); // 4 3 42
    favNumbers.add(n+1);
}
```

```
System.out.println();
System.out.println("Size: " + favNumbers.size()); // Size: 6
```

English: Despite adding elements, the iterator is not modified, and the loop executes exactly three times. Alternatively, if we had used a regular `ArrayList` object, a `ConcurrentModificationException` would have been thrown at runtime. The `CopyOnWrite` classes can use a lot of memory, since a new collection structure is created any time the collection is modified. Therefore, they are commonly used in multithreaded environment situations where reads are far more common than writes.

Türkçe: Element eklenmesine rağmen mevcut iterator değişmez ve loop tam üç kez çalışır. Normal bir `ArrayList` kullanılsaydı runtime'da `ConcurrentModificationException` atılırdı. Her değişiklikte yeni bir collection yapısı oluşturulduğu için `CopyOnWrite` class'ları çok memory kullanabilir. Bu nedenle read operation'ların write operation'lardan çok daha sık olduğu multithreaded ortamlarda yaygın biçimde kullanılır.

English: A `CopyOnWrite` instance is similar to an immutable object, as a new underlying structure is created every time the collection is modified.

Türkçe: Collection her değiştirildiğinde yeni bir underlying structure oluşturulduğu için `CopyOnWrite` instance'ı immutable object'e benzer.

English: Unlike a true immutable object, though, the reference to the object stays the same even while the underlying data is changed.

Türkçe: Ancak gerçek bir immutable object'in aksine, underlying data değişse bile object reference'ı aynı kalır.

English: Finally, Table 13.9 includes `LinkedBlockingQueue`, which implements the concurrent `BlockingQueue` interface. This class is just like a regular `Queue`, except that it includes overloaded versions of `offer()` and `poll()` that take a timeout. These methods wait (or block) up to a specific amount of time to complete an operation.

Türkçe: Son olarak Tablo 13.9, concurrent `BlockingQueue` interface'ini implement eden `LinkedBlockingQueue`'yu içerir. Bu class normal bir `Queue` gibidir; ayrıca timeout alan overload edilmiş `offer()` ve `poll()` sürümlerini sağlar. Bu method'lar operation'ın tamamlanması için belirtilen süreye kadar bekler, yani block eder.

Obtaining Synchronized Collections

English: Besides the concurrent collection classes that we have covered, the Concurrency API also includes methods for obtaining synchronized versions of existing nonconcurrent collection objects. These synchronized methods are defined in the Collections class. They operate on the inputted collection and return a reference that is the same type as the underlying collection. We list these static methods in Table 13.10.

Türkçe: Ele aldığımız `java.util.concurrent` collection class'larının yanında Java Collections API, mevcut nonconcurrent collection object'lerinin synchronized wrapper'larını elde etmeye yarayan method'lar da içerir. Bunlar `java.util.Collections` class'ında tanımlıdır; input collection üzerinde çalışır ve underlying collection ile aynı interface türünde reference döndürür. Static method'lar Tablo 13.10'da listelenmiştir.

English: TABLE 13.10 — Synchronized Collections methods - `static <T> Collection<T> synchronizedCollection(Collection<T> c)` - `static <T> List<T> synchronizedList(List<T> list)` - `static <K,V> Map<K,V> synchronizedMap(Map<K,V> m)` - `static <K,V> NavigableMap<K,V> synchronizedNavigableMap(NavigableMap<K,V> m)` - `static <T> NavigableSet<T> synchronizedNavigableSet(NavigableSet<T> s)` - `static <T> Set<T> synchronizedSet(Set<T> s)` - `static <K,V> SortedMap<K,V> synchronizedSortedMap(SortedMap<K,V> m)` - `static <T> SortedSet<T> synchronizedSortedSet(SortedSet<T> s)`

Türkçe: TABLO 13.10 — Synchronized Collections metotları - `static <T> Collection<T> synchronizedCollection(Collection<T> c)` - `static <T> List<T> synchronizedList(List<T> list)` - `static <K,V> Map<K,V> synchronizedMap(Map<K,V> m)` - `static <K,V> NavigableMap<K,V> synchronizedNavigableMap(NavigableMap<K,V> m)` - `static <T> NavigableSet<T> synchronizedNavigableSet(NavigableSet<T> s)` - `static <T> Set<T> synchronizedSet(Set<T> s)` - `static <K,V> SortedMap<K,V> synchronizedSortedMap(SortedMap<K,V> m)` - `static <T> SortedSet<T> synchronizedSortedSet(SortedSet<T> s)`

English: If you're writing code to create a collection and it requires synchronization, you should use the classes defined in Table 13.9. On the other hand, if you are passed a nonconcurrent collection and need synchronization, use the methods in Table 13.10.

Türkçe: Yeni bir collection oluşturuyor ve synchronization gerekiyorsa Tablo 13.9'daki concurrent class'ları kullanın. Buna karşılık size concurrent olmayan mevcut bir collection verilmişse ve synchronization gerekiyorsa Tablo 13.10'daki wrapper method'larını kullanın.

OCP tuzağı: Synchronized wrapper yalnız individual method call'ları synchronize eder. Iteration, stream traversal veya compound check-then-act sequence sırasında dönen wrapper object üzerinde external synchronization gerekebilir.

Identifying Threading Problems

Dil çalışması: Bu başlıktaki kelimeler için [ünite sözlüğüne](#) bak.

English: Now that you know how to write thread-safe code, let's talk about what qualifies as a threading problem. A threading problem can occur in multithreaded applications when two or more threads interact in an unexpected and undesirable way. For example, two threads may block each other from accessing a particular segment of code.

Türkçe: Thread-safe kod yazmayı ele aldığımıza göre şimdi nelerin threading problem sayıldığını inceleyelim. Multithreaded uygulamalarda iki veya daha fazla thread beklenmedik ve istenmeyen biçimde etkileşirse threading problem oluşabilir. Örneğin iki thread, belirli bir code segment'e erişimde birbirini block edebilir.

English: The Concurrency API was created to help eliminate potential threading issues common to all developers. As you have seen, the Concurrency API creates threads and manages complex thread interactions for you, often in just a few lines of code.

Türkçe: Concurrency API, geliştiricilerin karşılaştığı yaygın threading problem'lerini azaltmaya yardımcı olmak için oluşturulmuştur. Görüldüğü gibi birkaç satır kodla thread oluşturabilir ve karmaşık thread etkileşimlerini yönetebilir.

English: Although the Concurrency API reduces the potential for threading issues, it does not eliminate them. In practice, finding and identifying threading issues within an application is often one of the most difficult tasks a developer can undertake.

Türkçe: Concurrency API, thread'lerle ilgili sorunların ortaya çıkma olasılığını azaltsa da bunları tamamen ortadan kaldırmaz. Uygulamada bu sorunları bulup tanımlamak, bir geliştiricinin üstlenebileceği en zor işlerden biridir.

Dil çalışması: *although* ile beklenene aykırı sonucu ve *potential* sözcüğünü [grammar notlarında](#) ve [ünite sözlüğünde](#) karşılaştır.

Understanding Liveness

English: As you have seen in this chapter, many thread operations can be performed independently, but some require coordination. For example, synchronizing on a method requires all threads that call the method to wait for other threads to finish before continuing. You also saw earlier in the chapter that threads in a `CyclicBarrier` will each wait for the barrier limit to be reached before continuing.

Türkçe: Bu bölümde görüldüğü gibi birçok thread operation bağımsız yürütülebilir, fakat bazıları coordination gerektirir. Örneğin bir method üzerinde synchronization, o method'u çağıran thread'lerin devam etmeden önce lock'u tutan thread'i beklemesini gerektirir. Benzer biçimde `CyclicBarrier` kullanan thread'ler, devam etmeden önce bariyer limitine ulaşılmasını bekler.

English: What happens to the application while all of these threads are waiting? In many cases, the waiting is ephemeral, and the user has very little idea that any delay has occurred. In other cases, though, the waiting may be extremely long, perhaps infinite.

Türkçe: Bütün bu thread'ler beklerken uygulamaya ne olur? Çoğu durumda bekleme geçicidir ve kullanıcı gecikmeyi neredeyse hiç fark etmez. Bazı durumlardaysa bekleme son derece uzun, hatta sonsuz olabilir.

English: Liveness is the ability of an application to be able to execute in a timely manner. Liveness problems, then, are those in which the application becomes unresponsive or is in some kind of “stuck” state. More precisely, liveness problems are often the result of a thread entering a BLOCKING or WAITING state forever, or repeatedly entering/exiting these states. For the exam, there are three types of liveness issues with which you should be familiar: deadlock, starvation, and livelock.

Türkçe: Liveness (canlılık), uygulamanın zamanında ilerleyebilme yeteneğidir. Liveness problem'lerinde uygulama yanıt vermez veya bir tür “takılı” durumda kalır. Daha kesin olarak bunlar çoğu zaman bir thread'in sonsuza dek **BLOCKED** ya da **WAITING** state'inde kalmasından veya bu state'lere tekrar tekrar girip çıkmasından doğar. Sınav için üç liveness problem'i bilinmelidir: **deadlock**, **starvation** ve **livelock**.

Deadlock

English: Deadlock occurs when two or more threads are blocked forever, each waiting on the other. We can illustrate this principle with the following example. Imagine that our zoo has two foxes: Foxy and Tails. Foxy likes to eat first and then drink water, while Tails likes to drink water first and then eat. Furthermore, neither animal likes to share, and they will finish their meal only if they have exclusive access to both food and water.

Türkçe: Deadlock, her biri diğerini bekleyen iki veya daha fazla thread'in sonsuza kadar block olmasıdır. Hayvanat bahçemizde Foxy ve Tails adlı iki tilki olduğunu düşünelim. Foxy önce yiyip sonra su içmek; Tails ise önce su içip sonra yemek ister. İkisi de paylaşmayı sevmez ve öğününü ancak hem yiyeceğe hem suya exclusive access elde ederse tamamlar.

English: The zookeeper places the food on one side of the environment and the water on the other side. Although our foxes are fast, it still takes them 100 milliseconds to run from one side of the environment to the other.

Türkçe: Zookeeper yiyeceği alanın bir tarafına, suyu diğer tarafına koyar. Tilkilerimiz hızlı olsa da bir taraftan diğerine koşmaları 100 milisaniye sürer.

English: What happens if Foxy gets the food first and Tails gets the water first? The following application models this behavior:

Türkçe: Foxy önce yemeği, Tails de önce suyu alırsa ne olur? Aşağıdaki uygulama bu davranışı modellemektedir:

```
import java.util.concurrent.*;
class Food {}
class Water {}
public record Fox(String name) {
    public void eatAndDrink(Food food, Water water) {
        synchronized(food) {
            System.out.println(name() + " Got Food!");
            move();
        }
        synchronized(water) {
            System.out.println(name() + " Got Water!");
        }
    }
    public void drinkAndEat(Food food, Water water) {
        synchronized(water) {
            System.out.println(name() + " Got Water!");
            move();
        }
        synchronized(food) {
            System.out.println(name() + " Got Food!");
        }
    }
    public void move() {
        try { Thread.sleep(100); } catch (InterruptedException e) {}
    }
    public static void main(String[] args) {
        // Create participants and resources
        var foxy = new Fox("Foxy");
        var tails = new Fox("Tails");
        var food = new Food();
        var water = new Water();
        // Process data
        var service = Executors.newScheduledThreadPool(10);
        try {
            service.submit(() -> foxy.eatAndDrink(food,water));
            service.submit(() -> tails.drinkAndEat(food,water));
        } finally {
            service.shutdown();
        }
    }
}
```

English: In this example, Foxy obtains the food and then moves to the other side of the environment to obtain the water. Unfortunately, Tails already drank the water and is waiting for the food to become available. The result is that our program outputs the following, and it hangs indefinitely:

Türkçe: Foxy'nin yiyeceği, Tails'in de suyu önce aldığı bu olası interleaving'de ikisi diğer kaynağı bekler. Program aşağıdaki sample output'u üretir ve deadlock nedeniyle JVM süresiz biçimde açık kalır:

English: Foxy Got Food! Tails Got Water!

Türkçe: Literal çıktı: `Foxy Got Food! Tails Got Water!`

English: This example is considered a deadlock because both participants are permanently blocked, waiting on resources that will never become available.

Türkçe: Bu interleaving'de iki katılımcı da kalıcı olarak block olup diğerinin tuttuğu kaynağı beklediği için deadlock oluşur.

Java 17 editor notu: Deadlock bu programın garanti edilen sonucu değildir. Scheduler ikinci task'ı, ilk task iki monitor'ü de alıp bıraktıktan sonra başlatırsa iki task da tamamlanabilir. OCP analizinde “deadlock oluşabilir” ile “mutlaka oluşur” ayrımını koruyun.

Starvation

English: Starvation occurs when a single thread is perpetually denied access to a shared resource or lock. The thread is still active, but it is unable to complete its work as a result of other threads constantly taking the resource that it is trying to access.

Türkçe: Starvation, tek bir thread'in shared resource'a veya lock'a erişiminin sürekli engellenmesidir. Thread hâlâ aktiftir; fakat diğer thread'ler erişmeye çalıştığı kaynağı sürekli aldığı için işini tamamlayamaz.

English: In our fox example, imagine that we have a pack of very hungry, very competitive foxes in our environment. Every time Foxy stands up to go get food, one of the other foxes sees her and rushes to eat before her. Foxy is free to roam around the enclosure, take a nap, and howl for a zookeeper but is never able to obtain access to the food. In this example, Foxy literally and figuratively experiences starvation. It's a good thing that this is just a theoretical example!

Türkçe: Tilki örneğinde çok aç ve rekabetçi bir tilki sürüsü düşünün. Foxy ne zaman yiyeceğe gitmek için kalksa başka bir tilki onu görüp daha önce davranır. Foxy alanda dolaşabilir, uyuyabilir veya zookeeper'a uluyabilir; fakat yiyeceğe hiçbir zaman erişemez. Böylece starvation'ı hem gerçek hem mecaz anlamıyla yaşar. Neyse ki bu yalnız teorik bir örnektir.

Livelock

English: Livelock occurs when two or more threads are conceptually blocked forever, although they are each still active and trying to complete their task. Livelock is a special case of resource starvation in which two or more threads actively try to acquire a set of locks, are unable to do so, and restart part of the process.

Türkçe: Livelock, iki veya daha fazla thread aktif kalıp task'ını tamamlamaya çalıştığı hâlde kavramsal olarak sonsuza kadar ilerleyemediğinde oluşur. İki veya daha fazla thread'in bir lock kümesini aktif biçimde almaya çalışıp başaramadığı ve sürecin bir bölümünü tekrar başlattığı özel bir resource starvation durumudur.

English: Livelock is often a result of two threads trying to resolve a deadlock. Returning to our fox example, imagine that Foxy and Tails are both holding their food and water resources, respectively. They each realize that they cannot finish their meal in this state, so they both let go of their food and water, run to the opposite side of the environment, and pick up the other resource. Now Foxy has the water, Tails has the food, and neither is able to finish their meal!

Türkçe: Livelock çoğu zaman iki thread'in deadlock'ı çözmeye çalışmasından doğar. Tilki örneğine dönersek Foxy'nin yiyeceği, Tails'in suyu tuttuğunu düşünün. Bu durumda öğünlerini bitiremeyeceklerini fark edip ikisi de kaynaklarını bırakır, karşı tarafa koşar ve diğer kaynağı alır. Artık su Foxy'de, yiyecek Tails'tedir ve yine ikisi de öğününü tamamlayamaz.

English: If Foxy and Tails continue this process forever, it is referred to as livelock. Both Foxy and Tails are active, running back and forth across their area, but neither can finish their meal. Foxy and Tails are executing a form of failed deadlock recovery. Each fox notices that they are potentially entering a deadlock state and responds by releasing all of its locked resources. Unfortunately, the lock and unlock process is cyclical, and the two foxes are conceptually deadlocked.

Türkçe: Foxy ve Tails bu süreci sonsuza kadar sürdürürse livelock oluşur. İkisi de aktiftir ve alanda ileri geri koşar; fakat hiçbirini öğününü bitiremez. Bu, başarısız bir deadlock recovery biçimidir: Her tilki olası deadlock'ı fark edip tuttuğu bütün kaynakları bırakır. Ne var ki lock alma ve bırakma süreci döngüsel hâle gelir; tilkiler kavramsal olarak ilerleyemez.

English: In practice, livelock is often a difficult issue to detect. Threads in a livelock state appear active and able to respond to requests, even when they are stuck in an endless cycle.

Türkçe: Pratikte livelock'ı tespit etmek çoğu zaman zordur. Livelock durumundaki thread'ler sonsuz bir cycle'a sıkışmış olsalar bile aktif ve request'lere yanıt verebiliyor gibi görünür.

Managing Race Conditions

English: A race condition is an undesirable result that occurs when two tasks that should be completed sequentially are completed at the same time. We encountered examples of race conditions earlier in the chapter when we introduced synchronization.

Türkçe: Race condition, program doğruluğunun unsynchronized operation'ların relative timing veya interleaving'ine bağlı olmasıdır. Gerçek paralellik şart değildir; tek CPU üzerinde context switch ile de oluşabilir. Synchronization tanıtılırken bölümün önceki kısmında bunun örnekleri görülmüştü.

English: While Figure 13.4 shows a classical thread-based example of a race condition, we now provide a more illustrative example. Imagine that two zoo patrons, Olivia and Sophia, are signing up for an account on the zoo's new visitor website. Both of them want to use the same username, ZooFan, and each sends a request to create the account at the same time, as shown in Figure 13.6.

Türkçe: Şekil 13.4 klasik bir thread tabanlı race condition örneği gösteriyordu; şimdi daha açıklayıcı bir örnek ele alalım. Zoo ziyaretçileri Olivia ile Sophia'nın yeni ziyaretçi web sitesinde hesap açtığını düşünün. İkisi de aynı **ZooFan** username'ini ister ve Şekil 13.6'da gösterildiği gibi hesabı oluşturma request'ini aynı anda gönderir.

English: FIGURE 13.6 — Race condition on user creation: Olivia and Sophia each submit “Create ZooFan” to the zoo web server.

Türkçe: **Şekil 13.6 — Kullanıcı oluşturma race condition'ı:** Olivia ve Sophia, zoo web server'a ayrı ayrı “Create **ZooFan**” request'i gönderir.

English: What result does the web server return when both users attempt to create an account with the same username in Figure 13.6?

Türkçe: Şekil 13.6'da iki kullanıcı aynı username ile hesap açmaya çalıştığında web server hangi sonucu döndürür?

English: Possible Outcomes for This Race Condition • Both users are able to create accounts with the username ZooFan. • Neither user is able to create an account with the username ZooFan, and an error message is returned to both users. • One user is able to create an account with the username ZooFan, while the other user receives an error message.

Türkçe: Bu race condition için olası sonuçlar: (1) İki kullanıcı da ZooFan username'ile hesap açar. (2) Hiçbiri hesabı açamaz ve ikisine de error message döner. (3) Kullanıcılardan biri hesabı açar, diğeri error message alır.

English: The first outcome is really bad, as it leads to users trying to log in with the same username. Whose data do they see when they log in? The second outcome causes both users to have to try again, which is frustrating but at least doesn't lead to corrupt or bad data.

Türkçe: İlk sonuç çok kötüdür; iki kullanıcı aynı username ile login etmeye çalışır. Login olduklarında hangisinin verisini göreceklerdir? İkinci sonuç iki kullanıcıyı da yeniden denemeye zorlar; bu sinir bozucu olsa da en azından corrupt data üretmez.

English: The third outcome is often considered the best solution. Like the second situation, we preserve data integrity; but unlike the second situation, at least one user is able to move forward on the first request, avoiding additional race condition scenarios.

Türkçe: Üçüncü sonuç çoğu zaman en iyi çözüm sayılır. İkinci durumda olduğu gibi data integrity korunur; fakat ondan farklı olarak en az bir kullanıcı ilk request'te ilerleyebilir ve ek race condition senaryoları önlenir.

English: For the exam, you should understand that race conditions lead to invalid data if they are not properly handled. Even the solution where both participants fail to proceed is preferable to one in which invalid data is permitted to enter the system.

Türkçe: Sınav için race condition'ların doğru ele alınmazsa invalid data üretebileceğini bilmelisiniz. İki katılımcının da ilerleyemediği çözüm bile invalid data'nın sisteme girmesine izin verilmesinden daha iyidir.

Working with Parallel Streams

English: We conclude this chapter by combining what you learned in Chapter 10, "Streams," with the concepts you learned about in this chapter. One of the most powerful features of the Stream

Türkçe: Bu bölümü, Chapter 10 "Streams" konusunda öğrendiklerinizle bu chapter'daki concurrency kavramlarını birleştirerek bitiriyoruz. Stream API'nin en güçlü özelliklerinden biri

English: API is built-in concurrency support. Up until now, all of the streams you have worked with have been serial streams. A serial stream is a stream in which the results are ordered, with only one entry being processed at a time.

Türkçe: built-in concurrency support sağlamasıdır. Şimdiye kadar çalıştığınız stream'lerin tümü serial stream'di. Serial stream, pipeline'ı parallel yürütmez ve bir anda tek execution path kullanır; ordered olup olmaması ise source ile pipeline'ın ayrı bir özelliğidir.

English: A parallel stream is capable of processing results concurrently, using multiple threads. For example, you can use a parallel stream and the `map()` operation to operate concurrently on the elements in the stream, vastly improving performance over processing a single element at a time.

Türkçe: Parallel stream birden fazla thread kullanarak sonuçları concurrently işleyebilir. Örneğin `map()` operation'ını parallel stream üzerinde çalıştırarak element'leri eşzamanlı işleyebilir ve tek tek işlemeye göre performansı önemli ölçüde artırabilirsiniz.

English: Using a parallel stream can change not only the performance of your application but also the expected results. As you shall see, some operations also require special handling to be able to be processed in a parallel manner.

Türkçe: Parallel stream kullanmak yalnız uygulamanın performansını değil, beklenen sonuçları da değiştirebilir. Göreceğiniz gibi bazı operation'ların parallel işlenebilmesi için özel biçimde ele alınması gerekir.

English: The number of threads available in a parallel stream is proportional to the number of available CPUs in your environment.

Türkçe: JDK'nin varsayılan uygulaması parallel stream için çoğunlukla common `ForkJoinPool` parallelism'ini available processor sayısına göre belirler. Ancak exact thread count Java API garantisidir; configuration, caller participation ve runtime koşulları sonucu etkiler.

Creating Parallel Streams

English: The Stream API was designed to make creating parallel streams quite easy. For the exam, you should be familiar with two ways of creating a parallel stream.

Türkçe: Stream API, parallel streams oluşturulmasını oldukça kolay hale getirmek için tasarlanmıştır. Sınav için, bir parallel stream oluşturmanın iki yolunu bilmelisiniz.

```
Collection<Integer> collection = List.of(1,2);  
Stream<Integer> p1 = collection.stream().parallel();  
Stream<Integer> p2 = collection.parallelStream();
```

English: The first way to create a parallel stream is from an existing stream. Isn't this cool? Any stream can be made parallel! The second way to create a parallel stream is from a Java Collection class. We use both of these methods throughout this section.

Türkçe: İlk yol mevcut bir stream'i `parallel()` ile parallel hâle getirmektir; herhangi bir stream bu şekilde dönüştürülebilir. İkinci yol bir Java `Collection` üzerinden `parallelStream()` çağırmaaktır. Bu bölümde iki metod de kullanılacaktır.

English: The Stream interface includes a method `isParallel()` that can be used to test whether the instance of a stream supports parallel processing.

Türkçe: `Stream` interface'i, bir stream instance'ının parallel processing kullanıp kullanmadığını sınamak için `isParallel()` method'unu içerir.

English: Some operations on streams preserve the parallel attribute, while others do not.

Türkçe: Stream üzerindeki bazı operation'lar parallel niteliğini korur, bazıları korumaz.

Performing a Parallel Decomposition

English: A parallel decomposition is the process of taking a task, breaking it into smaller pieces that can be performed concurrently, and then reassembling the results. The more concurrent a decomposition, the greater the performance improvement of using parallel streams.

Türkçe: Parallel decomposition, bir task'ı concurrently yürütülebilecek küçük parçalara bölüp sonuçları yeniden birleştirme sürecidir. Decomposition ne kadar çok concurrent çalışmaya izin verirse parallel stream'in sağlayabileceği performans artışı da o kadar büyük olur.

English: Let's try it out. First, let's define a reusable function that "does work" just by waiting for five seconds.

Türkçe: Bunu deneyelim. Önce yalnızca beş saniye bekleyerek "iş yapan", yeniden kullanılabilir bir function tanımlayalım.

```
private static int doWork(int input) {
    try {
        Thread.sleep(5000);
    } catch (InterruptedException e) {}
    return input;
}
```

English: We can pretend that in a real application, this work might involve calling a database or reading a file. Now let's use this method with a serial stream.

Türkçe: Gerçek bir uygulamada bu işin database çağrısı yapmak veya file okumak olduğunu varsayabiliriz. Şimdi method'u serial stream ile kullanalım.

```
long start = System.currentTimeMillis();
List.of(1,2,3,4,5)
    .stream()
    .map(w -> doWork(w))
    .forEach(s -> System.out.print(s + " "));
```

```
System.out.println();
var timeTaken = (System.currentTimeMillis()-start)/1000;
System.out.println("Time: "+timeTaken+" seconds");
```

English: What do you think this code will output when executed as part of a main() method? Let's take a look:

Türkçe: Bu kod `main()` method'unun parçası olarak çalıştırıldığında ne yazdırır? Sonuca bakalım:

English: 1 2 3 4 5 Time: 25 seconds As you might expect, the results are ordered and predictable because we are using a serial stream. It also took around 25 seconds to process all five results, one at a time. What happens if we replace line 12 with one that uses a `parallelStream()`? The following is some sample output:

Türkçe: Sample output yaklaşık 1 2 3 4 5 Time: 25 seconds biçimindedir. Bu ordered source serial işlendiği için element'ler encounter order'a uygundur; elapsed time scheduling, system load ve clock resolution nedeniyle exact değildir. Line 12'de `parallelStream()` kullanılırsa aşağıdakine benzer bir çıktı alınabilir:

English: 3 2 1 5 4 Time: 5 seconds As you can see, the results are no longer ordered or predictable. The `map()` and `forEach()` operations on a parallel stream are equivalent to submitting multiple `Runnable` lambda expressions to a pooled thread executor and then waiting for the results.

Türkçe: Sample output yaklaşık 3 2 1 5 4 Time: 5 seconds biçiminde olabilir; hem element order hem elapsed time garanti edilmez. Parallel stream üzerindeki `map()` ve `forEach()` operation'ları, birden fazla `Runnable` lambda'yı pooled thread executor'a gönderip sonuçları beklemeye benzer.

English: What about the time required? In this case, our system had enough CPUs for all of the tasks to be run concurrently. If you ran this same code on a computer with fewer processors, it might output 10 seconds, 15 seconds, or some other value. The key is that we've written our code to take advantage of parallel processing when available, so our job is done.

Türkçe: Gerekli süre ne olur? Bu örnekte sistemde bütün task'ları concurrently çalıştırmaya yetecek CPU vardır. Aynı kod daha az processor bulunan bir bilgisayarda 10, 15 saniye veya başka bir süre yazdırabilir. Önemli nokta, kodu mümkün olduğunda parallel processing'den yararlanacak biçimde yazmış olmamızdır.

English: Ordering Results If your stream operation needs to guarantee ordering and you're not sure if it is serial or parallel, you can replace line 14 with one that uses `forEachOrdered()`:

Türkçe: Sonuçları sıralı işleme: Stream'in serial mı parallel mı olduğundan emin değilseniz ve encounter order garanti edilmeliyse line 14'te `forEachOrdered()` kullanabilirsiniz:

```
.forEachOrdered(s -> System.out.print(s + " "));
```

English: This outputs the results in the order in which they are defined in the stream:

Türkçe: Bu method sonuçları stream'deki encounter order'a göre yazdırır:

English: 1 2 3 4 5 Time: 5 seconds While we've lost some of the performance gains of using a parallel stream, our `map()` operation can still take advantage of the parallel stream.

Türkçe: Sample output yaklaşık 1 2 3 4 5 Time: 5 seconds biçimindedir; süre exact değildir. Encounter order'ı korumanın bir performans maliyeti bulunsa da `map()` operation'ı parallel stream'den yararlanmaya devam edebilir.

Processing Parallel Reductions

English: Besides potentially improving performance and modifying the order of operations, using parallel streams can impact how you write your application. A parallel reduction is a reduction operation applied to a parallel stream. The results for parallel reductions can differ from what you expect when working with serial streams.

Türkçe: Parallel stream performansı artırıp operation order'ını değiştirebildiği gibi uygulamanın nasıl yazılacağını da etkileyebilir. Parallel reduction, parallel stream'e uygulanan reduction operation'dır. Sonuçları, serial stream ile çalışırken beklediğinizden farklı olabilir.

Performing Order-Based Tasks

English: Since order is not guaranteed with parallel streams, methods such as `findAny()` on parallel streams may result in unexpected behavior. Consider the following example:

Türkçe: Parallel stream'de order garanti edilmediğinden `findAny()` gibi method'lar beklenmedik sonuç verebilir. Aşağıdaki örneği inceleyin:

```
System.out.print(List.of(1,2,3,4,5,6)
    .parallelStream()
    .findAny()
    .get());
```

English: The JVM allocates a number of threads and returns the value of the first one to return a result, which could be 4, 2, and so on. While neither the serial nor the parallel stream is guaranteed to return the first value, the serial stream often does. With a parallel stream, the results are likely to be more random.

Türkçe: `findAny()` herhangi bir element döndürebilir; 4, 2 veya başka bir value mümkündür. API, kaç thread ayrılacağını ya da "ilk biten thread'in value'su" biçiminde bir seçim algoritmasını garanti etmez. Ne serial ne parallel stream için first value garantisi vardır; serial uygulama pratikte çoğu zaman ilkini döndürse de bu davranışa güvenilmez.

English: What about operations that consider order, such as `findFirst()`, `limit()`, and `skip()`? Order is still preserved, but performance may suffer on a parallel stream as a result of a parallel processing task being forced to coordinate all of its threads in a synchronized-like fashion.

Türkçe: Peki `findFirst()`, `limit()` ve `skip()` gibi order-sensitive operation'lar? **Ordered stream** üzerinde encounter order korunur; ancak parallel task thread'leri koordine etmek zorunda kaldığı için performans düşebilir. Unordered stream'de bu order garantisi yoktur.

English: On the plus side, the results of ordered operations on a parallel stream will be consistent with a serial stream. For example, calling `skip(5).limit(2).findFirst()` will return the same result on ordered serial and parallel streams.

Türkçe: Olumlu yanı, ordered parallel stream operation'larının serial stream ile tutarlı sonuç vermesidir. Örneğin `skip(5).limit(2).findFirst()`, ordered serial ve parallel stream'de aynı sonucu döndürür.

English: Creating Unordered Streams All of the streams you have been working with are considered ordered by default. It is possible to create an unordered stream from an ordered stream, similar to how you create a parallel stream from a serial stream.

Türkçe: Unordered stream oluşturma: Şimdiye kadar kullandığınız stream'ler varsayılan olarak ordered kabul edilir. Serial stream'den parallel stream oluşturulmasına benzer biçimde ordered stream'den unordered stream oluşturulabilir.

```
List.of(1,2,3,4,5,6).stream().unordered();
```

English: This method does not reorder the elements; it just tells the JVM that if an order-based stream operation is applied, the order can be ignored. For example, calling skip(5) on an unordered stream will skip any 5 elements, not necessarily the first 5 required on an ordered stream.

Türkçe: Bu method element'leri fiziksel olarak yeniden sıralamak zorunda değildir; JVM'e order-based operation uygulanırken encounter order'ın yok sayılabileceğini bildirir. Örneğin unordered stream üzerinde skip(5), ordered stream'de zorunlu olan ilk beş yerine herhangi beş elementi atlayabilir.

English: For serial streams, using an unordered version has no effect. But on parallel streams, the results can greatly improve performance.

Türkçe: Serial stream'de unordered() çoğu zaman görünür bir reordering veya performans farkı oluşturmaz; yine de encounter-order garantisini kaldırır. Parallel stream'de order constraint'i kaldırmak performansı önemli ölçüde artırabilir.

```
List.of(1,2,3,4,5,6).stream().unordered().parallel();
```

English: Even though unordered streams will not be on the exam, if you are developing applications with parallel streams, you should know when to apply an unordered stream to improve performance.

Türkçe: Unordered stream sınav kapsamında olmasa da parallel stream kullanan uygulamalar geliştirirken performans için ne zaman unordered() uygulanacağını bilmek yararlıdır.

Combining Results with reduce()

English: As you learned in Chapter 10, the stream operation reduce() combines a stream into a single object. Recall that the first parameter to the reduce() method is called the identity, the second parameter is called the accumulator, and the third parameter is called the combiner. The following is the signature for the method:

Türkçe: Chapter 10'da öğrendiğiniz gibi reduce() stream'i tek bir object'te birleştirir. İlk parameter identity, ikinci accumulator, üçüncü combiner olarak adlandırılır. Method signature şöyledir:

English: `<U> U reduce(U identity, BiFunction<U,? super T,U> accumulator, BinaryOperator<U> combiner)`

Türkçe: Method signature aynen şöyledir: `<U> U reduce(U identity, BiFunction<U,? super T,U> accumulator, BinaryOperator<U> combiner)`.

English: We can concatenate a list of char values using the `reduce()` method, as shown in the following example:

Türkçe: Aşağıdaki örnekte `reduce()` ile `char` value'lardan oluşan list birleştirilebilir:

```
System.out.println(List.of('w', 'o', 'l', 'f')
    .parallelStream()
    .reduce("",
        (s1,c) -> s1 + c,
        (s2,s3) -> s2 + s3)); // wolf
```

English: The naming of the variables in this stream example is not accidental. We used `c` for char, whereas `s1`, `s2`, and `s3` are String values.

Türkçe: Bu stream örneğindeki variable adları rastlantı değildir: `c`, `char` value'yu; `s1`, `s2` ve `s3` ise `String` value'ları temsil eder.

English: On parallel streams, the `reduce()` method works by applying the reduction to pairs of elements within the stream to create intermediate values and then combining those intermediate values to produce a final result. Put another way, in a serial stream, `wolf` is built one character at a time. In a parallel stream, the intermediate values `wo` and `lf` are created and then combined.

Türkçe: Parallel stream'de `reduce()`, intermediate value'lar üretmek için reduction'ı element çiftlerine uygular; ardından bunları final result için birleştirir. Başka bir deyişle serial stream `wolf` sözcüğünü karakter karakter kurarken parallel stream önce `wo` ve `lf` gibi ara sonuçlar üretilip sonra birleştirilebilir.

English: With parallel streams, we now have to be concerned about order. What if the elements of a string are combined in the wrong order to produce `wlfo` or `flwo`? The Stream API prevents this problem while still allowing streams to be processed in parallel, as long as you follow one simple rule: make sure that the accumulator and combiner produce the same result regardless of the order they are called in.

Türkçe: Parallel stream'de order ve reduction contract birlikte dikkate alınmalıdır. Bir `String`in element'leri yanlış sırayla birleşip `wlfo` veya `flwo` üretirse ne olur? Ordered stream, geçerli reduction contract altında encounter order'ı koruyabilir; accumulator ile combiner'in arbitrary operand reordering karşısında commutative olması şart değildir.

English: While this is not in scope for the exam, the accumulator and combiner must be associative, non-interfering, and stateless. Don't panic; you don't need to know advanced math terms for the exam!

Türkçe: Ayrıntısı sınav kapsamında olmasa da accumulator ve combiner associative, non-interfering ve stateless olmalıdır. Endişelenmeyin; sınav için ileri matematik terimlerini bilmeniz gerekmez.

Java 17 contract'ı: Identity gerçek bir identity olmalı; accumulator ve combiner associative, stateless ve non-interfering çalışmalı; combiner da accumulator ile compatible olmalıdır. Commutativity genel bir zorunluluk değildir: `String::concat` ordered reduction için noncommutative olduğu hâlde geçerlidir.

English: While the requirements for the input arguments to the `reduce()` method hold true for both serial and parallel streams, you may not have noticed any problems in serial streams because the result was always ordered. With parallel streams, though, order is no longer guaranteed, and any argument that violates these rules is much more likely to produce side effects or unpredictable results.

Türkçe: `reduce()` input argument'larına ilişkin kurallar hem serial hem parallel stream için geçerlidir. Gösterilen ordered source serial işlendiğinde contract ihlali fark edilmeyebilir. Parallel partition ve merge adımları ise geçersiz identity, non-associative operation veya side effect'in öngörülemez sonucunu daha görünür hâle getirir; parallel olmak tek başına encounter order'ı kaldırmaz.

English: Let's take a look at an example using a problematic accumulator. In particular, order matters when subtracting numbers; therefore, the following code can output different values depending on whether you use a serial or parallel stream. We can omit a combiner parameter in these examples, as the accumulator can be used when the intermediate data types are the same.

Türkçe: Problemlili bir accumulator kullanarak bir örneğe bakalım. Özellikle, sayıları çıkarırken sıra önemlidir; bu nedenle, aşağıdaki kod seri veya parallel stream kullanıp kullanmadığınıza bağlı olarak farklı değerler çıkarabilir. Bu örneklerde combiner parametresini atlayabiliriz, çünkü ara veri türleri aynı olduğunda accumulator kullanılabilir.

```
System.out.println(List.of(1,2,3,4,5,6)
    .parallelStream()
    .reduce(0, (a, b) -> (a - b))); // PROBLEMATIC ACCUMULATOR
```

English: It may output -21, 3, or some other value.

Türkçe: Çıktı -21, 3 veya başka bir value olabilir.

English: You can see other problems if we use an identity parameter that is not truly an identity value. For example, what do you expect the following code to output?

Türkçe: Gerçek bir identity value olmayan parameter kullanılırsa başka sorunlar ortaya çıkar. Örneğin aşağıdaki kodun ne yazdırmasını beklersiniz?

```
System.out.println(List.of("w","o","l","f")
    .parallelStream()
    .reduce("X", String::concat)); // XwXoXlXf
```

English: On a serial stream, it prints Xwolf, but on a parallel stream, the result is XwXoXlXf. As part of the parallel process, the identity is applied to multiple elements in the stream, resulting in very unexpected data.

Türkçe: Serial stream Xwolf yazdırır; kaynak uygulamadaki sample parallel output XwXoXlXf'tir. Ancak "X", `String::concat` için gerçek identity olmadığından reduction contract ihlal edilir ve portable exact parallel output garanti edilemez.

English: Selecting a `reduce()` Method Although the one- and two-argument versions of `reduce()` support parallel processing, it is recommended that you use the three-argument version of `reduce()` when working with parallel streams. Providing an explicit combiner method allows the JVM to partition the operations in the stream more efficiently.

Türkçe: Bir `reduce()` method'u seçme: `reduce()` 'un bir ve iki argument'lı sürümleri parallel processing'i desteklese de parallel stream'de üç argument'lı sürüm önerilir. Explicit combiner verilmesi JVM'in stream operation'larını daha verimli partition etmesini sağlar.

Combining Results with `collect()`

English: Like `reduce()`, the Stream API includes a three-argument version of `collect()` that takes accumulator and combiner operators along with a supplier operator instead of an identity.

Türkçe: `reduce()` gibi Stream API de üç argument'lı bir `collect()` sürümü sağlar. Bu sürüm identity yerine supplier; ayrıca accumulator ve combiner operation'ları alır.

English: `<R> R collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R,R> combiner)`

Türkçe: Method signature aynen şöyledir: `<R> R collect(Supplier<R> supplier, BiConsumer<R,? super T> accumulator, BiConsumer<R,R> combiner)`.

English: Also, like `reduce()`, the accumulator and combiner operations must be able to process results in any order. In this manner, the three-argument version of `collect()` can be performed as a parallel reduction, as shown in the following example:

Türkçe: `reduce()` gibi üç argument'lı `collect()` de parallel reduction gerçekleştirebilir. Supplier, accumulator ve combiner stateless/non-interfering olmalı; accumulator ile combiner sonuç container'larını associative ve mutually compatible biçimde işlemelidir. `ArrayList::add/addAll` gibi order-sensitive operation'lar geçerlidir; commutativity şart değildir.

```
Stream<String> stream = Stream.of("w", "o", "l", "f").parallel();
SortedSet<String> set = stream.collect(ConcurrentSkipListSet::new,
Set::add,
Set::addAll);
System.out.println(set); // [f, l, o, w]
```

English: Recall that elements in a `ConcurrentSkipListSet` are sorted according to their natural ordering. You should use a concurrent collection to combine the results, ensuring that the results of concurrent threads do not cause a `ConcurrentModificationException`.

Türkçe: `ConcurrentSkipListSet` element'lerinin natural order'a göre sıralandığını hatırlayın. Kaynak, concurrent thread sonuçlarını birleştirirken `ConcurrentModificationException` oluşmasını önlemek için concurrent collection kullanılmasını önerir.

Java 17 editor notu: Üç argümanlı `collect()` için concurrent collection zorunlu değildir. Parallel partition'lar ayrı mutable container'larda biriktirilip combiner ile güvenli biçimde birleştirilebilir.

English: Performing parallel reductions with a collector requires additional considerations. For example, if the collection into which you are inserting is an ordered data set, such as a List, the elements in the resulting collection must be in the same order, regardless of whether you use a serial or parallel stream. This may reduce performance, though, as some operations cannot be completed in parallel.

Türkçe: Collector ile parallel reduction ek noktalar dikkate alınarak yapılmalıdır. Ordered upstream ile order-preserving collector kullanılıyorsa `List` gibi result collection'daki element'ler encounter order'ı korur. Unordered upstream veya `UNORDERED` collector için bu garanti yoktur. Order constraint'i bazı operation'ların parallel tamamlanmasını zorlaştırıp performansı düşürebilir.

Performing a Parallel Reduction on a Collector

English: While we covered the Collector interface in Chapter 10, we didn't go into detail about its properties. Every Collector instance defines a `characteristics()` method that returns a set of `Collector.Characteristics` attributes. When using a Collector to perform a parallel reduction, a number of properties must hold true. Otherwise, the `collect()` operation will execute in a single-threaded fashion.

Türkçe: Chapter 10'da `Collector` interface'i ele alınmış, fakat özellikleri ayrıntılandırılmamıştı. Her `Collector` instance'ı bir `Collector.Characteristics` attribute set'i döndüren `characteristics()` method'unu tanımlar. Collector ile concurrent parallel reduction yapılabilmesi için aşağıdaki koşullar sağlanmalıdır; aksi hâlde `collect()` partition sonuçlarını birleştirerek çalışır, tek shared result container'ında concurrent accumulation yapmaz.

English: Requirements for Parallel Reduction with `collect()` • The stream is parallel. • The parameter of the `collect()` operation has the `Characteristics.CONCURRENT` characteristic. • Either the stream is unordered or the collector has the characteristic `Characteristics.UNORDERED`.

Türkçe: `collect()` ile concurrent parallel reduction koşulları şunlardır: stream parallel olmalıdır; `Collector.Characteristics.CONCURRENT` taşınmalıdır; ayrıca stream unordered olmalı veya collector `Characteristics.UNORDERED` taşınmalıdır.

English: For example, while `Collectors.toSet()` does have the `UNORDERED` characteristic, it does not have the `CONCURRENT` characteristic. Therefore, the following is not a parallel reduction even with a parallel stream:

Türkçe: Örneğin `Collectors.toSet()` `UNORDERED` characteristic'ına sahip olsa da `CONCURRENT` değildir. Bu nedenle aşağıdaki işlem parallel stream kullanmasına rağmen tek shared result container'ına yapılan concurrent reduction değildir:

```
parallelStream.collect(Collectors.toSet()); // Parallel; not one shared concurrent result
```

Java 17 editor notu: Kaynak “parallel reduction” terimini burada dar anlamda, tek shared result container üzerinde **concurrent accumulation** için kullanıyor. Pipeline gerçekte parallel çalışabilir ve partition-local set'leri merge ederek parallel reduction yapabilir; yalnız `Collectors.toSet()` `CONCURRENT` değildir.

English: The `Collectors` class includes two sets of static methods for retrieving collectors, `toConcurrentMap()` and `groupingByConcurrent()`, both of which are `UNORDERED` and `CONCURRENT`. These methods produce `Collector` instances capable of performing parallel reductions efficiently. Like their nonconcurrent counterparts, there are overloaded versions that take additional arguments.

Türkçe: `Collectors` class'ı `toConcurrentMap()` ve `groupingByConcurrent()` adlı iki static method ailesi içerir; bunların collector'ları hem `UNORDERED` hem `CONCURRENT` characteristic'ına sahiptir. Bu method'lar parallel reduction'ı verimli yapabilen `Collector` instance'ları üretir. Nonconcurrent karşılıkları gibi ek argument alan overload'ları da vardır.

English: Here is a rewrite of an example from Chapter 10 to use a parallel stream and parallel reduction:

Türkçe: Chapter 10'daki bir örneğin parallel stream ve parallel reduction kullanacak biçimde yeniden yazımı şöyledir:

```
Stream<String> ohMy = Stream.of("lions", "tigers", "bears").parallel();
ConcurrentMap<Integer, String> map = ohMy
    .collect(Collectors.toConcurrentMap(String::length,
        k -> k,
        (s1, s2) -> s1 + "," + s2));
System.out.println(map); // Possible: {5=lions,bears, 6=tigers}
System.out.println(map.getClass()); // java.util.concurrent.ConcurrentHashMap
```

English: We use a `ConcurrentMap` reference, although the actual class returned is likely `ConcurrentHashMap`. The particular class is not guaranteed; it will just be a class that implements the interface `ConcurrentMap`.

Türkçe: `ConcurrentMap` reference'ı kullanılır; dönen gerçek class büyük olasılıkla `ConcurrentHashMap`'tir. Belirli class garanti edilmez, yalnızca `ConcurrentMap` interface'ini implement etmesi garanti edilir.

English: Finally, we can rewrite our `groupingBy()` example from Chapter 10 to use a parallel stream and parallel reduction.

Türkçe: Son olarak Chapter 10'daki `groupingBy()` örneği parallel stream ve parallel reduction kullanacak biçimde yeniden yazılabilir.

```
var ohMy = Stream.of("lions", "tigers", "bears").parallel();
ConcurrentMap<Integer, List<String>> map = ohMy.collect(
    Collectors.groupingByConcurrent(String::length));
System.out.println(map); // Possible: {5=[lions, bears], 6=[tigers]}
```

English: As before, the returned object can be assigned to a `ConcurrentMap` reference.

Türkçe: Önceki örnekte olduğu gibi dönen object bir `ConcurrentMap` reference'ına atanabilir.

Output notu: İki code comment'i sample output'tur. `ConcurrentMap` iteration order, `toConcurrentMap()` merge order ve `groupingByConcurrent()` içindeki per-key list order garanti edilmez; örneğin `lions,bears` yerine `bears,lions` görülebilir.

English: Avoiding Stateful Streams Side effects can appear in parallel streams if your lambda expressions are stateful. A stateful lambda expression is one whose result depends on any state that might change during the execution of a pipeline. For example, the following method that filters out even numbers is stateful:

Türkçe: Stateful stream'lerden kaçınma: Lambda expression stateful ise parallel stream'de side effect oluşabilir. Stateful lambda'nın sonucu, pipeline yürütülürken değişebilecek bir state'e bağlıdır. Örneğin çift sayıları filtreleyen aşağıdaki method dışarıdaki mutable listeyi değiştirdiği için stateful'dır:

```
public List<Integer> addValues(IntStream source) {
    var data = Collections.synchronizedList(new ArrayList<Integer>());
    source.filter(s -> s % 2 == 0)
        .forEach(i -> { data.add(i); }); // STATEFUL: DON'T DO THIS!
    return data;
}
```

English: Let's say this method is executed with a serial stream:

Türkçe: Bu metodun seri stream ile çalıştırıldığını varsayalım:

```
var list = addValues(IntStream.range(1, 11));
System.out.print(list); // [2, 4, 6, 8, 10]
```

English: Great, the results are in the same order that they were entered. But what if someone else passes in a parallel stream?

Türkçe: Sonuçlar input order ile aynıdır. Peki method'a parallel stream verilirse?

```
var list = addValues(IntStream.range(1, 11).parallel());
System.out.print(list); // Possible: [6, 8, 10, 2, 4]
```

English: Oh, no: our results no longer match our input order! The problem is that our lambda expression is stateful and modifies a list that is outside our stream. We can fix this solution by rewriting our stream operation to be stateless:

Türkçe: Bu kez sonuçlar input order ile eşleşmeyebilir. Sorun lambda expression'ın stateful olması ve stream dışındaki bir list'i değiştirmesidir. Stream operation stateless biçimde yeniden yazılarak sorun giderilebilir:

```
public List<Integer> addValuesBetter(IntStream source) {
    return source.filter(s -> s % 2 == 0)
        .boxed()
        .collect(Collectors.toList());
}
```

English: This method processes the stream and then collects all the results into a new list. It produces the same ordered result on both serial and parallel streams. It is strongly recommended that you avoid stateful operations when using parallel streams, to remove any potential data side effects. In fact, they should be avoided in serial streams since doing so limits the code's ability to someday take advantage of parallelization.

Türkçe: Bu method stream'i işler ve sonuçları yeni bir list'te toplar. Serial ve parallel stream'de aynı ordered sonucu üretir. Olası data side effect'lerini önlemek için özellikle parallel stream'lerde stateful operation'lardan kaçınılmalıdır. Kodun ileride parallel processing'den yararlanabilmesini sınırladıkları için serial stream'lerde de kullanılmamaları önerilir.

Summary

English: This chapter introduced you to threads and outlined some of the key concurrency concepts you need to know for the exam (and to be a better software developer!). You should know how to create and define the thread's work using a Runnable instance, as well as how to pause and interrupt the thread. When working with the Concurrency API, you should also know how to create threads using Callable lambda expressions.

Türkçe: Bu bölüm, thread'leri tanıttı ve sınavda bilmeniz gereken temel eşzamanlılık kavramlarını özetledi. Bir `Runnable` nesnesiyle thread'in yapacağı işi tanımlamayı; thread'i bekletmeyi ve ona kesme isteği göndermeyi bilmelisiniz. Concurrency API kullanırken `Callable` lambda ifadeleriyle görev oluşturmaya da öğrenmiş olmalısınız.

English: At this point, you should know how to concurrently execute tasks using `ExecutorService` like a pro. You should also know which `ExecutorService` instances are available, including scheduled and pooled services.

Türkçe: Bu aşamada `ExecutorService` ile görevleri eşzamanlı yürütebilmeli; zamanlanmış servisler ve thread pool'lar dahil mevcut yürütücü türlerini tanımalısınız.

English: Thread-safety is about protecting data from being corrupted by multiple threads modifying it at the same time. Java offers many tools to keep data safe, including atomic classes, synchronized methods/blocks, the Lock framework, and `CyclicBarrier`. The Concurrency API also includes numerous collection classes that handle multithreaded access for you. You should be familiar with the concurrent collections, including the `CopyOnWrite` classes, which create a new underlying structure any time the underlying collection is modified.

Türkçe: Thread safety, birden fazla thread'in paylaşılan veriyi değiştirmesi nedeniyle oluşabilecek bozulmaları önlemektir. Atomik sınıflar, `synchronized` metod ve bloklar ile `Lock` API'si atomiklik veya karşılıklı dışlama sağlayabilir. `CyclicBarrier`, aşamalar arasında eşgüdüm ve happens-before ilişkisi sağlar; tek başına karşılıklı dışlama veya atomiklik sağlamaz. Concurrency API, çok iş parçacıklı erişimi yöneten koleksiyonları da içerir. Alttaki yapı değiştiğinde yeni bir kopya oluşturan `CopyOnWrite` sınıflarını tanımalısınız.

English: When processing tasks concurrently, a variety of potential threading issues can arise. Deadlock, starvation, and livelock can result in programs that appear stuck, while race conditions can result in unpredictable data. For the exam, you need to know only the basic theory behind these concepts. In professional software development, however, finding and resolving such problems is a valuable skill.

Türkçe: Görevler eşzamanlı işlenirken çeşitli thread sorunları doğabilir. Deadlock, starvation ve livelock programın takılmış gibi görünmesine; race condition ise öngörülemeyen verilere yol açabilir. Kaynak, sınav için bu kavramların temel teorisinin bilinmesini ister. Mesleki yazılım geliştirmede bu sorunları bulup çözmek de değerli bir beceridir.

English: Finally, we discussed parallel streams and showed you how to use them to perform parallel decompositions and reductions. Parallel streams can greatly improve the performance of your application. They can also cause unexpected results since the processing is no longer ordered. Remember to avoid stateful lambda expressions, especially when working with parallel streams.

Türkçe: Son olarak paralel akışlarla veriyi parçalara ayırıp paralel işlemeyi ve sonuçları indirgemeyi ele aldık. Paralel akışlar performansı önemli ölçüde artırabilir. İşlemler farklı sıralarda yürüyebildiği için doğru sözleşme kurulmazsa beklenmedik sonuçlar da doğabilir. Sıralamayı koruyan terminal işlemlerin karşılaşma sırasını koruyabileceğini unutmayın. Özellikle paralel akışlarda dışarıdaki değişebilir duruma bağlı lambda ifadelerinden kaçının.

Exam Essentials

English: Be able to write thread-safe code. Thread-safety is about protecting shared data from concurrent access. A monitor can be used to ensure that only one thread processes a particular section of code at a time. In Java, monitors can be implemented with a synchronized block or method or using an instance of Lock. ReentrantLock has a number of advantages over using a synchronized block, including the ability to check whether a lock is available without blocking it, as well as supporting the fair acquisition of locks. To achieve synchronization, two or more threads must coordinate on the same shared object.

Türkçe: Thread-safe kod yazabilin. Amaç, paylaşılan veriyi eşzamanlı erişimin yol açabileceği bozulmalardan korumaktır. `synchronized` blok veya metot, nesnenin intrinsic monitor mekanizmasını kullanır; `Lock`, ayrıca sunulan açık bir kilitleme API'sidir. İkisi karşılıklı dışlama sağlayabilir; ancak aynı mekanizma değildir. `ReentrantLock`, beklemeden kilit edinmeyi deneme ve adil kilit edinimini destekleme gibi avantajlar sunar. Thread'ler aynı monitor veya aynı `Lock` nesnesi üzerinde eşgüdüm sağlamalıdır.

English: Be able to apply the atomic classes. An atomic operation is one that occurs without interference from another thread. The Concurrency API includes a set of atomic classes that are similar to the primitive classes, except that they ensure that operations on them are

Türkçe: Atomik sınıfları kullanabilin. Atomik işlem, başka bir thread'in gözünde bölünmez biçimde gerçekleşir. Concurrency API, primitive türlerle benzer amaçlara hizmet eden atomik sınıflar sunar.

English: performed atomically. Know the difference between an atomic variable and one marked with the `volatile` modifier.

Türkçe: Bu sınıfların atomik işlemleri bölünmez biçimde uygulanır. Atomik bir değişken ile `volatile` olarak bildirilmiş bir değişken arasındaki farkı bilin.

English: Create concurrent tasks with a thread executor service using `Runnable` and `Callable`. An `ExecutorService` creates and manages a single thread or a pool of threads. Instances of `Runnable` and `Callable` can both be submitted to a thread executor and will be completed using the available threads in the service. `Callable` differs from `Runnable` in that `Callable` returns a generic data type and can throw a checked exception. A `ScheduledExecutorService` can be used to schedule tasks at a fixed rate or with a fixed interval between executions.

Türkçe: `Runnable` ve `Callable` ile yürütücüye eşzamanlı görevler gönderin. `ExecutorService`, tek bir thread'i veya bir thread pool'u oluşturup yönetir. Her iki görev türü de yürütücüye gönderilebilir ve servisteki uygun thread'lerde çalıştırılır. `Callable`, `Runnable`'dan farklı olarak tür parametresiyle belirlenen bir değer döndürür ve checked exception fırlatabilir. `ScheduledExecutorService`, görevleri sabit aralıklarla veya bir çalışmanın bitişinden sonrakinin başlangıcına kadar sabit gecikmeyle zamanlayabilir.

English: Be able to use the concurrent collection classes. The Concurrency API includes numerous collection classes that include built-in support for multithreaded processing, such as `ConcurrentHashMap`. It also includes a class `CopyOnWriteArrayList` that creates a copy of its underlying list structure every time it is modified and is useful in highly concurrent environments.

Türkçe: Eşzamanlı koleksiyonları kullanabilin. Concurrency API, `ConcurrentHashMap` gibi çok iş parçacıklı işlemeyi destekleyen koleksiyonlar sunar. `CopyOnWriteArrayList`, değişiklik sırasında alttaki liste yapısını kopyalar; özellikle okumanın sık, yazmanın seyrek olduğu iş yüklerinde yararlıdır. Yalnız "yüksek eşzamanlılık" bulunması tek başına uygunluk ölçütü değildir.

English: Identify potential threading problems. Deadlock, starvation, and livelock are three threading problems that can occur and result in threads never completing their task. Deadlock occurs when two or more threads are blocked forever. Starvation occurs when a single thread is perpetually denied access to a shared resource. Livelock is a form of starvation where two or more threads are active but conceptually blocked forever. Finally, race conditions occur when two threads execute at the same time, resulting in an unexpected outcome.

Türkçe: Olası thread sorunlarını tanımlayın. Deadlock, starvation ve livelock görevlerin tamamlanmasını engelleyebilir. Deadlock durumunda iki veya daha fazla thread birbirini sonsuza kadar bekler. Starvation, bir thread'in paylaşılan kaynağa erişiminin sürekli engellenmesidir. Livelock durumunda thread'ler etkin kalır; ancak işte ilerleme sağlayamaz. Race condition, programın doğruluğunun eşgüdüm sağlanmamış işlemlerin zamanlamasına veya iç içe geçme sırasına bağlı olmasıdır; fiziksel olarak aynı anda yürütülmeleri şart değildir.

English: Understand the impact of using parallel streams. The Stream API allows for the easy creation of parallel streams. Using a parallel stream can cause unexpected results, since the order of operations may no longer be predictable. Some operations, such as `reduce()` and `collect()`, require special consideration to achieve optimal performance when applied to a parallel stream.

Türkçe: Paralel akış kullanımının etkisini anlayın. Stream API, paralel akış oluşturmayı kolaylaştırır. İşlemlerin yürütülme sırası önceden kestirilemeyebileceği için beklenmedik sonuçlar doğabilir. `reduce()` ve `collect()` gibi işlemlerde hem doğru hem verimli sonuç için paralel çalışmaya uygun sözleşmeler kurulmalıdır.

Review Questions

English: The answers to the chapter review questions can be found in the Appendix.

Türkçe: Chapter review sorularının cevapları Appendix'te bulunabilir.

Question 1 / Soru 1

English: 1. Given the following code snippet, which options correctly create a parallel stream? (Choose all that apply.)

Türkçe: 1. Aşağıdaki kod parçasında hangi seçenekler doğru biçimde paralel stream oluşturur? (Uygun olanların tümünü seçin.)

```
var c = new ArrayList<Thread>();  
var s = c.stream();  
var p = _____;
```

English: A. `new ParallelStream(s)`

Türkçe: A. `new ParallelStream(s)`

English: B. `c.parallel()`

Türkçe: B. `c.parallel()`

English: C. `s.parallelStream()`

Türkçe: C. `s.parallelStream()`

English: D. `c.parallelStream()`

Türkçe: D. `c.parallelStream()`

English: E. `new ParallelStream(c)`

Türkçe: E. `new ParallelStream(c)`

English: F. s.parallel()

Türkçe: F. s.parallel()

Question 2 / Soru 2

English: 2. Given that the sum of the numbers from 1 (inclusive) to 10 (exclusive) is 45, what are the possible results of executing the following program? (Choose all that apply.)

Türkçe: 2. 1 (inclusive) ile 10 (exclusive) arasındaki sayıların toplamı 45 olduğuna göre aşağıdaki programın olası sonuçları nelerdir? (Uygun olanların tümünü seçin.)

```
1: import java.util.concurrent.locks.*;
2: import java.util.stream.*;
3: public class Bank {
4:     private Lock vault = new ReentrantLock();
5:     private int total = 0;
6:     public void deposit(int value) {
7:         try {
8:             vault.tryLock();
9:             total += value;
10:        } finally { vault.unlock(); }
11:    }
12:    public static void main(String[] unused) {
13:        var bank = new Bank();
14:        IntStream.range(1, 10).parallel()
15:            .forEach(s -> bank.deposit(s));
16:        System.out.println(bank.total);
17:    } }
```

English: A. 45 is printed.

Türkçe: A. 45 yazdırılır.

English: B. A number less than 45 is printed.

Türkçe: B. 45'ten küçük bir sayı yazdırılır.

English: C. A number greater than 45 is printed.

Türkçe: C. 45'ten büyük bir sayı yazdırılır.

English: D. An exception is thrown.

Türkçe: D. Bir exception atılır.

English: E. None of the above, as the code does not compile.

Türkçe: E. Yukarıdakilerin hiçbiri; kod derlenmez.

Question 3 / Soru 3

English: 3. Which of the following statements about the Callable call() and Runnable run() methods are correct? (Choose all that apply.)

Türkçe: 3. `Callable.call()` ve `Runnable.run()` method'larıyla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English: A. Both methods return void.

Türkçe: A. İki method da `void` döndürür.

English: B. Both can throw unchecked exceptions.

Türkçe: B. Her ikisi de unchecked exception atabilir.

English: C. Both can be implemented with lambda expressions.

Türkçe: C. İkisi de lambda expression ile uygulanabilir.

English: D. Runnable returns a generic type.

Türkçe: D. `Runnable` generic bir type döndürür.

English: E. Both can throw checked exceptions.

Türkçe: E. İkisi de checked exception atabilir.

English: F. Callable returns a generic type.

Türkçe: F. `Callable` generic bir type döndürür.

Question 4 / Soru 4

English: 4. Which lines need to be changed to make the code compile? (Choose all that apply.)

Türkçe: 4. Kodun derlenmesi için hangi line'ların değiştirilmesi gerekir? (Uygun olanların tümünü seçin.)

```
ExecutorService service = // w1
    Executors.newSingleThreadScheduledExecutor();
service.scheduleWithFixedDelay(() -> {
    System.out.println("Open Zoo");
    return null; // w2
}, 0, 1, TimeUnit.MINUTES);
var result = service.submit(() -> // w3
    System.out.println("Wake Staff"));
System.out.println(result.get()); // w4
```

English: A. It compiles and runs without issue.

Türkçe: A. Derlenir ve sorunsuz çalışır.

English: B. Line w1

Türkçe: B. Line w1

English: C. Line w2

Türkçe: C. Line w2

English: D. Line w3

Türkçe: D. Line w3

English: E. Line w4

Türkçe: E. Line w4

English: F. It compiles but throws an exception at runtime.

Türkçe: F. Kod derlenir, ancak runtime'da exception atar.

Question 5 / Soru 5

English: 5. What statement about the following code is true?

Türkçe: 5. Aşağıdaki kodla ilgili hangi ifade doğrudur?

```
var value1 = new AtomicLong(0);
final long[] value2 = {0};
IntStream.iterate(1, i -> 1).limit(100).parallel()
    .forEach(i -> value1.incrementAndGet());
IntStream.iterate(1, i -> 1).limit(100).parallel()
    .forEach(i -> ++value2[0]);
System.out.println(value1+" "+value2[0]);
```

English: A. It outputs 100 100.

Türkçe: A. 100 100 yazdırır.

English: B. It outputs 100 99.

Türkçe: B. 100 99 yazdırır.

English: C. The output cannot be determined ahead of time.

Türkçe: C. Çıktı önceden belirlenemez.

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. It compiles but throws an exception at runtime.

Türkçe: E. Kod derlenir fakat runtime'da exception atar.

English: F. It compiles but enters an infinite loop at runtime.

Türkçe: F. Kod derlenir, ancak runtime'da sonsuz döngüye girer.

English: G. None of the above

Türkçe: G. Yukarıdakilerin hiçbiri

Question 6 / Soru 6

English: 6. Which statements about the following code are correct? (Choose all that apply.)

Türkçe: 6. Aşağıdaki kodla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
var data = List.of(2,5,1,9,8);
data.stream().parallel()
    .mapToInt(s -> s)
    .peek(System.out::print)
    .forEachOrdered(System.out::print);
```

English: A. The peek() method will print the entries in the sorted order: 12589.

Türkçe: A. `peek()` element'leri sorted order'da yazdırır: 12589.

English: B. The peek() method will print the entries in the original order: 25198.

Türkçe: B. `peek()` element'leri original order'da yazdırır: 25198.

English: C. The peek() method will print the entries in an order that cannot be determined ahead of time.

Türkçe: C. `peek()` element'leri önceden belirlenemeyen bir order'da yazdırır.

English: D. The forEachOrdered() method will print the entries in the sorted order: 12589.

Türkçe: D. `forEachOrdered()` element'leri sorted order'da yazdırır: 12589.

English: E. The `forEachOrdered()` method will print the entries in the original order: 25198.

Türkçe: E. `forEachOrdered()` element'leri original order'da yazdırır: 25198.

English: F. The `forEachOrdered()` method will print the entries in an order that cannot be determined ahead of time.

Türkçe: F. `forEachOrdered()` element'leri önceden belirlenemeyen bir order'da yazdırır.

English: G. The code does not compile.

Türkçe: G. Kod derlenmez.

Question 7 / Soru 7

English: 7. Fill in the blanks: _____ occur(s) when two or more threads are blocked forever but both appear active. _____ occur(s) when two or more threads try to complete a related task at the same time, resulting in invalid or unexpected data.

Türkçe: 7. Boşlukları doldurun: İki veya daha fazla thread sonsuza kadar ilerleyemez durumda kaldığı hâlde etkin görünüyorsa _____ oluşur. İki veya daha fazla thread aynı ilişkili görevi eşzamanlı tamamlamaya çalışarak geçersiz ya da beklenmedik veri ürettiğinde _____ oluşur.

English: A. Livelock, Deadlock

Türkçe: A. Livelock, Deadlock

English: B. Deadlock, Starvation

Türkçe: B. Deadlock, Starvation

English: C. Race conditions, Deadlock

Türkçe: C. Race conditions, Deadlock

English: D. Livelock, Race conditions

Türkçe: D. Livelock, Race conditions

English: E. Starvation, Race conditions

Türkçe: E. Starvation, Race conditions

English: F. Deadlock, Livelock

Türkçe: F. Deadlock, Livelock

Question 8 / Soru 8

English: 8. Assuming this class is accessed by only a single thread at a time, what is the result of calling the `countIceCreamFlavors()` method?

Türkçe: 8. Bu class'a aynı anda yalnızca bir thread'in eriştiğini varsayarsak `countIceCreamFlavors()` çağrısının sonucu nedir?

```
import java.util.stream.LongStream;
public class Flavors {
    private static int counter;
    public static void countIceCreamFlavors() {
        counter = 0;
        Runnable task = () -> counter++;
        LongStream.range(0, 500)
            .forEach(m -> new Thread(task).run());
        System.out.println(counter);
    }
}
```

English: A. The method consistently prints a number less than 500.

Türkçe: A. Method her zaman 500'den küçük bir sayı yazdırır.

English: B. The method consistently prints 500.

Türkçe: B. Method her zaman 500 yazdırır.

English: C. The method compiles and prints a value, but that value cannot be determined ahead of time.

Türkçe: C. Method derlenir ve bir değer yazdırır, fakat değer önceden belirlenemez.

English: D. The method does not compile.

Türkçe: D. Method derlenmez.

English: E. The method compiles but throws an exception at runtime.

Türkçe: E. Method derlenir fakat runtime'da exception atar.

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbirisi.

Question 9 / Soru 9

English: 9. Which happens when a new task is submitted to an `ExecutorService` in which no threads are available?

Türkçe: 9. Hiçbir worker thread'i müsait olmayan bir `ExecutorService`'e yeni bir task submit edildiğinde ne olur?

English: A. The executor throws an exception when the task is submitted.

Türkçe: A. Task submit edildiğinde executor exception atar.

English: B. The executor discards the task without completing it.

Türkçe: B. Executor task'ı tamamlamadan discard eder.

English: C. The executor adds the task to an internal queue and completes when there is an available thread.

Türkçe: C. Executor task'ı internal queue'ya ekler ve worker thread müsait olduğunda tamamlar.

English: D. The thread submitting the task waits on the submit call until a thread is available before continuing.

Türkçe: D. Task'ı submit eden thread, worker müsait olana kadar submission call üzerinde bekler.

English: E. The executor stops an existing task and starts the newly submitted one.

Türkçe: E. Executor mevcut bir task'ı durdurup yeni gönderilen task'ı başlatır.

Question 10 / Soru 10

English: 10. What is the result of executing the following code snippet?

Türkçe: 10. Aşağıdaki kod parçasını çalıştırmamanın sonucu nedir?

```
List<Integer> lions = new ArrayList<>(List.of(1,2,3));
List<Integer> tigers = new CopyOnWriteArrayList<>(lions);
Set<Integer> bears = new ConcurrentSkipListSet<>();
bears.addAll(lions);
for(Integer item: tigers) tigers.add(4); // x1
for(Integer item: bears) bears.add(5); // x2
System.out.println(lions.size() + " " + tigers.size()
    + " " + bears.size());
```

English: A. It outputs 3 6 4.

Türkçe: A. 3 6 4 yazdırır.

English: B. It outputs 6 6 6.

Türkçe: B. 6 6 6 yazdırır.

English: C. It outputs 6 3 4.

Türkçe: C. 6 3 4 yazdırır.

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. It compiles but throws an exception at runtime on line x1.

Türkçe: E. Derlenir fakat line x1'de runtime exception atar.

English: F. It compiles but throws an exception at runtime on line x2.

Türkçe: F. Derlenir fakat line x2'de runtime exception atar.

English: G. It compiles but enters an infinite loop at runtime.

Türkçe: G. Derlenir fakat runtime'da infinite loop'a girer.

Question 11 / Soru 11

English: 11. What statements about the following code are true? (Choose all that apply.)

Türkçe: 11. Aşağıdaki kodla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
Integer i1 = List.of(1, 2, 3, 4, 5).stream().findAny().get();
synchronized(i1) { // y1
    Integer i2 = List.of(6, 7, 8, 9, 10)
        .parallelStream()
        .sorted()
        .findAny().get(); // y2
    System.out.println(i1 + " " + i2);
}
```

English: A. The first value printed is always 1.

Türkçe: A. Yazdırılan ilk value her zaman 1 olur.

English: B. The second value printed is always 6.

Türkçe: B. Yazdırılan ikinci value her zaman 6 olur.

English: C. The code will not compile because of line y1.

Türkçe: C. Kod line y1 nedeniyle derlenmez.

English: D. The code will not compile because of line y2.

Türkçe: D. Kod line y2 nedeniyle derlenmez.

English: E. The code compiles but throws an exception at runtime.

Türkçe: E. Kod derlenir fakat runtime'da exception atar.

English: F. The output cannot be determined ahead of time.

Türkçe: F. Output önceden belirlenemez.

English: G. It compiles but waits forever at runtime.

Türkçe: G. Derlenir fakat runtime'da sonsuza kadar bekler.

Question 12 / Soru 12

English: 12. Assuming each call to `takeNap()` takes five seconds to execute without throwing an exception, what is the expected result of executing the following code snippet?

Türkçe: 12. Her `takeNap()` çağrısının exception atmadan beş saniyede tamamlandığını varsayarsak aşağıdaki kod parçasının beklenen sonucu nedir?

```
ExecutorService service = Executors.newFixedThreadPool(4);
try {
    service.execute(() -> takeNap());
    service.execute(() -> takeNap());
    service.execute(() -> takeNap());
} finally {
    service.shutdown();
}
service.awaitTermination(2, TimeUnit.SECONDS);
System.out.println("DONE!");
```

English: A. It will immediately print DONE!.

Türkçe: A. Hemen literal `DONE!` yazdırır.

English: B. It will pause for 2 seconds and then print DONE!.

Türkçe: B. İki saniye bekler, ardından literal `DONE!` yazdırır.

English: C. It will pause for 5 seconds and then print DONE!.

Türkçe: C. Beş saniye bekler, ardından literal `DONE!` yazdırır.

English: D. It will pause for 15 seconds and then print DONE!.

Türkçe: D. On beş saniye bekler, ardından literal `DONE!` yazdırır.

English: E. It will throw an exception at runtime.

Türkçe: E. Runtime'da exception atar.

English: F. None of the above, as the code does not compile.

Türkçe: F. Yukarıdakilerin hiçbirisi; kod derlenmez.

Question 13 / Soru 13

English: 13. What statements about the following code are true? (Choose all that apply.)

Türkçe: 13. Aşağıdaki kodla ilgili hangi ifadeler doğrudur? (Tüm geçerli olanları seçin.)

```
System.out.print(List.of("duck", "flamingo", "pelican")
    .parallelStream().parallel() // q1
    .reduce(0,
        (c1, c2) -> c1.length() + c2.length(), // q2
        (s1, s2) -> s1 + s2)); // q3
```

English: A. It compiles and runs without issue, outputting the total length of all strings in the stream.

Türkçe: A. Sorunsuz derlenip çalışır ve stream'deki bütün **String**'lerin toplam length'ini yazdırır.

English: B. The code will not compile because of line q1.

Türkçe: B. Kod line q1 nedeniyle derlenmez.

English: C. The code will not compile because of line q2.

Türkçe: C. Kod line q2 nedeniyle derlenmez.

English: D. The code will not compile because of line q3.

Türkçe: D. Kod line q3 nedeniyle derlenmez.

English: E. It compiles but throws an exception at runtime.

Türkçe: E. Kod derlenir, ancak runtime'da exception atar.

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbirisi.

Question 14 / Soru 14

English: 14. What statements about the following code snippet are true? (Choose all that apply.)

Türkçe: 14. Aşağıdaki kod parçasıyla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
Object o1 = new Object();
Object o2 = new Object();
var service = Executors.newFixedThreadPool(2);
var f1 = service.submit(() -> {
    synchronized (o1) {
        synchronized (o2) { System.out.print("Tortoise"); }
    }
});
var f2 = service.submit(() -> {
    synchronized (o2) {
        synchronized (o1) { System.out.print("Hare"); }
    }
});
f1.get();
f2.get();
```

English: A. The code will always output Tortoise followed by Hare.

Türkçe: A. Kod her zaman **Tortoise**, ardından **Hare** yazdırır.

English: B. The code will always output Hare followed by Tortoise.

Türkçe: B. Kod her zaman **Hare**, ardından **Tortoise** yazdırır.

English: C. If the code does output anything, the order cannot be determined.

Türkçe: C. Kod herhangi bir çıktı üretirse order önceden belirlenemez.

English: D. The code does not compile.

Türkçe: D. Kod derlenmez.

English: E. The code compiles but may produce a deadlock at runtime.

Türkçe: E. Kod derlenir, ancak runtime'da **deadlock** oluşabilir.

English: F. The code compiles but may produce a livelock at runtime.

Türkçe: F. Kod derlenir fakat runtime'da livelock oluşabilir.

English: G. It compiles but throws an exception at runtime.

Türkçe: G. Derlenir fakat runtime'da exception atar.

Question 15 / Soru 15

English: 15. Which statement about the following code snippet is correct?

Türkçe: 15. Aşağıdaki kod parçasıyla ilgili hangi ifade doğrudur?

```
2: var cats = Stream.of("leopard", "lynx", "ocelot", "puma")
3:   .parallel();
4: var bears = Stream.of("panda", "grizzly", "polar").parallel();
5: var data = Stream.of(cats, bears).flatMap(s -> s)
6:   .collect(Collectors.groupingByConcurrent(
7:     s -> !s.startsWith("p")));
8: System.out.println(data.get(false).size()
9:   + " " + data.get(true).size());
```

English: A. It outputs 3 4.

Türkçe: A. 3 4 yazdırır.

English: B. It outputs 4 3.

Türkçe: B. 4 3 yazdırır.

English: C. The code will not compile because of line 6.

Türkçe: C. Kod line 6 nedeniyle derlenmez.

English: D. The code will not compile because of line 7.

Türkçe: D. Kod line 7 nedeniyle derlenmez.

English: E. The code will not compile because of line 8.

Türkçe: E. Kod line 8 nedeniyle derlenmez.

English: F. It compiles but throws an exception at runtime.

Türkçe: F. Derlenir fakat runtime'da exception atar.

Question 16 / Soru 16

English: 16. Assuming one minute is enough time for all the threads within this program to complete, what are the possible results of executing the following program? (Choose all that apply.)

Türkçe: 16. Programdaki bütün thread'lerin tamamlanması için bir dakikanın yeterli olduğunu varsayarsak programın olası sonuçları nelerdir? (Uygun olanların tümünü seçin.)

```
public class RocketShip {
    private volatile int fuel;
    private void launch(int checks) {
        var p = new ArrayList<Thread>();
        for (int i = 0; i < checks; i++)
            p.add(new Thread(() -> fuel++));
        p.forEach(Thread::interrupt);
        p.forEach(Thread::start);
        p.forEach(Thread::interrupt);
    }
    public static void main(String[] args) throws Exception {
        var ship = new RocketShip();
        ship.launch(100);
        Thread.sleep(60*1000);
        System.out.print(ship.fuel);
    } }
```

English: A. It prints a number less than 100.

Türkçe: A. 100'den küçük bir sayı yazdırır.

English: B. It prints 100.

Türkçe: B. 100 yazdırır.

English: C. It prints a number greater than 100.

Türkçe: C. 100'den büyük bir sayı yazdırır.

English: D. It does not compile.

Türkçe: D. Kod derlenmez.

English: E. It compiles but throws an InterruptedException at runtime.

Türkçe: E. Derlenir fakat runtime'da InterruptedException atar.

Question 17 / Soru 17

English: 17. Which statements about methods in ReentrantLock are correct? (Choose all that apply.)

Türkçe: 17. ReentrantLock method'larıyla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English: A. The lock() method will attempt to acquire a lock without waiting indefinitely for it.

Türkçe: A. lock() süresiz beklemeden lock almaya çalışır.

English: B. The testLock() method will attempt to acquire a lock without waiting indefinitely for it.

Türkçe: B. testLock() süresiz beklemeden lock almaya çalışır.

English: C. The `attemptLock()` method will attempt to acquire a lock without waiting indefinitely for it.

Türkçe: C. `attemptLock()` method'u süresiz beklemeden lock almaya çalışacaktır.

English: D. By default, a `ReentrantLock` fairly releases to each thread in the order in which it was requested.

Türkçe: D. `ReentrantLock` varsayılan olarak lock'ı request sırasına göre fair biçimde thread'lere verir.

English: E. Calling the `unlock()` method once will release a resource so that other threads can obtain the lock.

Türkçe: E. `unlock()` method'unu bir kez çağırmak, diğer thread'lerin lock'u alabilmesi için resource'u serbest bırakır.

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbirisi.

Question 18 / Soru 18

English: 18. Which of the following are valid Callable expressions? (Choose all that apply.)

Türkçe: 18. Aşağıdakilerden hangileri geçerli `Callable` expression'dır? (Uygun olanların tümünü seçin.)

English: A. `a -> {return 10;}`

Türkçe: A. `a -> {return 10;}`

English: B. `() -> {String s = ""};`

Türkçe: B. `() -> {String s = ""};`

English: C. `() -> 5`

Türkçe: C. `() -> 5`

English: D. `() -> {return null}`

Türkçe: D. `() -> {return null}`

English: E. `() -> "The" + "Zoo"`

Türkçe: E. `() -> "The" + "Zoo"`

English: F. (int count) -> count+1

Türkçe: F. (int count) -> count+1

English: G. () -> {System.out.println("Giraffe"); return 10;}

Türkçe: G. () -> {System.out.println("Giraffe"); return 10;}

Question 19 / Soru 19

English: 19. What is the result of executing the following application? (Choose all that apply.)

Türkçe: 19. Aşağıdaki application çalıştırıldığında sonuç ne olur? (Uygun olanların tümünü seçin.)

```
import java.util.concurrent.*;
import java.util.stream.*;
public class PrintConstants {
    public static void main(String[] args) {
        var s = Executors.newScheduledThreadPool(10);
        DoubleStream.of(3.14159, 2.71828) // b1
            .forEach(c -> s.submit( // b2
                () -> System.out.println(10*c))); // b3
        s.execute(() -> System.out.println("Printed"));
    }
}
```

English: A. It compiles and outputs the two numbers followed by Printed.

Türkçe: A. Derlenir; iki sayıyı, ardından literal `Printed` metnini yazdırır.

English: B. The code will not compile because of line b1.

Türkçe: B. Kod line b1 nedeniyle derlenmez.

English: C. The code will not compile because of line b2.

Türkçe: C. Kod line b2 nedeniyle derlenmez.

English: D. The code will not compile because of line b3.

Türkçe: D. Kod line b3 nedeniyle derlenmez.

English: E. It compiles, but the output cannot be determined ahead of time.

Türkçe: E. Derlenir fakat output önceden belirlenemez.

English: F. It compiles but throws an exception at runtime.

Türkçe: F. Derlenir fakat runtime'da exception atar.

English: G. It compiles but waits forever at runtime.

Türkçe: G. Derlenir fakat runtime'da sonsuza kadar bekler.

Question 20 / Soru 20

English: 20. What is the result of executing the following program? (Choose all that apply.)

Türkçe: 20. Aşağıdaki program çalıştırıldığında sonuç ne olur? (Uygun olanların tümünü seçin.)

```
import java.util.*;
import java.util.concurrent.*;
import java.util.stream.*;
public class PrintCounter {
    static int count = 0;
    public static void main(String[] args) throws
        InterruptedException, ExecutionException {
        var service = Executors.newSingleThreadExecutor();
        try {
            var r = new ArrayList<Future<?>>();
            IntStream.iterate(0, i -> i+1).limit(5).forEach(
                i -> r.add(service.execute(() -> {count++;})) // n1
            );
            for(Future<?> result : r) {
                System.out.print(result.get()+" "); // n2
            }
        } finally { service.shutdown(); }
    } }
```

English: A. It prints 0 1 2 3 4

Türkçe: A. 0 1 2 3 4 yazdırır.

English: B. It prints 1 2 3 4 5

Türkçe: B. 1 2 3 4 5 yazdırır.

English: C. It prints null null null null null

Türkçe: C. null null null null null yazdırır.

English: D. It hangs indefinitely at runtime.

Türkçe: D. Runtime'da süresiz biçimde asılı kalır.

English: E. The output cannot be determined.

Türkçe: E. Output önceden belirlenemez.

English: F. The code will not compile because of line n1.

Türkçe: F. Kod line n1 nedeniyle derlenmez.

English: G. The code will not compile because of line n2.

Türkçe: G. Kod line n2 nedeniyle derlenmez.

Question 21 / Soru 21

English: 21. Given the following code snippet and blank lines on p1 and p2, which values guarantee that 1 is printed at runtime? (Choose all that apply.)

Türkçe: 21. Aşağıdaki kod parçasında p1 ve p2 boşluklarına hangi value'lar yazılırsa runtime'da 1 basılması garanti edilir? (Uygun olanların tümünü seçin.)

```
var data = List.of(List.of(1,2),
    List.of(3,4),
    List.of(5,6));
data._____ // p1
    .flatMap(s -> s.stream())
    ._____ // p2
    .ifPresent(System.out::print);
```

English: A. stream() on line p1, findFirst() on line p2

Türkçe: A. p1 satırında `stream()`, p2 satırında `findFirst()`

English: B. stream() on line p1, findAny() on line p2

Türkçe: B. p1 satırında `stream()`, p2 satırında `findAny()`

English: C. parallelStream() on line p1, findAny() on line p2

Türkçe: C. p1 satırında `parallelStream()`, p2 satırında `findAny()`

English: D. parallelStream() on line p1, findFirst() on line p2

Türkçe: D. p1 satırında `parallelStream()`, p2 satırında `findFirst()`

English: E. The code does not compile regardless of what is inserted into the blanks.

Türkçe: E. Boşluklara ne yazılırsa yazılsın kod derlenmez.

English: F. None of the above

Türkçe: F. Yukarıdakilerin hiçbirisi.

Question 22 / Soru 22

English: 22. Assuming one minute is enough time for the tasks submitted to the service executor to complete, what is the result of executing `countSheep()`? (Choose all that apply.)

Türkçe: 22. Executor service'e submit edilen task'ların tamamlanması için bir dakikanın yeterli olduğunu varsayarsak `countSheep()` çalıştırıldığında sonuç ne olur? (Uygun olanların tümünü seçin.)

```
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
public class BedTime {
    private AtomicInteger s1 = new AtomicInteger(0); // w1
    private int s2 = 0;
    private void countSheep() throws InterruptedException {
        var service = Executors.newSingleThreadExecutor(); // w2
        try {
            for (int i = 0; i < 100; i++)
                service.execute(() -> {
                    s1.getAndIncrement(); s2++; }); // w3
            Thread.sleep(60*1000);
            System.out.println(s1 + " " + s2);
        } finally { service.shutdown(); }
    }
    public static void main(String... nap) throws InterruptedException {
        new BedTime().countSheep();
    } }
```

English: A. The method consistently prints 100 99.

Türkçe: A. Method her zaman 100 99 yazdırır.

English: B. The method consistently prints 100 100.

Türkçe: B. Method her zaman 100 100 yazdırır.

English: C. The output cannot be determined ahead of time.

Türkçe: C. Output önceden belirlenemez.

English: D. The code will not compile because of line w1.

Türkçe: D. Kod line w1 nedeniyle derlenmez.

English: E. The code will not compile because of line w2.

Türkçe: E. Kod line w2 nedeniyle derlenmez.

English: F. The code will not compile because of line w3.

Türkçe: F. Kod line w3 nedeniyle derlenmez.

English: G. It compiles but throws an exception at runtime.

Türkçe: G. Derlenir fakat runtime'da exception atar.

Question 23 / Soru 23

English: 23. What is the result of executing the following application? (Choose all that apply.)

Türkçe: 23. Aşağıdaki application çalıştırıldığında sonuç ne olur? (Uygun olanların tümünü seçin.)

```
import java.util.concurrent.*;
import java.util.stream.*;
public class StockRoomTracker {
    public static void await(CyclicBarrier cb) { // j1
        try { cb.await(); } catch (Exception e) {}
    }
    public static void main(String[] args) {
        var cb = new CyclicBarrier(10,
            () -> System.out.println("Stock Room Full!")); // j2
        IntStream.iterate(1, i -> 1).limit(9).parallel()
            .forEach(i -> await(cb)); // j3
    } }
```

English: A. It outputs Stock Room Full!

Türkçe: A. Literal olarak `Stock Room Full!` yazdırır.

English: B. The code will not compile because of line j1.

Türkçe: B. Kod line j1 nedeniyle derlenmez.

English: C. The code will not compile because of line j2.

Türkçe: C. Kod line j2 nedeniyle derlenmez.

English: D. The code will not compile because of line j3.

Türkçe: D. Kod line j3 nedeniyle derlenmez.

English: E. It compiles but throws an exception at runtime.

Türkçe: E. Derlenir fakat runtime'da exception atar.

English: F. It compiles but waits forever at runtime.

Türkçe: F. Derlenir fakat runtime'da sonsuza kadar bekler.

Question 24 / Soru 24

English: 24. What statements about the following class definition are true? (Choose all that apply.)

Türkçe: 24. Aşağıdaki class definition ile ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
public final class TicketManager {  
    private int tickets;  
    private static TicketManager instance;  
    private TicketManager() {}  
    static synchronized TicketManager getInstance() { // k1  
        if (instance==null) instance = new TicketManager(); // k2  
        return instance;  
    }  
    public int getTicketCount() { return tickets; }  
    public void addTickets(int value) {tickets += value;} // k3  
    public void sellTickets(int value) {  
        synchronized (this) { // k4  
            tickets -= value;  
        } } }  
}
```

English: A. It compiles without issue.

Türkçe: A. Sorunsuz bir şekilde derlenir.

English: B. The code will not compile because of line k2.

Türkçe: B. Kod line k2 nedeniyle derlenmez.

English: C. The code will not compile because of line k3.

Türkçe: C. Kod line k3 nedeniyle derlenmez.

English: D. The locks acquired on k1 and k4 are on the same object.

Türkçe: D. k1 ve k4 üzerinde elde edilen locks aynı nesne üzerindedir.

English: E. The class correctly protects the tickets data from race conditions.

Türkçe: E. Class, `tickets` data'sını race condition'lardan doğru biçimde korur.

English: F. At most one instance of TicketManager will be created in an application that uses this class.

Türkçe: F. Bu class'ı kullanan bir uygulamada en fazla bir `TicketManager` instance'ı oluşturulur.

Question 25 / Soru 25

English: 25. Assuming an implementation of the `performCount()` method is provided prior to runtime, which of the following are possible results of executing the following application? (Choose all that apply.)

Türkçe: 25. `performCount()` implementation'ının runtime'dan önce sağlandığını varsayarsak aşağıdaki uygulama çalıştırıldığında hangi sonuçlar mümkündür? (Uygun olanların tümünü seçin.)

```
import java.util.*;
import java.util.concurrent.*;
public class CountZooAnimals {
    public static void performCount(int animal) {
        // IMPLEMENTATION OMITTED
    }
    public static void printResults(Future<?> f) {
        try {
            System.out.println(f.get(1, TimeUnit.DAYS)); // o1
        } catch (Exception e) {
            System.out.println("Exception!");
        }
    }
    public static void main(String[] args) throws Exception {
        final var r = new ArrayList<Future<?>>();
        ExecutorService s = Executors.newSingleThreadExecutor();
        try {
            for(int i = 0; i < 10; i++) {
                final int animal = i;
                r.add(s.submit(() -> performCount(animal))); // o2
            }
            r.forEach(f -> printResults(f));
        } finally { s.shutdown(); }
    } }
```

English: A. It outputs a number 10 times.

Türkçe: A. On kez bir sayı yazdırır.

English: B. It outputs a Boolean value 10 times.

Türkçe: B. On kez bir **Boolean** değeri yazdırır.

English: C. It outputs a null value 10 times.

Türkçe: C. On kez **null** yazdırır.

English: D. It outputs Exception! 10 times.

Türkçe: D. On kez literal **Exception!** yazdırır.

English: E. It hangs indefinitely at runtime.

Türkçe: E. Runtime'da süresiz olarak takılı kalır.

English: F. The code will not compile because of line o1.

Türkçe: F. Kod line o1 nedeniyle derlenmez.

English: G. The code will not compile because of line o2.

Türkçe: G. Kod line o2 nedeniyle derlenmez.

Appendix · Official Review Question Answers / Resmî Cevaplar

Aşağıdaki cevaplar kaynak Appendix bölümündeki sıra ve gerekçeleri korur. Türkçe bloklar doğal teknik çeviridir.

Official Answer 1

English: 1. D, F. There is no such class within the Java API called `ParallelStream`, so options A and E are incorrect. The method defined in the `Stream` class to create a parallel stream from an existing stream is `parallel()`; therefore, option F is correct, and option C is incorrect.

Türkçe: 1. D, F. Java API'de `ParallelStream` adlı bir class yoktur; dolayısıyla A ve E yanlıştır. Mevcut bir stream'i paralel hâle getiren `Stream` method'u `parallel()` olduğundan F doğru, C yanlıştır.

English: The method defined in the `Collection` class to create a parallel stream from a collection is `parallelStream()`; therefore, option D is correct, and option B is incorrect.

Türkçe: Bir collection'dan paralel stream oluşturan `Collection` method'u `parallelStream()` olduğundan D doğru, B yanlıştır.

Official Answer 2

English: 2. A, D. The `tryLock()` method returns immediately with a value of `false` if the lock cannot be acquired. Unlike `lock()`, it does not wait for a lock to become available. This code fails to check the return value on line 8, resulting in the protected code being entered regardless of whether the lock is obtained. In some executions (when `tryLock()` returns `true` on every call), the code will complete successfully and print 45 at runtime, making option A correct.

Türkçe: 2. A, D. `tryLock()`, lock alınmazsa `false` ile hemen döner; `lock()` 'tan farklı olarak lock'ın kullanılabilir olmasını beklemes. Kod line 8'de return value'yu kontrol etmediği için lock alınmasa da protected code'a girer. `tryLock()` her çağrıda `true` dönerse kod başarıyla tamamlanıp runtime'da 45 yazdırabilir; bu nedenle A doğrudur.

English: On other executions (when `tryLock()` returns `false` at least once), the `unlock()`

Türkçe: Diğer execution'larda, yani `tryLock()` en az bir kez `false` döndüğünde, `unlock()`

English: method on line 10 will throw an `IllegalMonitorStateException` at runtime, making option D correct. Option B would be possible if line 10 did not throw an exception.

Türkçe: çağrısı line 10'da runtime `IllegalMonitorStateException` atar; dolayısıyla D doğrudur. Line 10 exception atmasaydı B de mümkün olabilirdi.

Official Answer 3

English: 3. B, C, F. `Runnable` returns `void` and `Callable` returns a generic type, making options A and D incorrect and option F correct. All methods are capable of throwing unchecked exceptions, so option B is correct. Only `Callable` is capable of throwing checked exceptions, so option E is incorrect. Both `Runnable` and `Callable` are functional interfaces that can be implemented with a lambda expression, so option C is also correct.

Türkçe: 3. B, C, F. `Runnable.run()` `void`, `Callable.call()` generic bir type döndürür; A ve D yanlış, F doğrudur. İki method da unchecked exception atabildiği için B doğrudur. Yalnız `Callable.call()` checked exception bildirebildiğinden E yanlıştır. `Runnable` ve `Callable` lambda expression ile uygulanabilen functional interface'ler olduğundan C de doğrudur.

Official Answer 4

English: 4. B, C. The code does not compile, so options A and F are incorrect. The first problem is that although a `ScheduledExecutorService` is created, it is assigned to an `ExecutorService`. The type of the variable on line w1 would have to be updated to `ScheduledExecutorService` for the code to compile, making option B correct. The second problem is that `scheduleWithFixedDelay()` supports only `Runnable`, not `Callable`, and any attempt to return a value is invalid in a `Runnable` lambda expression; therefore, line w2 will also not compile, and option C is correct. The rest of the lines compile without issue, so options D and E are incorrect.

Türkçe: 4. B, C. Kod derlenmez; A ve F yanlıştır. İlk sorun, `ScheduledExecutorService` oluşturulmasına rağmen variable'ın `ExecutorService` türünde olmasıdır. Kodun derlenmesi için line w1'deki type `ScheduledExecutorService` olmalıdır; B doğrudur. İkinci sorun, `scheduleWithFixedDelay()` method'unun yalnız `Runnable` kabul etmesi ve `Runnable` lambda'dan value döndürmenin geçersiz olmasıdır. Line w2 de derlenmez; C doğrudur. Diğer satırlar sorunsuz derlendiğinden D ve E yanlıştır.

Official Answer 5

English: 5. C. The code compiles and runs without throwing an exception or entering an infinite loop, so options D, E, and F are incorrect. The key here is that the increment operator `++` is not atomic. While the first part of the output will always be 100, the second part is nondeterministic. It may output any value from 1 to 100, because the threads can overwrite each other's work. Therefore, option C is the correct answer, and options A and B are incorrect.

Türkçe: 5. C. Kod derlenip exception atmadan ve infinite loop'a girmeden çalışır; D, E ve F yanlıştır. Temel nokta `++` operation'ının atomic olmamasıdır. Çıktının ilk değeri her zaman 100, ikinci değeri nondeterministic'tir. Thread'ler birbirlerinin update'lerini ezebildiği için ikinci değer 1–100 arasında olabilir. C doğru; A ve B yanlıştır.

Official Answer 6

English: 6. C, E. The code compiles, so option G is incorrect. The `peek()` method on a parallel stream will process the elements concurrently, so the order cannot be determined ahead of time, and option C is correct. The `forEachOrdered()` method will process the elements in the order in which they are stored in the stream, making option E correct. None of the methods sort the elements, so options A and D are incorrect.

Türkçe: 6. C, E. Kod derlenir; G yanlıştır. Parallel stream'deki `peek()` element'leri concurrently işler, dolayısıyla order önceden belirlenemez ve C doğrudur. `forEachOrdered()` element'leri stream'deki encounter order'a göre işlediğinden E doğrudur. Bu method'ların hiçbiri element'leri sort etmez; A ve D yanlıştır.

Official Answer 7

English: 7. D. Livelock occurs when two or more threads are conceptually blocked forever, although they are each still active and trying to complete their task. A race condition is an undesirable result that occurs when two tasks that should have been completed sequentially are completed at the same time. For these reasons, option D is correct.

Türkçe: 7. D. Livelock, iki veya daha fazla thread aktif kalıp task'ını tamamlamaya çalıştığı hâlde kavramsal olarak sonsuza dek ilerleyemediğinde oluşur. Race condition, sıralı tamamlanması gereken iki task'ın aynı anda tamamlanmasıyla oluşan istenmeyen sonuçtur. Bu nedenle D doğrudur.

Official Answer 8

English: 8. B. Be wary of `run()` vs. `start()` on the exam! The method looks like it executes a task concurrently, but it runs synchronously. In each iteration of the `forEach()` loop, the process waits for the `run()` method to complete before moving on. For this reason, the code is thread-safe. Since the program consistently prints 500 at runtime, option B is correct.

Türkçe: 8. B. Sınavda `run()` ile `start()` ayrımına dikkat edin. Kod task'ı concurrent yürütüyor gibi görünse de `run()` ordinary method call olduğu için gerçekte synchronous ve sıralı çalışır. `forEach()` her iteration'da ilerlemeden önce `run()` in tamamlanmasını bekler. Bu nedenle kod thread-safe'tir ve runtime'da daima 500 yazdırır; B doğrudur.

English: Note that if `start()` had been used instead of `run()` (or the stream was parallel), then the output would be indeterminate, and option C would have been correct.

Türkçe: `run()` yerine `start()` kullanılsaydı veya stream parallel olsaydı output belirsiz olur ve C doğru olurdu.

Official Answer 9

English: 9. C. If a task is submitted to a thread executor, and the thread executor does not have any available threads, the call to the task will return immediately with the task being queued internally by the thread executor. For this reason, option C is the correct answer.

Türkçe: 9. C. Task executor'a submit edildiğinde müsait worker thread yoksa submission call hemen döner ve task executor'ın internal queue'suna alınır. Bu nedenle C doğrudur.

Java 17 editör notu: Yukarıdaki C, kaynağın resmî cevabıdır ve belirli `ThreadPoolExecutor` yapılandırmaları için geçerlidir; fakat generic `ExecutorService` interface'i her durumda internal queue garantilemez. Cached pool yeni worker oluşturabilir; bounded queue ve rejection policy kullanan bir executor task'ı reddedebilir. Bu nedenle soru, concrete executor türü verilmeden Java 17 açısından tek cevaplı değildir.

Official Answer 10

English: 10. A. The code compiles without issue, so option D is incorrect. The `CopyOnWriteArrayList` class is designed to preserve the original list on iteration, so the first loop will be executed exactly three times and, in the process, will increase the size of tigers to six elements. The `ConcurrentSkipListSet` class allows modifications, and since it enforces the uniqueness of its elements, the value 5 is added only once, leading to a total of four elements in bears. Finally, despite using the elements of lions to populate the collections, tigers and bears are not backed by the original list, so the size of lions is 3 throughout this program. For these reasons, the program prints 3 6 4, and option A is correct.

Türkçe: 10. A. Kod sorunsuz derlenir; D yanlıştır. `CopyOnWriteArrayList`, iteration sırasında özgün snapshot'ı koruduğu için ilk loop tam üç kez çalışır ve `tigers` size'ı altıya çıkar. `ConcurrentSkipListSet` değişikliğe izin verir; element uniqueness uyguladığı için 5 yalnız bir kez eklenir ve `bears` dört element içerir. `tigers` ve `bears`, `lions` element'lerinden oluşturulsa da özgün list tarafından backed değildir; `lions` size'ı program boyunca üçtür. Program 3 6 4 yazdırır ve A doğrudur.

Official Answer 11

English: 11. F. The code compiles and runs without issue, so options C, D, E, and G are incorrect.

Türkçe: 11. F. Kod sorunsuz bir şekilde derlenir ve çalışır, bu nedenle C, D, E ve G seçenekleri yanlıştır.

English: There are two important things to notice. First, synchronizing on the first variable doesn't impact the results of the code. Second, sorting on a parallel stream does not mean that `findAny()` will return the first record. The `findAny()` method will return the value from the first thread that retrieves a record. Therefore, the output is not guaranteed, and option F is correct. Option A looks correct, but even on serial streams, `findAny()` is free to select any element.

Türkçe: Dikkat edilmesi gereken iki önemli şey var. İlk olarak, ilk değişken üzerinde senkronize etmek kodun sonuçlarını etkilemez. İkincisi, bir parallel stream üzerinde sıralama yapmak, `findAny()` ilk kaydı iade edeceği anlamına gelmez. `findAny()` metodu, bir kaydı alan ilk thread değerini döndürür. Bu nedenle, çıkış garanti edilmez ve F seçeneği doğrudur. A seçeneği doğru görünüyor, ancak serial stream'lerde bile `findAny()` herhangi bir öğeyi seçmekte özgürdür.

Official Answer 12

English: 12. B. The code snippet submits three tasks to an `ExecutorService`, shuts it down, and then waits for the results. The `awaitTermination()` method waits a specified amount of time for all tasks to complete and the service to finish shutting down. Since each five-second task is still executing, the `awaitTermination()` method will return with a value of `false` after two seconds but not throw an exception. For these reasons, option B is correct.

Türkçe: 12. B. Kod üç task'ı `ExecutorService`'e submit eder, service'i shutdown eder ve sonuçları bekler. `awaitTermination()` bütün task'ların tamamlanıp service'in kapanması için belirtilen süre kadar bekler. Beşer saniyelik task'lar sürerken iki saniye sonunda exception atmadan `false` döner. Bu nedenle B doğrudur.

Java 17 editor notu: `awaitTermination()` checked `InterruptedException` bildirir. Snippet'i çevreleyen method bu exception'ı `catch` etmiyor veya `throws` ile bildirmiyorsa kod gösterildiği hâliyle **Does not compile**. Kaynak cevap B, bu checked exception'ın dış bağlamda ele alındığını varsayar.

Official Answer 13

English: 13. C. The code does not compile, so options A and E are incorrect. The problem here is that `c1` is an `Integer` and `c2` is a `String`, so the code fails to combine on line q2, since calling `length()` on an `Integer` is not allowed, and option C is correct. The rest of the lines compile without issue. Note that calling `parallel()` on an already parallel stream is allowed, and it may return the same object.

Türkçe: 13. C. Kod derlenmez; A ve E yanlıştır. `c1` bir `Integer`, `c2` bir `String` olduğundan line q2'de `Integer` üzerinde `length()` çağrılmaz; C doğrudur. Diğer satırlar sorunsuz derlenir. Zaten parallel olan stream'de `parallel()` çağrısına izin verilir ve aynı object dönebilir.

Official Answer 14

English: 14. C, E. The code compiles without issue, so option D is incorrect. Since both tasks are submitted to the same thread executor pool, the order cannot be determined, so options A and B are incorrect, and option C is correct. The key here is that the order in which the resources `o1` and `o2` are synchronized could result in a deadlock. For example, if the first thread gets a lock on `o1` and the second thread gets a lock on `o2` before either thread can get their second lock, the code will hang at runtime, making option E correct. The code cannot produce a livelock, since both threads are waiting, so option F is incorrect. Finally, if a deadlock does occur, an exception will not be thrown, so option G is incorrect.

Türkçe: 14. C, E. Kod sorunsuz derlenir; D yanlıştır. İki task aynı executor pool'a submit edildiği için output order bilinmez; A ve B yanlış, C doğrudur. `o1` ve `o2` üzerinde lock alma sırası deadlock oluşturabilir. Birinci thread `o1`i, ikinci thread `o2`yi alıp ikisi de ikinci lock'ı beklerse program runtime'da asılı kalır; E doğrudur. İki thread de beklediği için livelock oluşmaz ve F yanlıştır. Deadlock exception üretmediğinden G de yanlıştır.

Java 17 lifecycle notu: C ve E kaynak cevapları değişmez. Bununla birlikte snippet, `service.shutdown()` çağırmadığı için deadlock oluşmasa ve iki `get()` de dönse bile fixed pool'un non-daemon worker thread'leri JVM'i açık tutar. Standalone programda executor `finally` içinde kapatılmalıdır.

Official Answer 15

English: 15. A. The code compiles and runs without issue, so options C, D, E, and F are incorrect. The `collect()` operation groups the animals into those that do and do not start with the letter p. Note that there are four animals that do not start with the letter p and three animals that do. The logical complement operator (!) before the `startsWith()` method means that results are reversed, so the output is 3 4, and option A is correct, making option B incorrect.

Türkçe: 15. A. Kod derlenir ve sorunsuz çalışır, bu nedenle C, D, E ve F seçenekleri yanlıştır. `collect()` işlemi hayvanları p harfi ile başlayan ve yapmayanlara ayırır. P harfi ile başlamayan dört hayvan ve bunu yapan üç hayvan olduğunu unutmayın. `startsWith()` yönteminden önce mantıksal kompleman operatörü (!) sonuçların tersine çevrildiği anlamına gelir, bu nedenle çıktı 3 4'tür ve A seçeneği doğrudur, B seçeneğini yanlış yapar.

Official Answer 16

English: 16. A, B. The code compiles just fine. If the calls to `fuel++` are ordered sequentially, then the program will print 100 at runtime, making option B correct. On the other hand, the calls may overwrite each other. The `volatile` attribute only guarantees memory consistency, not thread-safety, making option A correct and option C incorrect. Option E is also incorrect, as no `InterruptedException` is thrown by this code. Remember, `interrupt()` only impacts a thread that is in a `WAITING` or `TIMED_WAITING` state. Calling `interrupt()` on a thread in a `NEW` or `RUNNABLE` state has no impact unless the code is running and explicitly checking the `isInterrupted()` method.

Türkçe: 16. A, B. Kod sorunsuz derlenir. `fuel++` çağrıları sequential yürütülürse program runtime'da 100 yazdırır ve B doğrudur. Öte yandan `update`'ler birbirini ezebilir. `volatile` yalnız memory consistency/visibility sağlar; thread-safety ve atomicity sağlamaz. Bu nedenle A doğru, C yanlıştır. Kod `InterruptedException` atmadığı için E de yanlıştır. `interrupt()` özellikle `WAITING` veya `TIMED_WAITING` state'indeki interruptible operation'ı etkiler. `NEW` thread'de etkisizdir; `RUNNABLE` thread'de `interrupt` status set edilir ve çalışan kod bunu kontrol etmedikçe execution kendiliğinden durmaz.

Official Answer 17

English: 17. F. The `lock()` method will wait indefinitely for a lock, so option A is incorrect. Options B and C are also incorrect, as the correct method name to attempt to acquire a lock is `tryLock()`. Option D is incorrect, as fairness is set to false by default and must be enabled by using an overloaded constructor. Finally, option E is incorrect because a thread that holds the lock may have called `lock()` or `tryLock()` multiple times. A thread needs to call `unlock()` once for each call to `lock()` and successful `tryLock()`. Option F is the correct answer since none of the other options are valid statements.

Türkçe: 17. F. `lock()` lock için süresiz bekleyebileceğinden A yanlıştır. Lock'ı beklemeden almaya çalışan doğru method `tryLock()` olduğundan B ve C yanlıştır. Fairness varsayılan olarak `false`'tur ve overload edilmiş constructor ile açılır; D yanlıştır. Thread, reentrant lock'ı `lock()` veya başarılı `tryLock()` çağrılarıyla birden fazla kez almış olabilir. Her `lock()` ve başarıyla sonuçlanan her `tryLock()` için bir `unlock()` gerekir; E yanlıştır. Hiçbir diğer ifade geçerli olmadığından F doğrudur.

Official Answer 18

English: 18. C, E, G. A Callable lambda expression takes no values and returns a generic type; therefore, options C, E, and G are correct. Options A and F are incorrect because they both take an input parameter. Option B is incorrect because it does not return a value. Option D is not a valid lambda expression, because it is missing a semicolon at the end of the return statement, which is required when inside braces {}.

Türkçe: 18. C, E, G. Callable lambda ifadesi parametre almaz ve generic bir tür döndürür; bu nedenle, C, E ve G seçenekleri doğrudur. A ve F seçenekleri yanlıştır çünkü her ikisi de bir giriş parametresi alır. B seçeneği yanlış çünkü bir değer döndürmez. D seçeneği geçerli bir lambda ifadesi değildir; braces ({}) içindeki `return` statement'ın sonunda gerekli semicolon (;) eksiktir.

Official Answer 19

English: 19. E, G. The application compiles and does not throw an exception. Even though the stream is processed in sequential order, the tasks are submitted to a thread executor, which may complete the tasks in any order. Therefore, the output cannot be determined ahead of time, and option E is correct. Finally, the thread executor is never shut down; therefore, the code will run but never terminate, making option G also correct.

Türkçe: 19. E, G. Uygulama derlenir ve bir exception atmaz. stream sıralı olarak işlense de, görevler herhangi bir sırada görevleri tamamlayabilecek bir thread yöneticiye gönderilir. Bu nedenle, çıktı önceden belirlenemez ve E seçeneği doğrudur. Son olarak, thread yürütücüsü asla kapanmaz; bu nedenle, kod çalışacak ancak asla sonlandırmayacak ve G seçeneğini de doğru hale getirecektir.

Official Answer 20

English: 20. F. The key to solving this question is to remember that the `execute()` method returns `void`, not a `Future` object. Therefore, line n1 does not compile, and option F is the correct answer. If the `submit()` method had been used instead of `execute()`, option C would have been the correct answer, as the output of the `submit(Runnable)` task is a `Future<?>` object that can only return `null` on its `get()` method.

Türkçe: 20. F. Temel kural şudur: `execute()` bir `Future` değil, `void` döndürür. Bu nedenle line n1 derlenmez ve F doğrudur. `execute()` yerine `submit()` kullanılsaydı C doğru olurdu; `submit(Runnable)` bir `Future<?>` döndürür ve bu future'ın başarılı `get()` sonucu `null` olur.

Official Answer 21

English: 21. A, D. The `findFirst()` method guarantees the first element in the stream will be returned, whether it is serial or parallel, making options A and D correct. While option B may consistently print 1 at runtime, the behavior of `findAny()` on a serial stream is not guaranteed, so option B is incorrect. Option C is likewise incorrect, with the output being random at runtime.

Türkçe: 21. A, D. `findFirst()`, stream serial veya parallel olsun ilk element'in dönmesini garanti eder; A ve D doğrudur. B runtime'da sürekli 1 yazdırıyor gibi görünebilse de serial stream'de bile `findAny()` davranışı garanti edilmez; B yanlıştır. C de yanlıştır; runtime output öngörülemez.

Official Answer 22

English: 22. B. The code compiles and runs without issue. The key aspect to notice in the code is that a single-thread executor is used, meaning that no task will be executed concurrently. Therefore, the results are valid and predictable, with 100 100 being the output, and option B is the correct answer. If a thread executor with more threads was used, then the `s2++` operations could overwrite each other, making the second value indeterminate at the end of the program. In this case, option C would be the correct answer.

Türkçe: 22. B. Kod derlenir ve sorunsuz çalışır. Single-thread executor kullanıldığından hiçbir task başka bir task'la concurrently yürütülmez. Sonuç öngörülebilir, output 100 100 ve B doğrudur. Birden fazla worker thread içeren executor kullanılıyorsa `s2++` operation'ları birbirinin update'ini ezebilir, ikinci değer nondeterministic olur ve C doğru sayılırdı.

Official Answer 23

English: 23. F. The code compiles without issue, so options B, C, and D are incorrect. The limit on the cyclic barrier is 10, but the stream can generate only up to 9 threads that reach the barrier; therefore, the limit can never be reached, and option F is the correct answer, making options A and E incorrect. Even if the `limit(9)` statement was changed to `limit(10)`, the program could still hang since the JVM might not allocate 10 threads to the parallel stream.

Türkçe: 23. F. Kod derlenir; dolayısıyla B, C ve D yanlıştır. `CyclicBarrier` limiti 10'dur, fakat stream yalnız dokuz element ve dolayısıyla dokuz `await()` çağrısı üretir. Limit karşılanamaz; program beklemeye devam eder ve F doğrudur. `limit(9)` değeri `limit(10)` yapılsa bile JVM parallel stream'e aynı anda on worker thread ayırmayabileceği için program yine takılı kalabilir.

Official Answer 24

English: 24. A, F. The class compiles without issue, so option A is correct. Since `getInstance()` is a static method and `sellTickets()` is an instance method, lines k1 and k4 synchronize on different objects, making option D incorrect. The class is not thread-safe because the `addTickets()` method is not synchronized, and option E is incorrect. One thread could call `sellTickets()` while another thread calls `addTickets()`, possibly resulting in bad data. Finally, option F is correct because the `getInstance()` method is synchronized.

Türkçe: 24. A, F. Class sorunsuz derlenir; A doğrudur. Static synchronized `getInstance()` `TicketManager.class` monitor'ünü, instance `sellTickets()` içindeki `synchronized(this)` ise instance monitor'ünü lock eder. Line k1 ve k4 farklı object'ler üzerinde synchronize olduğundan D yanlıştır. `addTickets()` synchronized olmadığı için class thread-safe değildir; bir thread `sellTickets()` çağırırken diğeri `addTickets()` çağırıp invalid data üretebilir. E yanlıştır. `getInstance()` synchronized olduğundan F doğrudur.

English: Since the constructor is private, this method is the only way to create an instance of `TicketManager` outside the class. The first thread to enter the method will set the instance variable, and all other threads will use the existing value. This is a singleton pattern.

Türkçe: Constructor `private` olduğundan class dışından `TicketManager` instance'ı edinmenin tek yolu bu method'dur. Method'a ilk giren thread `instance` variable'ını ayarlar; sonraki thread'ler aynı mevcut value'yu kullanır. Bu bir singleton pattern'dır.

Official Answer 25

English: 25. C, D. The code compiles and runs without issue, so options F and G are incorrect. The return type of `performCount()` is `void`, so `submit()` is interpreted as being applied to a `Runnable` expression. While `submit(Runnable)` does return a `Future<?>`, calling `get()` on it always returns `null`. For this reason, options A and B are incorrect, and option C is correct. The `performCount()` method can also throw a runtime exception, which will then be thrown by the `get()` call as an `ExecutionException`; therefore, option D is also a correct answer. Finally, it is also possible for our `performCount()` to hang indefinitely, such as with a deadlock or infinite loop. Luckily, the call to `get()` includes a timeout value.

Türkçe: 25. C, D. Kod sorunsuz bir şekilde derlenir ve çalışır, bu nedenle F ve G seçenekleri yanlıştır. `performCount()`'nin return type'isi `void`'dir, bu nedenle `submit()` bir `Runnable` ifadesine uygulandığı şeklinde yorumlanır. `submit(Runnable)` bir `Future<?>` döndürürken, üzerinde `get()` çağrısı her zaman `null` döndürür. Bu nedenle, A ve B seçenekleri yanlıştır ve C seçeneği doğrudur. `performCount()` metodu, daha sonra `get()` çağrısı tarafından `ExecutionException` olarak atılacak bir runtime exception de atabilir; bu nedenle, D seçeneği de doğru bir cevaptır. Son olarak, `performCount()` deadlock veya sonsuz döngü gibi süresiz olarak asılması da mümkündür. Neyse ki, `get()` çağrısı bir zaman aşımı değeri içerir.

English: While each call to `Future.get()` can wait up to a day for a result, it will eventually finish, so option E is incorrect.

Türkçe: `Future.get()`'a yapılan her çağrı bir sonuç için bir güne kadar bekleyebilirken, sonunda bitecektir, bu nedenle E seçeneği yanlıştır.

Java 17 teknik düzeltme: Kaynağın resmî anahtarı C ve D'yi verir; Java 17 davranışına göre E de mümkün bir sonuçtur. `Future.get(1, TimeUnit.DAYS)` yalnız bekleyen `get()` çağrısını `TimeoutException` ile sonlandırır, çalışan task'ı `cancel` etmez. `performCount()` sonsuz döngüye girer veya deadlock yaşarsa `single executor`'ın `non-daemon worker`'ı çalışmayı sürdürür; `shutdown()` da bu task'ı `interrupt` etmez. Dolayısıyla teknik olarak savunulabilir seçenek kümesi **C, D ve E**'dir. Kaynak resmî cevabı, kaynak sadakati için hemen üstte değiştirilmeden korunmuştur.

Kapsam doğrulaması

- Ana bölüm kaynak sayfaları: 721–784
- Ek cevap kaynağı sayfaları: 951–955
- Resmî cevap hedefi: 1–25
- Kod blokları özgün dilinde tutulmuştur.
- Çeviri ayrıntıları ünite vocabulary ve grammar kaynaklarıyla desteklenir.